

五项目结合式 嵌入式面试补充讲义

从 216 道基础题到五个真实代码仓库的机制、证据、取舍与验证

P1	手持体温检测仪	FreeRTOS · 融合 · CRC
P2	智能送药小车	ISR · FSM · Tick
P3	三维 LiDAR 感知	UDP · epoll · 有界队列
P4	多源光电融合	线程 · 快照 · RAII
P5	I.MX6ULL 状态监测器	DTS · SPI · IIO

学习版 · 面试版 · 实验版

证据等级：CODE / HOST / CROSS / HIL / TODO-HIL

目录

五项目结合式嵌入式面试补充讲义	6
这套讲义解决什么问题	6
五个项目	6
证据等级	6
原题分级规则	7
目录与阅读路线	7
面试回答模板	8
内容维护原则	8
一次完整的学习闭环	8
01 C/C++、内存与资源生命周期	10
1. 编译单元、链接与限定符	10
2. 内存区域、栈堆与分配	12
3. 指针、引用、所有权与 RAII	14
4. 对象模型与现代 C++	16
5. 本章概念辨析	18
6. 本章掌握度检查	18
02 ARM、中断与 FreeRTOS 实时设计	20
1. ARM 与异常基础	20
2. 实时系统与 FreeRTOS	22
3. 错误演化案例	26
4. 本章掌握度检查	27
03 Linux 进程、线程、同步与虚拟内存	28
1. 系统调用、进程与线程	28
2. 同步、数据竞争与快照	29
3. 线程生命周期与退出协议	31
4. 虚拟内存与地址	32
5. 用户态故障定位	33
6. 本章概念辨析	35
7. 本章掌握度检查	35
04 Linux I/O、事件循环与网络	36
1. 网络分层与项目边界	36
2. TCP、UDP 与实时传感器数据	36
3. Socket 生命周期	38
4. I/O 多路复用	38
5. 过载、重连与离线输入	41

6. 数据结构与算法边界（原题 140~159）	42
7. 本章掌握度检查	43
05 Linux 内核、设备模型与 IIO 驱动	44
1. 内核职责与上下文	44
2. 中断上下半部与内核同步	45
3. 模块、设备模型与设备树	46
4. SPI、regmap 与 IIO	48
5. 用户态算法与内核边界	51
6. P5 上板时的真实验证链	52
7. 本章掌握度检查	52
06 五个项目怎样讲成可信的工程故事	53
P1 手持体温检测仪	53
P2 智能送药小车	54
P3 三维 LiDAR 感知	55
P4 多源光电数据融合	57
P5 I. MX6ULL 设备姿态与异常冲击监测器	58
五项目横向对比	60
面试时的真实性规则	60
07 原 216 题分级映射与双向索引	61
使用说明	61
一、C/C++（1~71）	61
二、硬件、ARM 与 FreeRTOS（72~99）	64
三、操作系统、网络与算法（100~159）	65
四、嵌入式 Linux（160~216）	67
A 级覆盖统计	69
按项目反查	69
08 40 个核心问题的三层追问树	70
1. volatile 能保证线程安全吗？	70
2. const 和不可变对象是同一回事吗？	70
3. RAII 为什么不等于智能指针？	70
4. shared_ptr 为什么不是默认选择？	70
5. move 到底移动了什么？	71
6. 为什么不能把协议字节直接强转成结构体？	71
7. ISR 为什么越短越好？	71
8. Cortex-M 的 SVC、PendSV、SysTick 分别做什么？	71
9. Queue、Semaphore、Mutex 怎么选？	72
10. 优先级反转怎样发生？	72
11. 为什么长度 1 队列有价值？	72

12. 为什么“加大队列”不能解决持续过载？	72
13. 非阻塞 FSM 比 delay 好在哪？	72
14. Tick 回绕为什么用无符号减法？	73
15. 超时、重试、退避、看门狗有什么区别？	73
16. 进程和线程的真正区别是什么？	73
17. 普通 bool 停线程为什么是 UB？	73
18. Atomic 能保护结构体快照吗？	74
19. Condition variable 为什么必须有谓词？	74
20. detach 有什么实际风险？	74
21. 怎样证明线程能优雅退出？	74
22. 死锁和优先级反转如何区分？	74
23. fd 是文件吗？	75
24. epoll 为什么适合 P5？	75
25. epoll 一定比 poll 快吗？	75
26. ET 为什么必须读到 EAGAIN？	75
27. 一次 read 为什么不等于一帧？	76
28. TCP 和 UDP 在 LiDAR 场景怎么选？	76
29. 网络字节序只是 ntohs 吗？	76
30. 为什么需要指数退避？	76
31. 设备树怎样触发 probe？	76
32. probe 为什么必须 fail-fast？	77
33. regmap 解决什么问题？	77
34. IIO 与自定义字符设备怎么选？	77
35. hrtimer 中为什么不能读 SPI？	77
36. 自旋锁与 mutex 怎样选？	78
37. IIO scan 为什么有 padding？	78
38. sysfs 为什么不能写死 device0？	78
39. 交叉编译通过能证明什么？	78
40. 为什么算法不放内核？	78
不会时的统一收口	79
实验手册：把回答变成可复现证据	80
1. 为什么面试准备必须包含实验	80
2. P1：融合条件与 Flash CRC 边界	80
3. P2：非阻塞状态机与 Tick 回绕	80
4. P3：有界队列、过载策略与优雅退出	81
5. P4：重连退避、快照与生命周期	82
6. P5：领域算法、回放与故障状态机	82
7. P5：伪 sysfs 的设备发现与扫描配置	83

8. P5: epoll、timerfd、signalfd 与半帧解析	83
9. P5: ARM 用户态交叉编译	84
10. P5: vendor 4.1.15 驱动对象编译	84
11. fd 泄漏与错误路径观察	84
12. Sanitizer 与并发诊断的适用边界	85
13. 实板 HIL 闭环: 尚未通过的项目	85
14. 实验复盘模板	86
面试速记卡与掌握度检查	87
1. 使用方式	87
2. C/C++ 与内存	87
3. ARM、ISR 与 FreeRTOS	88
4. Linux 进程、线程与同步	89
5. Linux I/O 与网络	90
6. Linux 驱动与 IIO	91
7. 高频辨析表	92
8. 五项目 30 秒卡	92
9. 追问失分警报	93
10. 掌握度清单	93
11. 随机抽测规则	94
术语表: 用准确语言回答	95
1. C/C++ 与运行时	95
2. ARM、实时与 FreeRTOS	96
3. Linux 进程、线程与内存	97
4. I/O 与网络	98
5. Linux 驱动、设备树与 IIO	99
6. 项目和证据术语	100
7. 容易说错的完整句式	100

五项目结合式嵌入式面试补充讲义

这套讲义解决什么问题

这不是另一份孤立的“八股答案合集”，而是《嵌入式八股-赛博二哈》精华版的项目化补充说明。原资料负责给出题目范围，本讲义负责回答四个更难的问题：

1. 这个概念为什么存在，解决了什么工程问题？
2. 它在代码运行过程中到底怎样发生？
3. 五个项目里哪里真正用到了它？
4. 面试时怎样既说清楚，又不把没有做过的事情包装成项目经验？

默认读者已经会写基本 C/C++，使用过 STM32 和 Linux 命令，但对内存模型、并发、I/O 与驱动执行上下文缺少系统联系。首次出现的关键术语会解释；后续章节会直接使用。

五个项目

代号	项目	主要证据域	GitHub
P1	手持体温检测仪	FreeRTOS、队列、mutex、临界区、Flash/CRC	当前代码
P2	智能送药小车	ISR 边界、最新值邮箱、非阻塞 FSM、Tick 回绕	当前代码
P3	三维 LiDAR 感知	UDP、epoll、有界队列、条件变量、线程退出	当前代码
P4	多源光电数据融合	ROS/Qt 线程、atomic/mutex、快照、RAII、重连	当前代码
P5	I.MX6ULL 状态监测器	DTS、SPI、regmap、IIO、hrtimer、epoll 服务	当前代码

证据等级

正文只使用以下五种证据标识：

标识	含义	能证明什么	不能证明什么
CODE	已存在于仓库	设计已经落到源码	目标硬件一定正常
HOST	宿主测试通过	纯算法、状态机或并发策略满足测试	外设时序与实时性
CROSS	目标工具链通过	目标架构和指定 API 能编译	模块可加载、板端运行稳定
HIL	硬件在环通过	指定硬件、版本和实验条件下结果成立	所有环境和长期量产可靠性
TODO-HIL	待实板验证	边界已明确	不得在简历中写成“已验证”

此外，RELATED 表示知识与项目有关，但当前代码并未实现该机制；BASE 表示纯基础知识。它们不是较低级的 CODE，而是不同性质的内容。

原题分级规则

- A级：项目中存在真实代码证据。重点题使用“九段式”：原题、结论、从零解释、机制、项目场景、代码证据、设计取舍、追问误区、30 秒/2 分钟回答。
- B级：与项目相邻，正文解释如何迁移到项目，但明确说明没有直接实现。
- C级：通用算法、Web 或体系结构基础，不强行绑定项目，给出正确回答和学习边界。

完整的 216 题分级和双向索引见 [07-question-mapping.md](#)。

目录与阅读路线

文件	内容	建议掌握结果
01-c-cpp-memory.md	C/C++、内存、对象生命周期、RAII、C++11	能解释“对象和资源何时有效”
02-arm-freertos.md	ARM、中断、FreeRTOS、实时设计	能画出 ISR 到任务的数据流
03-linux-process-thread.md	进程、线程、同步、虚拟内存	能定位数据竞争与退出问题
04-linux-io-network.md	fd、epoll、socket、TCP/UDP	能解释事件循环与流式解析
05-linux-driver.md	驱动模型、DTS、SPI、IIO、内核上下文	能从设备树讲到用户态采样
06-project-stories.md	五项目 30 秒/2 分钟/深挖故事	能把八股自然带回项目
07-question-mapping.md	原 216 题双向映射	能从原题快速找到补充讲解
08-follow-up-tree.md	高频题三层追问树	能承受连续追问而不背串
09-experiments.md	可运行实验、失败注入与恢复	能用输出证明理解
10-flashcards.md	速记卡、辨析表、掌握度检查	面试前快速复盘
11-glossary.md	中英文术语、缩写和边界	表达更准确

阅读不按天数安排，而按依赖关系推进：

```

语言与资源所有权
  ↓
任务/线程与同步
  ↓
实时边界与过载策略
  ↓
Linux fd 与事件循环
  ↓
设备树 → 驱动 → IIO → 用户服务
  ↓
项目故事、追问树与实验复现
  
```

每一层达到以下五级才算真正掌握：

1. 能定义：不用术语循环解释术语。
2. 能画图：画出对象、线程或数据的流向。

3. 能指代码：说出仓库、文件和函数。
4. 能跑实验：知道命令、预期结果和失败表现。
5. 能扛追问：能说明取舍、反例和验证边界。

面试回答模板

30 秒版本

定义：它解决什么问题。
 场景：我在哪个项目遇到。
 实现：具体使用了哪个机制和函数。
 取舍：为什么这样选。
 边界：什么验证过，什么还没验证。

2 分钟版本

背景 → 原问题会造成什么后果
 机制 → 执行时发生什么
 实现 → 数据流、线程/中断边界、关键函数
 取舍 → 与替代方案比较
 验证 → 测试、交叉编译、HIL 分级
 复盘 → 如果继续迭代会怎样做

不会时的诚实收口

可以说：

这个机制我没有在当前项目中直接实现。我的项目使用的是……，二者共同点是……，区别是……。如果要迁移，我会先验证……，所以我不会把它描述成现有项目成果。

不要用模糊的“应该差不多”“底层帮我做了”掩盖边界。准确知道自己没有做过什么，本身就是工程能力。

内容维护原则

1. 原资料只作为题目范围，不复制其大段答案。
2. `main` 链接用于日常浏览，`study-step-*` 标签用于固定证据。
3. 新增项目能力时先补测试和证据，再升级题目等级。
4. `HOST/CROSS` 不得改写成 `HIL`。
5. 业务参数、硬件阈值和性能数字没有实验记录就不写确定结论。

一次完整的学习闭环

不要以“看完章节”作为结束条件。选一道 A 级题后，先合上文档用自己的语言回答 30 秒；接着在纸上画出数据流、线程或中断边界；然后打开仓库找到对应文件与函数，实际运行相关测试；最后从追问树中随机选三个问题继续回答。任何一步卡住，都回到机制或实验，而不是继续背更长的句子。

建议为每道重点题维护一条简短复盘记录：第一次错误答案、造成错误的概念混淆、代码证据、失败注入结果、当前验证等级。这样能看到自己从“术语记忆”到“工程判断”的变化，也能避免几周后只记得标准答案却忘了为什么。

当代码发生变化时，按下面顺序维护资料：

1. 修改实现并增加能失败的回归测试。
2. 更新项目自己的 `INTERVIEW_EVIDENCE.md`，写清能力和边界。
3. 更新本讲义的项目故事、相关 A 级题与实验。
4. 检查 216 题映射是否仍然唯一、完整。
5. 重新生成 PDF，逐页抽查目录、表格、代码块和链接。
6. 只有获得真实板端记录后，才把 `TODO-HIL` 升级为 `HIL`。

最终目标不是把五个项目说得“无所不能”，而是形成稳定的推理路径：先判断问题属于对象生命周期、并发同步、实时边界、I/O 状态机还是驱动上下文；再选最贴近的项目证据；最后主动说明取舍和验证范围。这样即使面试官换一种问法，也能从机制重新推导，而不是等待命中背过的题目。

01 | C/C++、内存与资源生命周期

本章对应原题 1~71。A 级题不追求背完标准条款，而是建立一条主线：对象存在哪里、谁拥有它、谁可以并发访问、什么时候失效、失败路径怎样释放资源。

1. 编译单元、链接与限定符

A01 | 原题 4：什么时候使用 `const`？

- 一句话结论：`const` 是接口契约，限制“通过当前表达式修改对象”，但不自动解决并发同步，也不保证底层对象永远不变。
- 从零解释：`const int *p` 表示不能通过 `p` 修改所指值；`int * const p` 表示指针本身不能改指向；成员函数后的 `const` 表示不能通过该函数修改非 `mutable` 成员。
- 底层机制：编译器据类型系统拒绝非法写入；常量是否放进只读段取决于存储期、链接和实现。用强制转换写入本来就是 `const` 的对象会产生未定义行为。
- 项目场景：P5 的 `Monitor::update(const Sample& sample)` 用只读引用表达输入样本不会被算法修改；P3 的队列查询和 P4 的 `latest()` 也用 `const` 成员函数表达只读快照。
- 代码证据：`libcondition/src/condition_monitor.cpp::Monitor::update`，`src/mainwindow/include/gnss_output.h::latest`。证据：CODE、HOST。
- 工程取舍：传大对象使用 `const&` 避免复制；小标量直接传值通常更简单。不要为了“性能”把所有参数都改成引用。
- 常见误区：把 `const` 当作线程安全；返回指向内部可变缓存的 `const` 指针仍可能因其他线程修改而失效。
- 30 秒回答：`const` 主要表达接口只读约束。项目中采样输入和快照读取使用 `const` 引用或 `const` 成员函数，既避免无谓复制，也让调用者知道函数不会经该接口修改对象；但并发一致性仍由 `mutex/atomic` 保证。
- 2 分钟展开：继续说明顶层/底层 `const`、`const` 成员函数、逻辑 `const`，以及为什么 `mutex` 常声明为 `mutable` 以允许 `const` 查询函数加锁。

B01 | 原题 1~3：`static`、`extern`、`extern "C"`

`static` 在文件作用域提供内部链接，在函数局部让对象拥有静态存储期，在类中表示成员属于类型。`extern` 只声明符号在其他翻译单元定义，不分配新的定义；`extern "C"` 改变语言链接，常用于 C++ 调用 C 驱动接口，主要解决名字改编，不等于“按 C 语法编译函数体”。

五项目中大量 STM32 BSP 头文件采用 C 接口，Linux C++ 应用链接 C 系统调用和第三方 C 库。当前项目没有专门封装跨语言 ABI，因此标为 **RELATED**，不要说成自己设计了 C ABI。

A02 | 原题 6、7: `volatile` 的作用与正确用法

- 一句话结论: `volatile` 要求对该对象的访问按抽象机可观察规则发生, 适合内存映射寄存器; 它不提供原子性、互斥或线程间 `happens-before`。
- 从零解释: 硬件可以在程序不知道的時刻改变寄存器。若编译器把读取缓存到普通寄存器, 循环可能永远看不到变化, 因此寄存器声明需要 `volatile`。
- 底层机制: `volatile` 约束编译器优化, 不会自动生成锁, 也不能保证 `counter++` 不被打断。CPU 缓存一致性和 C++ 内存序是另一层问题。
- 项目场景: P1/P2 通过 CMSIS 的 `volatile` 寄存器定义访问 GPIO、USART; P4 的跨线程停止标志使用 `std::atomic<bool>`, 而不是 `volatile`。
- 代码证据: STM32 `Libraries/CMSIS` 寄存器类型; P4 `src/mainwindow/include/qnode.h` 中 `atomic` 标志。证据: CODE。
- 工程取舍: 硬件寄存器用 `volatile`; 线程通信用 `atomic/mutex`; ISR 与任务共享复合数据优先用 RTOS 队列, 而不是裸全局变量。
- 常见误区: 说“`volatile` 每次从内存读, 所以线程安全”; 在单核 MCU 上误以为禁止编译器优化就等于不可中断。
- 30 秒回答: `volatile` 解决编译器不能假设对象不变的问题, 典型是硬件寄存器。它不解决竞争, 所以多线程标志在光电项目里用 `atomic`, 复合结果用 `mutex`; STM32 ISR 到任务的数据用 `FromISR` 队列传递。
- 2 分钟展开: 区分可见性、原子性和有序性, 并举 `volatile uint32_t x; x++` 仍是读一改一写三个阶段的例子。

A03 | 原题 14、50: 绝对地址和硬件寄存器访问

- 一句话结论: 把固定地址转换成指向正确宽度的 `volatile` 指针, 再通过受控接口访问; 地址、宽度、对齐和访问属性都必须来自芯片手册。
- 从零解释: MCU 外设寄存器映射到地址空间, 读写该地址会触发外设行为, 而不是访问普通 RAM。
- 底层机制: 总线矩阵把地址译码到外设; 访问宽度错误可能触发总线错误或误写相邻寄存器。读清零、写 1 清零等语义还要求按手册操作。
- 项目场景: P1/P2 使用 STM32 标准外设库/CMSIS, 不在业务层散落魔法地址; P5 使用 `regmap` 把 SPI 寄存器协议集中封装。
- 代码证据: STM32 `Libraries/CMSIS`, P5 `kernel/icm20608_iio.c::icm20608_regmap_config`。证据: CODE。
- 工程取舍: 优先使用芯片厂商定义和驱动框架; 只有启动代码或极底层 BSP 才直接操作绝对地址。
- 常见误区: 遗漏 `volatile`、使用错误整数宽度、忽略对齐、把 CPU 内存映射 I/O 和 SPI 芯片内部寄存器混为一谈。

- 30 秒回答：MCU 的 MMIO 可用指向固定地址的 `volatile` 指针访问，但项目里优先用 CMSIS；SPI IMU 不是 CPU 直接寻址，必须经 SPI 控制器，驱动中用 `regmap` 表达寄存器和值宽度。
- 2 分钟展开：比较 MMIO、port I/O 和外部总线寄存器，并解释为什么 Linux 用户态不能随意解引用物理地址。

2. 内存区域、栈堆与分配

A04 | 原题 21：C/C++ 程序的内存分区

- 一句话结论：常见进程映像包含代码、只读数据、已初始化数据、BSS、堆、映射区和各线程栈；MCU 链接脚本则把这些段映射到 Flash/RAM。
- 从零解释：源代码中的对象根据存储期和链接属性进入不同段。局部自动对象通常在栈，动态对象来自堆，但优化后未必真的落内存。
- 底层机制：编译器产生段，链接器布局，加载器或启动代码把数据搬到目标位置并清零 BSS。
- 项目场景：P1/P2 的任务栈和 FreeRTOS heap 都占有限 SRAM；P3~P5 的 Linux 进程由虚拟内存管理，线程各有栈但共享堆。
- 代码证据：P1/P2 `FreeRTOSConfig.h`、`heap_4.c` 和任务创建栈深；P5 CMake 产物经 ARM ELF 检查。证据：CODE、CROSS。
- 工程取舍：MCU 避免大局部数组和频繁动态分配；Linux 仍要明确所有权，但不必把堆一概视为禁用。
- 常见误区：把“代码段只读”说成所有 `const` 都在代码段；把线程栈和进程栈当成同一块。
- 30 秒回答：MCU 更关注 Flash/RAM 段和任务栈预算，Linux 更关注虚拟地址空间与每线程栈。五项目中 RTOS 创建后立即断言，Linux 侧用 `RAII` 管理动态资源。
- 2 分钟展开：画出 `.text/.rodata/.data/.bss/heap/stack`，再说明 ELF 与链接脚本/加载器的关系。

A05 | 原题 22、23：堆、栈和大局部数组

- 一句话结论：栈分配快且自动回收，但容量有限；大局部数组会直接消耗当前线程/任务栈，在 MCU 上尤其危险。
- 从零解释：每次函数调用建立栈帧，局部数组的大小会计入栈帧。FreeRTOS 每个任务有独立栈，溢出可能破坏相邻内存。
- 底层机制：栈指针移动完成分配；没有逐对象边界检查时，越界可能直到异常或数据损坏才暴露。
- 项目场景：P1/P2 开启 `level-2` 栈溢出 hook，并为任务指定栈深；P5 的 `II0 scan` 是固定小对象，点云等大对象不放线程栈。
- 代码证据：P1 `Core/main.c::vApplicationStackOverflowHook`、`FreeRTOSConfig.h`；P2 任务创建。证据：CODE。

- 工程取舍：固定小缓冲可放栈；大或生命周期长的缓冲使用静态区、专用内存池或受所有权管理的堆，并做上限。
- 常见误区：认为“2048 字节一定不大”；真正标准取决于任务栈预算、调用深度、中断嵌套和 ABI。
- 30 秒回答：大局部数组是否安全取决于当前栈预算。STM32 每个 FreeRTOS 任务栈很小，所以我启用栈溢出检查，并避免在深调用链创建大缓冲。
- 2 分钟展开：说明高水位监控、最坏调用深度和中断栈的区别。

A06 | 原题 25、29: malloc/free 与 new/delete

- 一句话结论：malloc 管理原始字节，new 同时分配并构造对象；delete 析构并释放。二者不可交叉配对。
- 从零解释：C++ 对象可能有构造不变量和析构动作，因此仅得到一块内存并不等于对象已经建立。
- 底层机制：new 表达式通常调用 `operator new` 后执行构造；delete 先执行析构再调用 `operator delete`。数组版本还要正确处理多个元素。
- 项目场景：P3/P4 的 C++ 对象交由容器、栈对象或明确智能指针管理；P5 的 fd 不用 new，而用值类型 RAII 包装。
- 代码证据：P5 `apps/conditiond.cpp::UniqueFd`；P4 各线程对象的析构/stop。证据：CODE。
- 工程取舍：优先值语义与 `make_unique`；只有确需共享生命周期才使用 `shared_ptr`。MCU 实时路径避免不可控分配。
- 常见误区：`free(new T)`、`delete malloc()`、忘记虚析构、把 placement new 当普通分配。
- 30 秒回答：malloc/free 只处理字节，new/delete 还处理对象生命周期。项目中我尽量用值语义和 RAII，让异常路径不需要手写多套释放代码。
- 2 分钟展开：补充失败行为、`nothrow`、`operator new` 可重载以及数组 delete 的配对要求。

A07 | 原题 32: 内存泄漏如何检测与避免

- 一句话结论：泄漏的根因是所有权或退出路径不清，而不只是“忘记 free”；避免优先于事后工具检测。
- 从零解释：进程仍持有无法再使用的内存、fd、线程或内核对象，都属于资源泄漏的工程问题。
- 底层机制：异常、早返回、构造一半失败和线程未 join 是最常见的分支遗漏。
- 项目场景：P3 构造 socket/epoll 任何一步失败都关闭已取得 fd；P4 停止线程后 join；P5 `UniqueFd`、`BufferGuard` 和内核 `devm` 管理不同资源。
- 代码证据：P3 `src/lslidar_ls_driver/src/input.cc::InputSocket`；P5 `apps/conditiond.cpp`、`kernel/icm20608_iio.c::probe`。证据：CODE。

- 工程取舍：RAII 解决结构性泄漏；Valgrind/ASan、`/proc/<pid>/fd` 和长期监控负责发现遗漏。
- 常见误区：只盯堆内存，忽略 fd、线程、订阅者和设备 buffer 状态。
- 30 秒回答：我把泄漏看成所有权问题。LiDAR 的 fd 失败路径显式回收，驱动服务用 RAII，光电线程必须 `stop+join`；工具检测是第二道防线。
- 2 分钟展开：描述如何用重复连接/断开测试、fd 数量曲线和 sanitizer 定位增长点。

A08 | 原题 33：字节对齐与 padding

- 一句话结论：对齐让 CPU 以支持的边界访问对象，编译器可能插入 padding；二进制协议不能直接假设 C 结构布局等于线上的字节布局。
- 从零解释：不同成员有不同对齐要求，结构大小通常补齐到最大成员对齐的倍数。
- 底层机制：未对齐访问可能变慢或触发异常；ABI 决定类型布局，但编译选项和架构可能不同。
- 项目场景：P5 七个 16 位 IIO 通道为 14 字节，时间戳要求 8 字节对齐，因此 scan 中存在 padding，push 前必须清零。
- 代码证据：P5 `kernel/icm20608_iio.c` 的 scan 结构与 `memset`。证据：CODE、CROSS。
- 工程取舍：协议解析使用逐字段或 `memcpy`，不用 `#pragma pack` 粗暴覆盖所有结构；必要时加 `static_assert(sizeof(...))`。
- 常见误区：只会算 `sizeof`，不理解 padding 可能泄露未初始化内核内存。
- 30 秒回答：IIO scan 的时间戳对齐会产生填充字节，我在驱动 push 前清零整个 scan，不仅保证布局，也避免把内核未初始化数据送到用户态。
- 2 分钟展开：画出偏移 0~13 的通道、14~15 padding、16~23 时间戳。

3. 指针、引用、所有权与 RAII

A09 | 原题 40~43：指针、引用和传参

- 一句话结论：指针适合表达可空、可改指向和底层地址；引用适合表达必需别名。值传递强调独立副本，`const` 引用强调只读借用。
- 从零解释：语法差异背后真正重要的是接口语义，而不是笼统说“引用效率高”。小标量值传递往往最好。
- 底层机制：实现上引用通常通过地址传递，但语言规则不允许把引用当可空指针随意操作。
- 项目场景：P5 `Monitor::update(const Sample&)` 表达必需只读输入；`UniqueFd` 用值类型转移所有权；可选输出才使用指针/`null`。
- 代码证据：P5 `libcondition` 与 `apps/conditiond.cpp`。证据：CODE、HOST。
- 工程取舍：是否使用引用由所有权和可选性决定，不用“底层都是指针”抹平语义。
- 常见误区：返回局部对象引用、保存超出调用期的借用、把 `const` 引用误认为深层不可变。

- 30 秒回答：项目接口用引用表达必需借用，用指针表达可选或底层句柄，用值/移动表达所有权转移。选择首先服务语义，其次才是复制成本。
- 2 分钟展开：比较 `T`、`const T&`、`T*`、`unique_ptr<T>` 四种接口对调用者的承诺。

A10 | 原题 44、45：野指针与悬空引用

- 一句话结论：指针指向的对象已经结束生命周期、从未初始化或被错误释放后仍被使用，就会形成悬空/野指针。
- 从零解释：地址数值可能仍然“看起来正常”，但地址对应对象已经不存在，访问属于未定义行为。
- 底层机制：释放后的内存可能被重新分配；并发线程继续访问已析构对象会把偶现故障放大。
- 项目场景：P4 原 `detached` 线程可能在 `QNode` 或设备对象析构后继续访问成员，重构为停止标志 + `join/wait`。
- 代码证据：P4 `src/mainwindow/src/qnode.cpp`、`rtsp_capture.cpp::stop_thread`。证据：CODE。
- 工程取舍：让拥有者控制线程寿命；裸观察指针不得超过拥有者；容器重分配后不保留旧迭代器。
- 常见误区：释放后置 `null` 只能保护一个指针副本，其他别名仍可能悬空。
- 30 秒回答：光电项目里最危险的不是一个明显的 `null`，而是 `detached` 线程持有对象地址。现在析构前先通知停止并 `join`，再释放资源，从生命周期上消除悬空访问。
- 2 分钟展开：补充 `use-after-free`、`double free`、迭代器失效以及 ASan 的检测方式。

A11 | 原题 46~48：智能指针与 RAII

- 一句话结论：智能指针只是 RAII 的一种；RAII 是让资源获取与对象生命周期绑定，使所有退出路径自动释放。
- 从零解释：`unique_ptr` 表示唯一所有权，`shared_ptr` 通过引用计数共享所有权，`weak_ptr` 观察但不延长寿命。
- 底层机制：析构在作用域退出时确定执行，包括异常展开；`shared_ptr` 控制块维护强/弱引用计数。
- 项目场景：P5 用自定义 `UniqueFd` 管 `fd`、`BufferGuard` 管 `IIO enable` 状态，说明 RAII 不只管理堆内存；内核用 `devm` 做 `device` 生命周期资源管理。
- 代码证据：P5 `apps/conditiond.cpp::UniqueFd/BufferGuard`、`kernel/icm20608_iio.c::probe`。证据：CODE、HOST。
- 工程取舍：能用栈或 `unique_ptr` 就不用 `shared_ptr`；共享所有权增加计数、线程同步和环依赖风险。
- 常见误区：从同一裸指针构造两个 `shared_ptr`；在析构函数抛异常；把 RAII 等同于“智能指针”。

- 30 秒回答：项目中 fd 和设备 buffer 都不是内存，但同样用 RAII 管理。`UniqueFd` 不可复制可移动，析构关闭 fd；`BufferGuard` 保证异常退出也关闭 buffer。
- 2 分钟展开：说明控制块、循环引用、weak_ptr 和自定义 deleter，并对比内核 devm。

4. 对象模型与现代 C++

A12 | 原题 54、55、58、59：构造、复制、移动和深浅拷贝

- 一句话结论：拥有资源的类型必须明确复制和移动语义；默认浅拷贝可能导致两个对象释放同一资源。
- 从零解释：构造建立不变量，复制产生独立逻辑对象，移动转移资源，析构结束生命周期。
- 底层机制：编译器可生成特殊成员函数，但一旦类型直接拥有 fd/指针等资源，默认逐成员复制往往错误。
- 项目场景：P5 `UniqueFd` 删除复制构造/赋值，实现移动并把源 fd 设为 -1；因此一个 fd 始终只有一个关闭责任人。
- 代码证据：P5 `apps/conditiond.cpp:24-42`。证据：`CODE`、`HOST`。
- 工程取舍：优先 Rule of Zero，让标准容器/智能指针拥有资源；自定义轻量句柄时才实现 Rule of Five。
- 常见误区：把移动说成一定零成本；移动后对象仍必须可析构但值未指定。
- 30 秒回答：`UniqueFd` 明确禁止复制、允许移动，移动后源对象置无效，避免二次 close。这是深浅拷贝和 Rule of Five 在真实资源上的落地。
- 2 分钟展开：说明复制省略、noexcept 对容器移动的影响和自赋值处理。

B02 | 原题 51~67：面向对象、多态与虚函数

五个项目更多使用组合、接口和 RAII，而不是复杂继承。P3 的输入源接口体现运行时多态，P5 则刻意用小型值类型避免继承。需要掌握 vptr/vtable、override、虚析构和对象切片，但不要声称为项目设计重点。

若基类通过基类指针删除派生对象，析构必须为 virtual，否则派生资源可能不释放。构造阶段对象尚未形成完整派生类型，因此构造函数不能 virtual；构造/析构内调用虚函数也不会按普通运行期多态工作。

A13 | 原题 68：容器选择与失效规则

- 一句话结论：容器选择由访问模式、稳定性、容量上界和内存局部性决定，不是背复杂度表。
- 从零解释：vector 连续、缓存友好但扩容使指针/迭代器失效；deque 分段；list 节点稳定但局部性差；unordered_map 平均常数查找但有哈希和重哈希成本。
- 底层机制：底层布局决定分配次数、cache miss 和失效规则。
- 项目场景：P3 使用 deque/有界队列表达 FIFO 和容量；P5 使用 `std::array` 接收固定数量 epoll 事件和 scan 帧。

- 代码证据: P3 `core/bounded_queue.h`, P5 `apps/conditiond.cpp`。证据: CODE、HOST。
- 工程取舍: 实时链路首先设置容量上界; 固定大小优先 array; 不要为 0(1) 查找盲目引入 `unordered_map`。
- 常见误区: 只报平均复杂度, 不讲最坏情况、迭代器失效与分配行为。
- 30 秒回答: 我在 LiDAR 队列里先确定有界和过载语义, 再选择容器; 在 `epoll` 事件数组里数量固定, 用 array 避免动态分配。
- 2 分钟展开: 把访问模式、插入位置、稳定性和缓存局部性逐项比较。

A14 | 原题 69: 项目里真正使用的 C++11 特性

- 一句话结论: C++11 的价值不是罗列关键字, 而是更准确表达所有权、并发和类型约束。
- 从零解释: 项目最相关的是 `move`、`unique_ptr`、`atomic`、`thread`、`mutex`、`condition_variable`、`lambda`、`chrono`、`enum class`、`nullptr`、`override`。
- 底层机制: 这些特性分别把过去靠约定的资源转移、线程同步和类型边界变成编译器可检查的结构。
- 项目场景: P3 用 `mutex/condition_variable` 管有界队列; P4 用 `atomic` 和线程; P5 用移动型 `UniqueFd`、`chrono` 和 `lambda`/容器。
- 代码证据: P3 `core/bounded_queue.h`, P4 `qnode.h`, P5 `conditiond.cpp`。证据: CODE、HOST。
- 工程取舍: P5 板端停在 C++14, 因为 Linaro GCC 4.9 不完整支持目标 C++17 库; 标准选择服从部署工具链。
- 常见误区: 把 C++11 答成关键字清单; 认为用了智能指针就没有生命周期问题。
- 30 秒回答: 我主要用 C++11 的 `move/RAll`、`atomic`、`mutex/condition_variable` 和 `chrono`。它们分别解决资源所有权、跨线程同步和时间表达, 不是为“语法新”而使用。
- 2 分钟展开: 逐个关联 P3/P4/P5, 并说明为什么没有在板端强推 `filesystem`。

A15 | 原题 70: 左值、右值与移动语义

- 一句话结论: 右值引用允许从即将销毁的对象转移资源; 命名后的右值引用变量本身仍是左值。
- 从零解释: 左值通常有可识别身份, 右值通常是临时结果。`std::move` 只是类型转换, 不执行搬运。
- 底层机制: 移动构造读取源对象资源句柄并把源对象置为可析构状态。
- 项目场景: P5 `UniqueFd(UniqueFd&&)` 转移 `fd`; 若没有移动语义, 就只能传裸 `int` 或冒险复制关闭责任。
- 代码证据: P5 `apps/conditiond.cpp::UniqueFd`。证据: CODE。
- 工程取舍: 移动类型应尽量 `noexcept`; 不是所有小对象都值得显式移动。

- 常见误区：return 时到处写 `std::move` 反而阻止返回值优化；使用被移动对象的旧值。
- 30 秒回答：`std::move` 不移动资源，只把表达式转成可移动类别。真正转移发生在 `UniqueFd` 移动构造里，源 `fd` 被设为 `-1`。
- 2 分钟展开：解释完美转发不是本项目重点，并区分 `universal reference` 与普通右值引用。

A16 | 原题 71: Lambda 表达式

- 一句话结论：lambda 是匿名函数对象，关键不是语法短，而是捕获列表决定它持有什么状态和生命周期风险。
- 从零解释：`[&]` 按引用捕获，`[=]` 按值捕获；显式捕获更容易审查。
- 底层机制：编译器生成闭包类型，捕获成员成为对象字段；引用捕获不会延长外部对象寿命。
- 项目场景：异步线程或回调若按引用捕获短生命周期对象，会形成与 `detached` 线程类似的悬空风险。P4 的修复原则同样适用于 lambda 回调。
- 代码证据：P4 生命周期修复是风险证据；当前核心代码没有把复杂 lambda 作为主要机制，因此同时标记 `RELATED`。
- 工程取舍：同步短调用可简化；跨线程回调优先显式捕获稳定所有者或值，不使用宽泛 `[&]`。
- 常见误区：认为按值捕获后成员对象一定深拷贝；捕获 `this` 只复制指针。
- 30 秒回答：lambda 的核心是闭包和捕获生命周期。异步场景我避免默认引用捕获，尤其不能让回调保存已经析构对象的 `this` 指针。
- 2 分钟展开：补充 `mutable`、泛型 lambda 和与 `std::function` 类型擦除的开销区别。

5. 本章概念辨析

容易混淆	正确边界	项目证据
const vs immutable	const 限制当前接口；线程安全和深层不可变另算	P4 快照仍需 mutex
volatile vs atomic	前者约束编译器访问，后者提供并发原子语义	STM32 寄存器 vs P4 标志
malloc vs new	原始字节 vs 分配并构造对象	Linux C++ 资源优先 RAII
pointer vs reference	可空/可改指向 vs 必需别名	P5 Sample const&
smart pointer vs RAII	智能指针是 RAII 的一种	UniqueFd/BufferGuard
copy vs move	复制逻辑对象 vs 转移资源	UniqueFd 禁复制可移动
structure layout vs wire format	ABI 布局不等于协议布局	I10 scan padding、网络解析

6. 本章掌握度检查

- 能否解释为什么 `volatile bool stop` 仍然不是正确的线程退出协议？

- 能否画出 **UniqueFd** 移动前后的唯一所有权？
- 能否解释 **l10 scan** 为什么有 2 字节 padding，以及为什么必须清零？
- 能否说出 **P4 detached** 线程与野指针之间的因果链？
- 能否区分 **CODE/HOST/CROSS/HIL**，不把交叉编译说成实板运行？

02 | ARM、中断与 FreeRTOS 实时设计

本章对应原题 72~99，重点不是背寄存器数量，而是理解一条实时链路：硬件事件到达 → 中断快速记录 → RTOS 原语交接 → 任务处理 → 有界超时与故障恢复。

1. ARM 与异常基础

B01 | 原题 72~74：ARM 寄存器、状态寄存器与架构差异

ARM 寄存器数量必须先说明架构和工作模式，不能脱离 Cortex-M3、ARMv7-A 或 ARMv8-A 给单一数字。STM32F103 使用 Cortex-M3，异常入口由硬件自动压栈部分上下文；I.MX6ULL 使用 Cortex-A7，运行 Linux，涉及 MMU、特权级和更完整的进程上下文。

项目真正相关的是：P1/P2 依赖 Cortex-M 异常模型和 FreeRTOS 端口；P5 依赖 ARM Linux 交叉编译。ARMv8 的 AArch64、异常级别等不是当前硬件的直接实现，标记为 **RELATED**。

A01 | 原题 75、76：中断产生、向量和嵌套

- 一句话结论：中断是硬件事件打断正常控制流，处理器保存最小上下文、查向量进入 ISR，退出时恢复现场；嵌套由优先级与屏蔽状态决定。
- 从零解释：外设置位状态并向 NVIC 请求，NVIC 仲裁后处理器进入异常。向量表保存各异常入口地址。
- 底层机制：Cortex-M 硬件自动压栈 R0-R3、R12、LR、PC、xPSR；高优先级中断可以抢占低优先级中断，具体还受优先级分组影响。
- 项目场景：P2 的 USART1 接收中断读取字节并通知任务；P1 的 SVC 向量必须正确转交 FreeRTOS 端口，才能启动首任务。
- 代码证据：P2 `Core/stm32f10x_it.c::USART1_IRQHandler`；P1 `Core/stm32f10x_it.c::SVC_Handler`。证据：CODE。
- 工程取舍：ISR 只做确定、短小、不可阻塞的工作；复杂状态机留给任务。
- 常见误区：把中断向量说成“中断号”；认为函数调用也会由硬件自动保存同样上下文。
- 30 秒回答：以小车 USART 为例，外设触发 NVIC 后进入向量表对应 ISR，ISR 只收字节和发 FromISR 队列；业务判断由控制任务完成。这样减少中断占用和系统抖动。
- 2 分钟展开：画出外设标志、NVIC、自动压栈、ISR、PendSV/SVC 与任务切换关系。

A02 | 原题 77、80、83：ISR 的能力边界

- 一句话结论：中断响应快，但会增加延迟和抢占复杂度；ISR 没有普通调用者，不能依赖返回值传业务结果，也不能调用可能阻塞或不可重入的接口。
- 从零解释：ISR 在特殊上下文运行，调度器和库函数对其有额外限制。
- 底层机制：`printf` 可能持锁、动态分配或等待串口；在 ISR 中调用会造成死锁、长延迟或重入破坏。

- 项目场景：P2 ISR 不做 OLED、printf、病房匹配或电机控制，只形成两字节帧并 `xQueueOverwriteFromISR`。
- 代码证据：P2 `Core/stm32f10x_it.c`。证据：CODE。
- 工程取舍：调试中断使用环形 trace、计数器或把事件发给日志任务；异常紧急信息也要采用 ISR-safe 路径。
- 常见误区：说“ISR 完全不能有参数/返回值”而忽略 C 函数签名与硬件入口的区别；真正约束是业务不能依靠普通调用协议。
- 30 秒回答：ISR 的关键是快和可预测。小车中断只组帧、覆盖最新值队列并请求调度，所有可能阻塞或耗时的操作都移到任务。
- 2 分钟展开：说明 `pxHigherPriorityTaskWoken`、`portYIELD_FROM_ISR` 和为什么普通 RTOS API 不能在 ISR 使用。

A03 | 原题 78、79：软中断与函数调用的区别

- 一句话结论：软中断/软件触发异常改变执行上下文和优先级，由异常机制调度；普通函数调用仍在当前上下文和调用栈连续执行。
- 从零解释：函数调用是同步控制转移；软件异常可用于特权切换、系统服务或延迟处理。
- 底层机制：P1 的 SVC 用异常返回机制恢复首任务上下文；PendSV 通常承担 Cortex-M RTOS 上下文切换。
- 项目场景：P1 原空 `SVC_Handler` 会破坏 FreeRTOS 启动链，重构后转到 `vPortSVCHandler`。
- 代码证据：P1 `Core/stm32f10x_it.c::SVC_Handler`、FreeRTOS port。证据：CODE。
- 工程取舍：应用层不直接滥用软中断；交给 RTOS 端口实现架构相关上下文切换。
- 常见误区：把 Linux softirq、Cortex-M SVC 和“软件调用一个 ISR 函数”混为一谈。
- 30 秒回答：普通函数不会改变特权和异常上下文；SVC 是架构异常。体温仪必须把 SVC 交给 FreeRTOS 端口，首任务才会按异常返回恢复上下文。
- 2 分钟展开：对比 SVC、PendSV、SysTick 的职责，再说明 Linux softirq 是另一套概念。

A04 | 原题 81、82：主循环/任务与中断共享数据

- 一句话结论：volatile 只能保证访问不被省略；共享对象还需要原子性、一致性和可重入设计，最佳方案通常是消息传递。
- 从零解释：ISR 可以在任务读到一半时改写多字节对象；同一函数若使用静态缓冲或非重入外设，也可能被中断重入。
- 底层机制：Cortex-M
对某些自然对齐字宽访问是单指令，但复合操作和结构体更新仍不是原子事务。
- 项目场景：P2 用 FromISR 队列把完整视觉帧交给任务；P1 用队列传温度/RFID，而不是让任务轮询裸全局结构体。
- 代码证据：P2 `xQueueOverwriteFromISR`，P1 `prvSendLatest`。证据：CODE、HOST。

- 工程取舍：简单单字标志可用临界区/原子；数据帧优先队列；共享外设用 mutex，但 ISR 不能拿普通 mutex。
- 常见误区：给整个结构体加 volatile 就认为快照一致；在 ISR 调用使用 mutex 的 I2C 函数。
- 30 秒回答：我的项目尽量不让 ISR 与任务共享可变结构。ISR 形成完整消息，通过 FromISR 队列交接；这样同步点和数据所有权都更清楚。
- 2 分钟展开：举出撕裂读取、读改写竞争和函数重入三个不同问题。

2. 实时系统与 FreeRTOS

A05 | 原题 86：实时系统与通用 Linux 的区别

- 一句话结论：实时性的核心是截止时间和最坏响应上界，不是平均速度快；普通 Linux 优先吞吐和公平，RTOS 更强调可预测调度。
- 从零解释：硬实时错过期限即失败，软实时偶发超时只降低质量。FreeRTOS 不是自动实时，设计不当仍会长时间关中断或阻塞。
- 底层机制：抢占优先级、临界区长度、中断延迟、锁和最坏执行时间共同决定响应。
- 项目场景：P2 电机控制和视觉事件要求及时但当前没有硬截止时间测量；P3 LiDAR 关注有界延迟，通过 DropOldest 避免积压。
- 代码证据：P2 非阻塞 FSM；P3 BoundedQueue。证据：CODE、HOST，实时上界为 TODO-HIL。
- 工程取舍：用有界队列和超时建立可分析性；不把“用了 FreeRTOS”写成“满足硬实时”。
- 常见误区：把实时等同于高主频；把 Linux 一概说成不能实时。
- 30 秒回答：实时看最坏响应而不是平均性能。小车重构减少 ISR 工作和阻塞 delay，LiDAR 用丢旧控制排队延迟，但没有硬件时序测量，所以只声称结构上改善可预测性。
- 2 分钟展开：说明 PREEMPT_RT、优先级、WCET 和测量方法。

A06 | 原题 87：FreeRTOS 配置与中断优先级

- 一句话结论：必须配置时钟、最大优先级、可调用 RTOS API 的中断优先级边界、堆、栈检查和 hook；数值优先级与逻辑高低要分清。
- 从零解释：Cortex-M 数值越小优先级越高。高于 configMAX_SYSCALL_INTERRUPT_PRIORITY 的中断不能调用 FreeRTOS API。
- 底层机制：RTOS 通过 BASEPRI 屏蔽一部分中断保护内核临界区，最高紧急中断仍可响应但不得碰内核对象。
- 项目场景：P1/P2 使用 FromISR API、启用 malloc failed 和 level-2 stack overflow hook，并对创建结果断言。
- 代码证据：两个 STM32 项目的 FreeRTOSConfig.h、Core/main.c。证据：CODE。
- 工程取舍：中断优先级规划写入系统设计，不在各驱动随意设置；创建失败应在启动阶段显式停机/报警。

- 常见误区：把 library priority 和寄存器移位后的 priority 混用；所有中断都设最高。
- 30 秒回答：配置重点不仅是 tick，还包括可调用内核 API 的中断优先级边界。项目中 ISR 使用专用 API，任务/队列创建立即断言，并启用堆耗尽和栈溢出 hook。
- 2 分钟展开：解释 BASEPRI 和 configPRIO_BITS，再讲如何审查每个 NVIC priority。

A07 | 原题 88：优先级反转

- 一句话结论：高优先级任务等待低优先级任务持锁，而中优先级任务不断抢占低优先级任务，会形成优先级反转；mutex 的优先级继承只能缓解特定情况。
- 从零解释：真正阻塞高任务的并不是低任务执行时间本身，而是低任务无法获得 CPU 释放资源。
- 底层机制：优先级继承临时提升持锁者；释放 mutex 后恢复。二值信号量通常没有所有者与继承语义。
- 项目场景：P1 I2C 总线使用 mutex，临界区只复制短小共享数据，不把 RTC/I2C 轮询包进去。
- 代码证据：P1 Core/main.c 的 I2C mutex 与短临界区。证据：CODE。
- 工程取舍：减少锁持有时间、避免嵌套锁、使用单一所有者任务，通常比依赖继承更稳。
- 常见误区：把任务切换都叫优先级反转；认为提高高任务优先级能解决。
- 30 秒回答：我用 mutex 串行化 I2C，但把慢 I/O 放在临界区外。mutex 可做优先级继承，不过设计重点仍是缩短持锁时间和减少共享资源。
- 2 分钟展开：画出 L 持锁、H 阻塞、M 抢占的时间线，并区分反转、死锁和饥饿。

A08 | 原题 89：FreeRTOS 任务调度

- 一句话结论：调度器选择最高优先级就绪任务；同优先级可时间片轮转，任务通过阻塞等待事件释放 CPU。
- 从零解释：任务常见状态为运行、就绪、阻塞和挂起。延时、队列等待和信号量等待应进入阻塞态，而不是忙等。
- 底层机制：SysTick 提供时基，PendSV 完成上下文切换，SVC 启动首任务。
- 项目场景：P1 修正 SVC；P2 控制 FSM 周期执行、传感器任务有界等待；不再用长 delay 完成整个动作。
- 代码证据：P1 SVC_Handler；P2 CartState_Step。证据：CODE、HOST。
- 工程取舍：周期任务用 vTaskDelayUntil 更利于稳定周期；事件任务用阻塞队列，不轮询。
- 常见误区：认为优先级越高越实时；高优先级忙循环会饿死系统。
- 30 秒回答：FreeRTOS 每次运行最高优先级就绪任务。项目中事件通过队列唤醒，状态机周期推进；SVC/PendSV 属于端口层，不在业务里自己切上下文。
- 2 分钟展开：说明抢占、时间片、tickless 可选项和任务状态转移。

A09 | 原题 90: FreeRTOS IPC 怎么选

- 一句话结论：队列传数据，信号量传事件/计数，mutex 保护有所有者的资源，事件组表达多个布尔条件，任务通知适合轻量一对一通知。
- 从零解释：选择同步原语前先回答“传不传数据、是否有所有者、能否丢、谁唤醒谁”。
- 底层机制：队列复制固定大小 item；mutex 带所有权；通知存放在 TCB 内，开销小但耦合一对一。
- 项目场景：P1/P2 传感器使用队列，I2C 使用 mutex；“最新值”数据使用长度 1 mailbox/overwrite。
- 代码证据：P1 `prvSendLatest`，P2 `xQueueOverwrite`。证据：CODE、HOST。
- 工程取舍：关键命令不能静默 overwrite；传感器最新值可以丢旧。同步语义必须与业务语义一致。
- 常见误区：用 binary semaphore 保护资源却期待优先级继承；把全局变量和信号量分开导致数据/事件错位。
- 30 秒回答：我的选择先看语义。体温和视觉帧只关心最新值，用长度 1 队列；I2C 有明确资源所有者，用 mutex；状态机命令由单控制任务消费。
- 2 分钟展开：给出 queue/semaphore/mutex/event group/notification 决策表。

A10 | 队列满与最新值邮箱（原题 90 的项目追问）

- 一句话结论：队列容量是资源上限，满时必须显式选择阻塞、拒绝新值或丢弃旧值；最新值语义适合传感器状态，不适合交易命令。
- 从零解释：如果生产速度长期高于消费速度，无界队列只会把故障推迟成内存耗尽和高延迟。
- 底层机制：长度 1 overwrite 始终保留最近完整样本；P1 的 `prvSendLatest` 在普通队列满时先取出最旧再重试。
- 项目场景：P1 温度/RFID、P2 视觉帧/重量强调新鲜；P3 LiDAR 点云流水线使用 DropOldest 并统计 dropped。
- 代码证据：P1 `Core/main.c::prvSendLatest`；P2 `xQueueOverwrite`；P3 `BoundedQueue`。证据：CODE、HOST。
- 工程取舍：丢旧换有界延迟，但必须有丢弃计数和业务确认；离线完整处理则可 Block。
- 常见误区：只增大队列，不分析持续过载；丢数据却没有指标。
- 30 秒回答：实时传感器链路更怕旧数据积压，因此我选择丢旧保新；离线处理才允许阻塞。策略和 dropped 指标必须一起设计。
- 2 分钟展开：用到达率、服务率和排队延迟解释为什么“更大队列”不等于修复。

A11 | 非阻塞有限状态机

- 一句话结论：把长动作拆成状态、事件和进入时间，每次调用只推进一步，使任务持续响应新输入和故障。
- 从零解释：连续 delay 会占住该任务的业务流程，即使调度器能运行其他任务，这个控制逻辑也无法处理新事件。
- 底层机制：状态保存跨调用上下文，`now - entered` 判断超时，输出由当前状态唯一决定。
- 项目场景：P2 的转向、提示、掉头和送达流程由 `CartState_Step` 推进，电机只有控制任务一个写入者。
- 代码证据：P2 `APP/cart_control/cart_state.c`、`Core/main.c::vControlTask`。证据：CODE、HOST。
- 工程取舍：状态数增加但可测试、可中断；小动作仍可短延时，但不能影响故障响应要求。
- 常见误区：把 switch 写法当状态机，却没有明确事件、进入/退出动作和非法状态处理。
- 30 秒回答：我把小车转向从阻塞 delay 改成时间驱动 FSM，控制任务是电机唯一写者；测试直接输入时间和传感器状态，验证每次输出。
- 2 分钟展开：画出 FOLLOW→TURN→RECOVER 等状态及超时边。

A12 | Tick 回绕

- 一句话结论：用同宽无符号差值判断经过时间，可以自然处理计数器回绕；不要比较绝对 tick 大小。
- 从零解释：固定宽度无符号算术按模运算，`now - start` 在一次回绕后仍得到正确短间隔。
- 底层机制：前提是目标时间间隔小于可无歧义表示的范围，通常要求不超过计数周期的一半。
- 项目场景：P2 `prvElapsed` 和宿主测试覆盖 `UINT32_MAX` 附近进入状态后的回绕。
- 代码证据：P2 `APP/cart_control/cart_state.c::prvElapsed`、`tests/host/test_cart_state.c`。证据：CODE、HOST。
- 工程取舍：统一使用单调 tick 做持续时间；日历时间用于展示，不用于控制超时。
- 常见误区：`if (now >= deadline)` 在回绕附近失败；有符号转换引入未定义/实现相关行为。
- 30 秒回答：小车状态超时用 `(uint32_t)(now-start) >= duration`，宿主测试从最大值跨回零验证，因此不依赖绝对 tick 排序。
- 2 分钟展开：举 `start=0xffffffff0`，`now=0x10` 的模减法例子。

A13 | 传感器等待为什么必须超时

- 一句话结论：任何依赖外部设备电平、串口或网络的数据等待都必须有上界，否则一个故障外设可永久占住任务。
- 从零解释：忙等直到 DOUT 改变，在传感器断线时永远不返回，上层无法进入降级或错误状态。

- 底层机制：超时把“永久等待”变成显式错误返回，错误才能沿调用链传播。
- 项目场景：P2 HX711 DOUT 等待加入 100 ms 超时，采样结构带 valid 标志，界面显示 fault。
- 代码证据：P2 APP/hx711/hx711.c::HX711_Read、Core/main.c。证据：CODE；真实断线为 TODO-HIL。
- 工程取舍：超时值要基于器件转换时间和调度抖动；过短产生误报，过长拖慢恢复。
- 常见误区：只在最外层加看门狗，不让驱动返回可诊断错误。
- 30 秒回答：HX711 等待现在有 100 ms 上限，失败通过 valid 状态交给任务，而不是把整个传感器任务卡死。
- 2 分钟展开：说明超时、重试、退避和系统看门狗分别解决不同层级的问题。

B02 | 原题 91~99：接口、存储结构与流水线

- SPI、I2C、UART 的选择取决于拓扑、速率、引脚、全双工、寻址和距离。P1/P2 使用 MCU 外设，P5 使用 SPI IMU。
- Harvard/Von Neumann、RISC/CISC、流水线、Cache 和存储层次属于理解 ARM 的基础，但项目没有修改 CPU 微架构，标为 BASE/RELATED。
- NOR/NAND、SRAM/DRAM/SDRAM 的区别要从随机访问、擦写粒度、是否刷新和典型用途回答，不要仅背“谁快”。

3. 错误演化案例

案例一：ISR 里直接控制电机

```

串口中断执行病房匹配和转向
↓
ISR 执行时间随业务分支变化
↓
更高频串口/定时器中断延迟
↓
电机由 ISR 与任务共同写入
↓
输出顺序依赖抢占时机，故障难复现

```

P2 的修复是 ISR 只发布帧，控制任务成为唯一电机写入者。

案例二：队列无限变大

```

生产速度持续高于消费速度
↓
队列等待时间增加
↓
系统处理的是越来越旧的数据
↓
内存耗尽或控制响应失效

```

P1/P2/P3 根据实时数据新鲜度选择 overwrite/DropOldest，并保留过载可观测性。

4. 本章掌握度检查

- 能否画出 USART IRQ → FromISR queue → 控制任务 → 电机的完整路径？
- 能否解释为什么 mutex 有优先级继承而 binary semaphore 通常没有？
- 能否给出 Tick 回绕的数值例子？
- 能否说明“使用 FreeRTOS”不等于已经证明硬实时？
- 能否指出 HX711 超时值仍需实车验证？

03 | Linux 进程、线程、同步与虚拟内存

本章对应原题 100~114、160~180、193~205 的用户态部分。主线是：进程提供隔离，线程提供同地址空间并发，同步原语维护不变量，生命周期协议保证能够退出。

1. 系统调用、进程与线程

A01 | 原题 100：系统调用和库函数

- 一句话结论：系统调用是用户态请求内核服务的受控入口；库函数是用户态接口，可能封装系统调用，也可能完全在用户态执行。
- 从零解释：应用不能直接操作内核对象，需通过 `open/read/epoll_wait` 等接口进入内核。
- 底层机制：架构特定指令切换到内核入口，内核检查参数和权限，执行后返回；`glibc` 通常提供包装。
- 项目场景：P3 调 `socket/epoll`，P5 调 `open/read/epoll/signalfd/timerfd`；`std::memcpy` 则只是用户态库函数。
- 代码证据：P5 `apps/conditiond.cpp`。证据：`CODE`、`HOST`。
- 工程取舍：减少无意义系统调用，但不要为了“性能”绕开正确阻塞模型；批量 `read/bulk read` 可降低频率。
- 常见误区：认为 `printf` 一定对应一个 `write`；`stdio` 还有用户态缓冲。
- 30 秒回答：系统调用跨用户/内核边界，库函数不一定。我的驱动服务用 `epoll_wait` 阻塞等待内核事件，用 `read` 取 IIO 数据，而状态机计算完全在用户态。
- 2 分钟展开：说明 `errno`、短读、`EINTR` 和为什么系统调用返回成功也不代表完整业务帧。

B01 | 原题 101~103：进程状态与调度

Linux 常用状态可概括为运行/就绪、可中断睡眠、不可中断睡眠、停止和僵尸。调度算法的理论分类包括先来先服务、时间片、优先级、多级反馈；Linux

实际调度需要区分普通类与实时类。五个项目没有修改调度器，因此这些内容标为 `BASE`。

项目关联点是：阻塞在 `epoll_wait` 或条件变量的线程不占用 CPU；忙轮询会让任务保持 `runnable`，浪费调度资源。

A02 | 原题 104、204：进程与线程、线程栈

- 一句话结论：进程拥有独立虚拟地址空间和资源容器；同进程线程共享代码、全局区、堆和 `fd`，但每个线程有自己的栈、寄存器上下文和调度状态。
- 从零解释：线程切换通常不需要更换整个地址空间，但共享内存意味着任何未同步读写都可能竞争。
- 底层机制：内核为每个调度实体保存上下文；线程栈映射在同一进程地址空间的不同区域。
- 项目场景：P4 的 `R0S` 回调、Qt 工作线程和采集线程共享对象，因此必须用 `atomic/mutex`；P5 选择单线程 `epoll`，减少共享状态。

- 代码证据：P4 `qnode.h/qnode.cpp`；P5 `conditiond.cpp`。证据：CODE、HOST。
- 工程取舍：I/O 多路复用适合多个 fd；CPU 密集并行或阻塞第三方库才更需要线程池。
- 常见误区：说线程“没有独立内存”；线程有独立栈，但栈仍在共享地址空间，错误指针可访问其他线程栈。
- 30 秒回答：光电项目多线程共享传感器对象，所以必须同步；驱动服务把 `ll0`、`timerfd`、`signalfd` 放进单线程 `epoll`，减少线程数量和退出协调成本。
- 2 分钟展开：比较隔离、切换成本、故障影响和通信方式。

B02 | 原题 105：内核线程和用户线程

Linux 内核线程只在内核地址空间执行，通常没有普通用户地址空间；用户线程属于进程并通过系统调用进入内核。P5 的 `ll0 threaded handler` 是内核执行上下文，但不等同于一个由本项目创建的常驻 `kthread`，因此只标 RELATED。

2. 同步、数据竞争与快照

A03 | 原题 106：进程/线程同步方法

- 一句话结论：同步手段由共享范围和语义决定：线程内常用 `mutex`、`condition_variable`、`atomic`；进程间可用管道、socket、共享内存配合同步对象等。
- 从零解释：互斥保护临界区，条件变量等待状态变化，`atomic` 保护单对象原子操作；三者不能互相简单替代。
- 底层机制：`mutex` 建立同步顺序；`condition_variable` 释放锁并睡眠，唤醒后重新加锁和检查谓词；`atomic` 按内存序约束访问。
- 项目场景：P3 有界队列使用 `mutex+condition_variable`；P4 单值标志用 `atomic`、复合快照用 `mutex`。
- 代码证据：P3 `core/bounded_queue.h`；P4 各传感器 `latest()`。证据：CODE、HOST。
- 工程取舍：单值状态优先 `atomic`；跨多个字段的不变量用 `mutex`；消息传递能减少共享。
- 常见误区：`atomic bool` 能“顺便”保护旁边结构体；`condition variable` 不带谓词直接 `wait`。
- 30 秒回答：我的项目按不变量选同步：停止标志是独立单值，用 `atomic`；传感器结果包含多个相关字段，用同一 `mutex` 复制完整快照；队列用条件变量阻塞等待。
- 2 分钟展开：用 GNSS 经度/纬度来自不同帧说明撕裂快照问题。

A04 | 普通 `bool`、`volatile` 和 `atomic` 的边界

- 一句话结论：并发无同步地读写普通 `bool` 是 `data race`；`volatile` 仍不能修复；`atomic` 提供该对象的原子访问和内存序。
- 从零解释：`data race` 在 C++ 内存模型中是未定义行为，问题不只是偶尔读旧值。

- 底层机制：编译器可基于“程序没有 data race”的假设优化；atomic 操作建立规定的可见性和顺序。
- 项目场景：P4 的 `capture_thread_flag`、`close_flag` 等改为 atomic；复杂结果没有拆成多个 atomic，而使用 mutex。
- 代码证据：P4 `src/mainwindow/include/qnode.h`、`rtsp_capture.h`。证据：CODE。
- 工程取舍：默认 `memory_order_seq_cst` 更易推理；只有测量证明必要才降低内存序。
- 常见误区：认为 x86 “本来就原子”所以普通变量安全；语言层 UB 先于硬件直觉。
- 30 秒回答：volatile 不建立线程同步。光电项目的停止标志用 atomic，复合结果用 mutex，因为多个字段需要作为一个快照保持一致。
- 2 分钟展开：说明 atomic 只保护自身，发布对象时还要考虑构造完成与可见性。

A05 | 一致性快照

- 一句话结论：读取多个相关字段时，必须在同一同步边界下取得快照，否则每个字段单独“线程安全”仍可能来自不同采样时刻。
- 从零解释：GNSS 的纬度、经度、姿态和序号共同描述一帧；读到一半被写线程更新会产生不存在的组合。
- 底层机制：写者和读者使用同一 mutex，读者在锁内复制，锁外处理，既保证一致性又缩短持锁时间。
- 项目场景：P4 各传感器提供 `latest()`，QNode 取得局部 snapshot 后再做 UI/发送处理。
- 代码证据：P4 `gnss_output.h/.cpp::latest`、`qnode.cpp`。证据：CODE。
- 工程取舍：复制有成本，但结构较小且换来明确一致性；点云大对象可用 shared immutable buffer 或双缓冲进一步优化。
- 常见误区：为每个字段设置一个 mutex/atomic；锁内进行网络发送和磁盘写入。
- 30 秒回答：我把复合传感器结果作为整体快照，在同一 mutex 下复制到局部变量，随后释放锁再处理，避免字段跨帧又控制持锁时间。
- 2 分钟展开：比较 mutex 快照、sequence lock、copy-on-write 和双缓冲的适用条件。

A06 | 条件变量为什么必须带谓词

- 一句话结论：等待条件必须在锁保护下循环检查，因为可能虚假唤醒、通知先于等待或多个消费者竞争同一数据。
- 从零解释：notify 只是提示“状态可能变化”，不是把条件本身永久保存。
- 底层机制：`wait(lock, predicate)` 原子地释放锁并睡眠，返回时重新持锁并再次检查 predicate。
- 项目场景：P3 `BoundedQueue::push/pop` 的谓词同时包含容量/数据和 shutdown；关闭时 `notify_all`。

- 代码证据: P3
`src/lslidar_ls_driver/include/.../core/bounded_queue.h`。证据: CODE、HOST。
- 工程取舍: shutdown 必须进入谓词, 否则退出线程可能永远睡眠; notify_one/all 按等待者关系选择。
- 常见误区: 用 if 代替 while; 修改条件不加锁; 先销毁队列再唤醒线程。
- 30 秒回答: 条件变量不保存业务条件。LiDAR 队列把“有数据或已关闭”写进谓词, 关闭时 notify_all, 所以消费者能从空队列等待中退出。
- 2 分钟展开: 讲解 lost wakeup、虚假唤醒和多消费者竞争。

A07 | 原题 107、108: 死锁条件与预防

- 一句话结论: 互斥、占有且等待、不可剥夺、循环等待同时成立才可能死锁; 工程上通过固定锁顺序、缩小临界区、避免嵌套和超时/消息传递破坏条件。
- 从零解释: 死锁不是“线程很慢”, 而是等待图中形成无法自行解除的环。
- 底层机制: 两个线程分别持有 A/B 再等待对方资源是最小环。
- 项目场景: P2 采用单一电机写入者减少共享锁; P3 shutdown 先关闭队列唤醒等待者, 再 join; P4 不在快照锁内放慢 I/O。
- 代码证据: P2 `vControlTask`; P3 `PipelineController::stop`; P4 `latest()`。证据: CODE、HOST。
- 工程取舍: 锁层次适合多个锁; actor/单写者结构从设计上删除部分锁。
- 常见误区: 把优先级反转、活锁、饥饿都叫死锁; 只靠 try_lock 隐藏设计问题。
- 30 秒回答: 我优先减少共享: 小车电机只有一个写者; 流水线退出时先 shutdown queue 解除等待, 再 join。若必须多锁, 则固定顺序且不在锁内 I/O。
- 2 分钟展开: 画资源等待图, 并分别指出项目如何破坏四个条件之一。

3. 线程生命周期与退出协议

A08 | detach 与 join

- 一句话结论: detach 放弃等待和生命周期控制; 对象成员线程通常应有明确 stop 协议并 join。
- 从零解释: 线程函数捕获 this 后, 对象析构并不自动停止线程。
- 底层机制: detached 线程独立继续, 访问已析构成员形成 use-after-free; 进程退出只是粗暴终止, 并非正确清理。
- 项目场景: P4 移除 detached spinner 和工作线程, 析构时置停止标志、唤醒/等待、join, 再释放设备。
- 代码证据: P4 `QNode::~~QNode`、`RTSPCapture::stop_thread`。证据: CODE。

- 工程取舍：线程所有者与对象所有者对齐；若后台任务确需独立，使用进程级服务或显式共享生命周期。
- 常见误区：认为 `detach` 能防止主线程阻塞；它只是让问题不可等待。
- 30 秒回答：原 `detached` 线程可能在对象销毁后继续访问。重构后所有成员线程都有 `stop+join`，设备在 `join` 后释放。
- 2 分钟展开：说明 `stop flag`、阻塞唤醒、`join` 顺序和析构异常安全。

A09 | 正确停止流水线的顺序

- 一句话结论：先发布停止意图并解除所有阻塞，再等待线程结束，最后销毁线程依赖的对象。
- 从零解释：只改 `atomic flag` 不足以唤醒正在 `condition_variable/epoll/read` 中阻塞的线程。
- 底层机制：关闭队列改变谓词并 `notify_all`；关闭/写 `eventfd` 等方式解除 I/O 等待；`join` 建立线程结束同步。
- 项目场景：P3 Pipeline 从下游到上游停止，`queue shutdown` 解除 `push/pop`；P4 各设备线程先 `stop` 再 `wait`。
- 代码证据：P3 `PipelineController::stop`、`BoundedQueue::shutdown`；P4 `qnode.cpp`。证据：CODE、HOST。
- 工程取舍：停止顺序依据数据流；下游先停止可防止继续产生副作用，但要保证上游不会永久阻塞，项目通过 `shutdown` 解除。
- 常见误区：先析构 `queue` 再 `join`；只设置标志；无限等待无超时和日志。
- 30 秒回答：退出协议不是一个 `bool`。LiDAR 先 `shutdown` 队列唤醒等待者，再按数据流停止并 `join`，最后资源析构。
- 2 分钟展开：画出 `producer→queue→consumer` 的停止时序和不同顺序的风险。

4. 虚拟内存与地址

B03 | 原题 109~114、172、203：地址、MMU 与虚拟内存

需要区分：

- 虚拟地址：进程看到的地址，由页表映射。
- 物理地址：内存控制器看到的地址。
- 总线地址/DMA 地址：设备发起访问时使用，可能经过 IOMMU 或平台转换。
- 页表和 TLB：页表保存映射，TLB 缓存近期转换。

虚拟内存提供隔离、按需分配、共享映射和比物理内存更灵活的地址空间。P5 的 Linux 用户程序只能通过设备文件/`sysfs`

访问驱动，不能把设备树物理地址直接当用户指针。项目没有实现页表或 DMA，因此标 BASE/RELATED。

B04 | 原题 173~179: fork、exec、父子进程与僵尸

`fork` 创建逻辑子进程，通常通过写时复制共享物理页；`exec` 用新程序映像替换当前进程；子进程退出后父进程未 `wait` 会留下僵尸记录。当前五项目没有用 `fork/exec` 构建进程模型，不能包装为项目能力。

P5 部署为 `systemd` 服务，进程监督交给 `systemd`，而不是应用自行 `double-fork` 守护化。这个选择可以作为现代 Linux 服务管理的工程说明。

B05 | 原题 102: Linux IPC

管道/命名管道适合字节流；Unix domain socket 支持双向与 `fd` 传递；消息队列保留消息边界；共享内存吞吐高但同步最复杂；信号适合有限通知。当前项目主要使用线程内同步、ROS `topic` 和网络 `socket`，未直接实现 POSIX 共享内存或 System V IPC，标为 **RELATED**。

5. 用户态故障定位**A10 | 原题 160: 段错误和程序跑飞怎样定位**

- 一句话结论：先保存现场和可复现输入，再从 `core/backtrace`、日志、sanitizer 和资源指标逐层缩小，不靠猜。
- 从零解释：段错误只说明非法内存访问被 MMU 捕获，根因可能是越界、悬空、栈溢出、竞态或 ABI 不一致。
- 底层机制：`core dump` 保存寄存器和内存映像；带符号二进制可把地址还原到调用栈。
- 项目场景：P4 原生命周期风险和别名强转都可能产生偶现故障；重构用 `join`、`mutex snapshot` 和 `memcpy` 降低 UB。
- 代码证据：P4 `INTERVIEW_EVIDENCE.md` 和相关实现。证据：**CODE**；未制造真实 `core`，故为 **RELATED** 调试流程。
- 工程取舍：开发构建保留 `-g`，测试启用 ASan/UBSan；目标机资源有限时可离线分析 `core`。
- 常见误区：只看最后一条日志；优化构建无符号；复现前就修改多个地方。
- 30 秒回答：我会固定版本和输入，启用 `core`，使用 `gdb bt/thread apply all bt`；内存类问题再用 ASan/Valgrind，竞态用 TSan 或同步审查，并对 `fd`/内存趋势做长稳观察。
- 2 分钟展开：按“复现→现场→最小化→假设→验证”描述调试闭环。

B06 | 原题 161~168: 命令、编译、库和链接

- `gcc -E/-S/-c/link` 对应预处理、编译、汇编和链接。
- 静态库内容复制进入最终产物；动态库在加载/运行时解析，减小重复但引入版本和部署依赖。
- `make` 根据目标、依赖和命令决定增量构建；CMake 生成具体构建系统，不是编译器。
- 软链接保存路径，硬链接增加同一 `inode` 的目录项。

P5 的 ARM 工具链文件与 `file` 输出是交叉编译证据；P3/P4 的 ROS/catkin 依赖说明动态库版本是部署风险。

A11 | 原题 169、170：文件描述符与 inode

- 一句话结论：fd 是进程文件描述符表中的整数索引，指向内核打开文件描述；inode 表示文件对象元数据，目录项把名字映射到 inode。
- 从零解释：socket、epoll、timerfd、signalfd 和 IIO 设备都可以表现为 fd，因此能统一进入事件循环。
- 底层机制：dup/fork 可让多个 fd 指向同一打开文件描述，共享偏移/状态的细节需区分。
- 项目场景：P5 用 fd 统一设备、时钟、信号；P3 管理 UDP socket 与 epoll fd。
- 代码证据：P5 `UniqueFd`、P3 `InputSocket`。证据：CODE、HOST。
- 工程取舍：用 `-1` 表示无所有权，设置 `CLOEXEC`，异常路径也关闭。
- 常见误区：fd 等于文件；close 一个重复 fd 会自动关闭所有引用。
- 30 秒回答：fd 是进程表索引，不只代表磁盘文件。我的服务把 IIO、timerfd、signalfd 都注册进 epoll，并用 `UniqueFd` 管理关闭责任。
- 2 分钟展开：说明 file description、inode、dentry 和设备 file_operations 的关系。

A12 | 原题 171：/proc/<pid> 能看什么

- 一句话结论：`/proc/<pid>` 是内核导出的进程运行视图，可检查 maps、status、fd、task 和调度/内存状态。
- 从零解释：数字目录名就是 PID；内容不是普通持久文件，而是读取时由内核生成。
- 底层机制：fd/ 软链接可观察打开资源，maps 查看映射，status 查看线程和内存概要。
- 项目场景：P3/P4/P5 长稳时可采样 fd 数、线程数和 RSS，验证重连/启动停止是否泄漏。
- 代码证据：属于验证方法 `RELATED`，当前仓库未提交长期曲线。
- 工程取舍：轻量现场先用 proc，复杂内存问题再用 profiler/sanitizer。
- 常见误区：把 VIRT 当实际物理占用；忽略共享页和缓存。
- 30 秒回答：我会比较 `/proc/pid/fd`、`task` 和 `status` 的时间序列，判断断流重连后是否持续增长，再结合日志定位创建路径。
- 2 分钟展开：说明 smaps、fdinfo 和 limits 的用途。

A13 | 原题 180：管程思想

- 一句话结论：管程把共享状态、互斥和条件等待封装成一组操作，使调用者不能绕开同步协议。
- 从零解释：它不是某个单独 Linux API，而是一种结构化并发抽象。
- 底层机制：同一时刻一个执行流进入受保护操作；条件变量等待状态谓词。

- 项目场景：P3 `BoundedQueue` 封装队列、mutex、两个 `condition_variable` 和 `shutdown`，不让调用者直接碰内部 `deque`。
- 代码证据：P3 `core/bounded_queue.h`。证据：`CODE`、`HOST`。
- 工程取舍：封装增加少量接口代码，但把不变量集中，远好于散落的共享容器和外部锁。
- 常见误区：把管程等同于 `mutex`；缺少条件谓词和封装边界。
- 30 秒回答：LiDAR 的 `BoundedQueue`
就是管程式封装：队列状态、锁、条件变量和关闭协议在同一类中，外部只能 `push/pop/shutdown`。
- 2 分钟展开：列出容量、关闭后唤醒、丢弃计数这些类不变量。

6. 本章概念辨析

概念	核心区别	项目落点
进程 vs 线程	地址空间隔离 vs 共享地址空间调度实体	P4 多线程、P5 单进程事件循环
atomic vs mutex	单对象原子语义 vs 复合不变量	P4 标志与快照
notify vs condition	通知不是状态，谓词才是	P3 <code>BoundedQueue</code>
detach vs join	放弃生命周期控制 vs 可等待结束	P4 修复
fd vs file	表索引 vs 内核对象/打开描述	P3/P5
虚拟地址 vs 物理地址	进程视图 vs 硬件内存地址	P5 用户/内核边界

7. 本章掌握度检查

- 能否解释普通 `bool` 跨线程访问为什么是语言层 UB？
- 能否画出 `mutex` 快照并说明锁内只复制、锁外处理？
- 能否说清 `shutdown`、`notify_all`、`join`、析构的顺序？
- 能否解释一个 `epoll fd`、一个 `socket fd` 和 `inode` 的关系边界？
- 能否给出段错误定位的证据链，而不是只说“用 `gdb`”？

04 | Linux I/O、事件循环与网络

本章对应原题 115~159、181~189。主线是：文件描述符统一事件源，非阻塞 I/O 允许一个线程管理多个源，协议解析必须接受短读、丢包和重连。

1. 网络分层与项目边界

B01 | 原题 115~119：OSI/TCP-IP 分层

面试不需要机械背七层，而要能把项目数据放进层次：

```
应用层：LiDAR 私有报文、RTSP、ROS topic、conditiond JSON 输出
传输层：UDP (LiDAR)、TCP (RTSP 控制及常见流媒体承载)
网络层：IPv4
数据链路层：以太网
物理层：网线/PHY
```

分层的价值是把“报文含义、可靠性、寻址、帧传输”分给不同协议。项目没有自己实现 TCP/IP 栈，因此这些题为 **RELATED**，但能用实际数据路径解释。

C01 | 原题 120~131：HTTP、HTTPS、Cookie、DNS 与浏览器流程

这些题属于通用网络基础，不应强行绑定当前五项目：

- HTTP 是应用层请求/响应协议；1.1 的持久连接、2 的多路复用与头部压缩要区分。
- HTTPS 是 HTTP 运行在 TLS
上，通常用非对称密码完成身份认证/密钥协商，用对称密码传输数据；证书把公钥和身份由 CA 签名绑定。
- Cookie 是浏览器保存并随请求发送的数据，Session 通常是服务端状态，Token 是调用者携带的凭据表示；三者不是同一层面的互斥选项。
- DNS 把名称解析为地址，浏览器访问还涉及缓存、连接、TLS、HTTP、资源解析与渲染。

如果面试官问到，只按基础回答；除非项目真实增加 HTTP/TLS 上报，否则不要说“ESP8266 上传”就等于自己实现了 HTTPS。

2. TCP、UDP 与实时传感器数据

A01 | 原题 132、133、134：TCP 头部、握手与挥手

- 一句话结论：TCP 用序号、确认、窗口、重传和状态机提供可靠有序字节流；三次握手同步双方初始序号和能力，关闭方向独立所以常见四次挥手。
- 从零解释：TCP 不保留应用消息边界，发送两次可能一次读出，发送一次也可能多次读完。
- 底层机制：SYN 消耗一个序号；ACK 确认已收到的下一个序号；FIN 只关闭一个发送方向。
- 项目场景：P4 的 RTSP 连接由库管理
TCP/媒体协议，项目重构关注断流后的恢复，而不是自行实现握手。
- 代码证据：P4 `rtsp_capture.cpp`。证据：**RELATED**，重连策略为 **CODE/HOST**。
- 工程取舍：使用成熟协议库；应用仍需处理连接成功后断流、超时和重新初始化资源。

- 常见误区：说“三次握手保证数据不丢”；可靠性还依赖序号、ACK、重传和流量/拥塞控制。
- 30 秒回答：TCP 三次握手确认双向收发和初始序号，关闭两个方向所以常见四次。我的 RTSP 项目不重写 TCP，而是处理连接断开后的有界退避和线程退出。
- 2 分钟展开：画 SYN、SYN-ACK、ACK 和 FIN/ACK 状态，说明应用边界不存在。

B02 | 原题 135~137: TIME_WAIT、2MSL 与断网

主动关闭方进入 TIME_WAIT，使迟到的旧报文在网络中消失，并能重发最后 ACK。2MSL 来自报文最大生存时间的往返考虑。拔网线不会立即神奇删除内核连接状态，应用何时发现取决于后续发送、超时、keepalive 和协议心跳。

P4 的 RTSP 不能只等待 TCP 立即报错，而要把“持续读不到有效帧”也纳入失败判断；当前项目验证的是退避序列，不声称覆盖所有 TCP 状态。

A02 | 原题 138、139: TCP 与 UDP 如何选择

- 一句话结论：TCP 提供可靠有序字节流和拥塞控制；UDP 提供无连接数据报，保留消息边界但不保证到达、顺序或唯一性。
- 从零解释：实时传感器可能宁愿丢旧包也不愿重传导致延迟；配置/控制命令通常更看重可靠性。
- 底层机制：UDP 每个 datagram 独立，接收缓冲不足可能截断/丢弃；TCP 把数据视为连续字节序列。
- 项目场景：P3 LiDAR 使用 UDP 连续接收点云包，P4 RTSP 由协议库管理，P1 ESP8266 上报接口不夸大具体可靠协议。
- 代码证据：P3 `src/input.cc::InputSocket`。证据：CODE；真实网络性能为 TODO-HIL。
- 工程取舍：UDP 需要长度、来源、序号/时间戳和丢包指标；TCP 需要处理半包、积压与重连。
- 常见误区：说 UDP “一定更快”；网络拥塞时不受控发送可能更差。
- 30 秒回答：LiDAR 持续高速采样更重视新鲜度，所以使用 UDP；代码仍检查来源、长度和错误，并在后续队列统计丢弃。可靠控制链才更适合 TCP。
- 2 分钟展开：比较可靠性、边界、连接状态、拥塞控制、广播/组播和应用补偿。

A03 | 原题 181: 网络字节序与安全解析

- 一句话结论：网络字节序是大端；解析字节流必须先检查长度，再用转换函数或逐字节/memcpy 读取，不能直接解引用未对齐强转指针。
- 从零解释：不同 CPU 端序可能不同，协议约定统一顺序才能互通。
- 底层机制：`ntohs/ntohl` 转换整数；浮点和私有格式需要协议明确。未对齐和严格别名错误会产生 UB。
- 项目场景：P3 PCAP/UDP 路径检查帧长；P4 浮点解析改用 memcpy；P5 IIO scan 使用明确通道布局。

- 代码证据：P3 `InputPCAP::getPacket`，P4 串口解析，P5 scan 解码。证据：CODE。
- 工程取舍：性能敏感时用 bulk read 后逐字段解析；正确性检查不能省。
- 常见误区：所有主机都是小端所以直接 cast；把结构体通过 socket 发送就认为跨平台。
- 30 秒回答：我先验证报文长度，再按协议端序解析；浮点采用 memcpy 避免未对齐和严格别名问题，不直接把字节指针强转为结构体。
- 2 分钟展开：给出 0x1234 在大/小端内存中的顺序，并说明 IIO 是本机 ABI，不等于网络协议。

3. Socket 生命周期

A04 | 原题 182：客户端/服务端 socket 基本流程

- 一句话结论：服务端通常 `socket→bind→listen→accept→read/write→close`；客户端 `socket→connect→read/write→close`；UDP 不需要 `listen/accept`。
- 从零解释：监听 socket 接收连接请求，accept 返回新的已连接 fd；二者生命周期不同。
- 底层机制：阻塞、非阻塞、超时和错误由 fd flags 与系统调用语义决定。
- 项目场景：P3 是 UDP 接收端，`socket→bind→设置非阻塞→注册 epoll`；构造任何一步失败都清理已有 fd。
- 代码证据：P3 `src/lslidar_ls_driver/src/input.cc::InputSocket`。证据：CODE。
- 工程取舍：单 socket 用 poll 也够；项目保留 epoll 以统一未来多个传感器 fd，但不声称单 fd 性能更高。
- 常见误区：忘记 bind 的地址/端口字节序；构造失败泄漏 socket；EINTR 直接当致命错误。
- 30 秒回答：LiDAR 是 UDP 服务端式接收，不需要 `listen/accept`。创建、bind、nonblock 和 epoll 注册逐步进行，每个失败分支关闭已取得资源。
- 2 分钟展开：描述 `recvfrom` 的来源地址、过滤策略和 graceful stop。

B03 | 原题 183：并发服务器用进程还是线程

多进程隔离强、故障影响小、通信成本高；多线程共享数据方便但需要同步；事件循环适合大量 I/O 等待。当前项目不是通用并发服务器，但 P5 的“一个 epoll 线程管理三个 fd”可作为第三种结构对比，不能把它说成高并发服务器压测结果。

4. I/O 多路复用

A05 | 原题 184：什么是 I/O 多路复用

- 一句话结论：一个线程阻塞等待多个 fd 的就绪事件，避免为每个 I/O 源创建线程或忙轮询。
- 从零解释：就绪只表示某次操作“现在不会按原方式阻塞”，不保证能读到完整应用消息。
- 底层机制：内核维护等待关系，事件发生时唤醒等待者并返回就绪集合。
- 项目场景：P5 同时等待 IIO、`timerfd`、`signalfd`；P3 等待 UDP socket。

- 代码证据: P5 `apps/conditiond.cpp`, P3 `input.cc`。证据: `CODE`、`HOST`。
- 工程取舍: 事件循环减少线程协调, 但长 CPU 任务不能堵塞 loop, 需拆分或投递工作池。
- 常见误区: 把异步 I/O、非阻塞 I/O 和多路复用当同义词。
- 30 秒回答: `conditiond` 用 `epoll` 把传感器、周期超时和退出信号统一成 fd 事件, 一个线程阻塞等待, 不需要忙轮询或三条线程。
- 2 分钟展开: 解释 `readiness`、短读、`EAGAIN` 和事件处理公平性。

A06 | 原题 185、188: `select`、`poll`、`epoll` 对比

- 一句话结论: `select/poll` 每轮传递并扫描关注集合; `epoll` 把兴趣集合保留在内核并返回就绪事件, 适合较大、长期的 fd 集合, 但不是任何场景都更快。
- 从零解释: `select` 使用 `fd_set` 且常有 `FD_SETSIZE` 限制; `poll` 使用数组; `epoll` 通过 `create/ctl/wait` 分离注册与等待。
- 底层机制: 复杂度讨论必须包含活跃比例、系统调用次数、缓存行为和 fd 数量, 不能只背 $O(n)/O(1)$ 。
- 项目场景: P3 单 socket 用 `poll` 足够, `epoll` 用于扩展统一; P5 三类长期 fd 使用 `epoll` 使控制流一致。
- 代码证据: P3 `InputSocket`, P5 `addEpoll/main`。证据: `CODE`、`HOST`。
- 工程取舍: fd 很少时可读性更重要; `epoll` 带来注册状态和 LT/ET 语义成本。
- 常见误区: 说 `epoll` 完全没有遍历; 用户仍要处理返回的就绪数组。
- 30 秒回答: `epoll` 的优势是兴趣集合留在内核, 只返回就绪项。P5 用它统一 `llio`、`timerfd`、`signalfd`; P3 当前单 socket 选择更多是架构一致性, 不夸大性能。
- 2 分钟展开: 逐项比较接口、集合更新、数量限制、返回方式和适用规模。

A07 | 原题 186: `epoll` 生命周期与错误处理

- 一句话结论: `epoll` 本身也是 fd; 创建、注册、等待、删除/关闭都要纳入资源所有权, 事件必须检查 `ERR/HUP` 和 `EINTR`。
- 从零解释: `epoll_ctl(ADD)` 建立关注关系, `epoll_wait` 返回事件; 被监控 fd 关闭后关系由内核处理, 但应用仍需明确关闭顺序。
- 底层机制: `CLOEXEC` 防止 `exec` 后泄漏; `wait` 被信号打断返回 `EINTR` 不一定是业务故障。
- 项目场景: P3 析构分别关闭 `epoll/socket`; P5 所有 fd 用 `UniqueFd`, 正常/异常路径一致。
- 代码证据: P3 `InputSocket::~~InputSocket`, P5 `UniqueFd`。证据: `CODE`、`HOST`。
- 工程取舍: 用 `RAII` 集中关闭; 错误事件转换为上层状态, 不在循环静默继续。
- 常见误区: 只处理 `EPOLLIN`; fd 默认初始化为 0 导致误关 `stdin`; 重复 `close`。
- 30 秒回答: 我用 -1 表示无所有权、`RAII` 管 `epoll` 和源 fd, 并处理 `EINTR/ERR/HUP`。创建中途失败不会留下半初始化 fd。

- 2 分钟展开：说明 `close` 与 `epoll interest` 的关系、`CLOEXEC` 和 `fd` 复用风险。

A08 | 原题 187：LT、ET 与读到 EAGAIN

- 一句话结论：LT 在就绪时持续通知；ET 通常只在状态边沿通知，因此必须使用非阻塞 `fd` 并循环处理到 `EAGAIN`。
- 从零解释：ET 如果只读一次，缓冲区剩余数据可能没有新的边沿，再也等不到通知。
- 底层机制：事件表示内核缓冲状态变化，不等于应用帧完成。
- 项目场景：P5 使用 LT 但仍采用非阻塞 `drain` 和 `pending` 缓冲，能处理一次多帧与尾部半帧。
- 代码证据：P5 `apps/conditiond.cpp` 读取循环。证据：CODE、HOST。
- 工程取舍：LT 更容易写对；ET 可减少重复通知，但代码和公平性控制更复杂。
- 常见误区：ET 使用阻塞 `fd`；遇到 `EAGAIN` 当错误；一次 `read` 等于一帧。
- 30 秒回答：ET 必须读到 `EAGAIN`。我的服务即使用 LT，也按非阻塞方式 `drain`，并把不足 24 字节的 `scan` 留到下一次拼接。
- 2 分钟展开：描述 `read` 返回 10、38 字节时 `pending` 如何形成 24 字节帧。

A09 | 短读、半包与帧解析

- 一句话结论：字节流接口只返回当前可用字节，应用必须自己维护帧边界、剩余字节和输入上限。
- 从零解释：`read` 可能少于请求、等于多帧之和或被 `EINTR` 打断。
- 底层机制：内核缓冲、调度和驱动分批提交决定返回大小，不受应用结构体大小约束。
- 项目场景：P5 `pending` 累积 IIO `scan`；P3 UDP 保留报文边界但仍检查截断；P2 USART ISR 自己组成固定两字节业务帧。
- 代码证据：P5 `conditiond.cpp`、P3 `InputPCAP/getPacket`、P2 USART ISR。证据：CODE、HOST。
- 工程取舍：固定帧简单但版本扩展弱；长度字段需验证最大值，避免内存攻击/错误。
- 常见误区：在 TCP/IIO 上直接 `read(fd, &struct, sizeof struct)` 并假设完整。
- 30 秒回答：我把 I/O 当字节流，`pending` 保存残片，只有累积满 24 字节才解码；UDP 路径则按数据报检查最小长度和截断。
- 2 分钟展开：说明固定长度、`delimiter`、`length-prefix` 三种 framing 方案。

A10 | `signalfd`：把信号并入事件循环

- 一句话结论：先在线程中屏蔽信号，再用 `signalfd` 读取信号事件，可避免异步 `signal handler` 的严格函数限制。
- 从零解释：普通 `handler` 可在任意指令间打断程序，能安全调用的函数很少，不能随意锁 `mutex`、`iostream` 或分配内存。

- 底层机制：被屏蔽信号排队到 `signalfd`，`read` 返回 `signalfd_siginfo`，处理发生在正常线程控制流。
- 项目场景：P5 把 `SIGINT/SIGTERM` 与 `ll0/timerfd` 一同注册 `epoll`，收到后走正常 `RAII` 退出。
- 代码证据：P5 `apps/conditiond.cpp`。证据：`CODE`、`HOST`。
- 工程取舍：Linux 特有但简化事件循环；便携程序可使用 `self-pipe/eventfd` 模式。
- 常见误区：创建 `signalfd` 却未先 `block` 信号；在 `handler` 中调用非 `async-signal-safe` 函数。
- 30 秒回答：`conditiond` 不在 `signal handler` 改复杂共享状态，而是用 `signalfd` 把 `SIGTERM` 变成普通可读事件，随后自然离开循环并由 `RAII` 清理。
- 2 分钟展开：说明多线程程序应在哪个时机设置 `signal mask`。

A11 | `timerfd` 与单调时钟

- 一句话结论：`timerfd` 把定时事件变成 `fd`；超时判断应使用单调时钟，避免系统时间校准导致跳变。
- 从零解释：读 `timerfd` 得到累计到期次数，不能假设每次只过期一次。
- 底层机制：`CLOCK_MONOTONIC` 不受墙上时间手动修改影响，适合持续时间。
- 项目场景：P5 用 `timerfd` 周期检查传感器超时，模拟模式另一个 `timerfd` 产生样本。
- 代码证据：P5 `apps/conditiond.cpp::makeTimer`。证据：`CODE`、`HOST`。
- 工程取舍：统一进入 `epoll`，避免专用定时线程；长任务可能造成多次到期累积，需要读取计数。
- 常见误区：用 `time()` 做控制超时；不读取 `timerfd` 导致一直可读。
- 30 秒回答：我的服务用 `CLOCK_MONOTONIC timerfd` 做超时检查，读取累计到期次数，并与设备和信号共用 `epoll`。
- 2 分钟展开：比较 `timerfd`、`POSIX timer`、`sleep` 和 `RTOS tick`。

5. 过载、重连与离线输入

A12 | 有界队列与反压

- 一句话结论：容量上限把无限资源风险变成可选择的过载策略；反压必须从业务价值决定阻塞、拒绝或丢旧。
- 从零解释：短时突发可由队列吸收，持续过载无法靠容量解决。
- 底层机制：P3 的 `Block` 等待 `not_full`，`DropOldest` 在锁内弹出队首并记 `dropped`。
- 项目场景：`LiDAR` 实时链路丢旧保新；离线 `PCAP` 可阻塞；P4 点云保存队列最多 3 帧。
- 代码证据：P3 `bounded_queue.h/test_bounded_queue.cpp`，P4 `save_point_data`。证据：`CODE`、`HOST`。

- 工程取舍：新鲜度与完整性不能同时无限保证；指标让过载可见。
- 常见误区：只写“线程安全队列”不写容量和 shutdown；把丢弃隐藏。
- 30 秒回答：实时链路选择 DropOldest 控制延迟，离线链路可 Block；两者都必须有界并记录 dropped。
- 2 分钟展开：结合 Little 定律解释排队时间与在途数量。

A13 | 断流重连与指数退避

- 一句话结论：失败后立即循环重连会形成 CPU、日志和网络风暴；指数退避设上限，成功后复位。
- 从零解释：短暂故障先快速恢复，持续故障逐渐降低尝试频率。
- 底层机制：等待序列 250、500、1000、2000、4000、5000 ms；分成小步等待保持停止响应。
- 项目场景：P4 RTSPCapture 使用 ReconnectBackoff，宿主测试覆盖增长、封顶和 reset。
- 代码证据：P4 reconnect_backoff.h、rtsp_capture.cpp、host test。证据：CODE、HOST。
- 工程取舍：上限避免恢复太慢；生产系统还可增加 jitter，防止多设备同步重连。
- 常见误区：异常后无 sleep；每次失败重新创建对象却不释放旧资源。
- 30 秒回答：RTSP 重连采用封顶指数退避，成功后 reset；等待按 50 ms 小步进行，因此 stop 不会被 5 秒整段阻塞。
- 2 分钟展开：解释 jitter、最大次数、熔断和健康状态上报。

A14 | PCAP/EOF 与正常生命周期

- 一句话结论：EOF 是离线输入的正常结束状态，不应 abort；解析前必须验证捕获长度和协议偏移。
- 从零解释：abort 跳过正常清理并生成崩溃语义，会把一次回放结束误判成程序错误。
- 底层机制：PCAP 捕获长度可能小于原始帧长度，过滤器也可能拒绝帧。
- 项目场景：P3 InputPCAP::getPacket 检查 null、截断、EOF 和过滤结果，用返回状态让上层收尾。
- 代码证据：P3 src/input.cc::InputPCAP。证据：CODE。
- 工程取舍：离线回放可重现输入，是网络解析测试的重要工具；真实设备性能仍需 HIL。
- 常见误区：按固定偏移无长度检查；EOF 直接 abort；把 PCAP 通过等同实时网络通过。
- 30 秒回答：PCAP 回放结束不是崩溃。我让输入层返回 EOF 状态并正常停止流水线，同时对每层报文偏移先做截断检查。
- 2 分钟展开：说明 caplen、len、过滤和 loop replay 的状态设计。

6. 数据结构与算法边界（原题 140~159）

这些题全部纳入索引，但不包装成项目经验：

- 顺序表与链表：连续内存/随机访问与节点插入/稳定性的权衡。
- 环形链表检测：快慢指针；找入口可在相遇后从头同步走。
- 循环队列：用 head/tail 和空满约定，或额外 size；项目中的有界队列并非无锁环形队列。
- 树、红黑树、B+ 树、图、最短路径、哈希、快排和堆：按不变量、复杂度与使用场景掌握。

P3 的 BoundedQueue、P5

的固定数组可用于解释“数据结构为业务语义服务”，但不能说项目实现了红黑树或 B+ 树。

7. 本章掌握度检查

- 能否解释 epoll 就绪为什么不等于完整消息？
- 能否手推一次 read 返回 50 字节时两个 24 字节 scan 和 2 字节残片？
- 能否说明 UDP 适合 LiDAR 的理由和必须补偿的风险？
- 能否区分 LT、ET、blocking、nonblocking 四个概念？
- 能否解释指数退避为什么还需要最大值和停止响应？

05 | Linux 内核、设备模型与 IIO 驱动

本章对应原题 190~216。它以 P5 为主证据，完整串起：

```
设备树节点
→ SPI core 创建 spi_device
→ of_match 匹配 spi_driver
→ probe + regmap + WHO_AM_I
→ IIO channel / trigger / buffer
→ sysfs + /dev/iio:deviceX
→ condition epoll
→ 用户态状态机
```

1. 内核职责与上下文

B01 | 原题 190：Linux 内核五大子系统

常见回答是进程调度、内存管理、虚拟文件系统、网络和设备驱动。更重要的是说明边界：P5 的驱动通过 SPI/IIO/VFS

暴露设备机制，用户态服务通过调度与虚拟内存运行；项目没有修改内核核心子系统。

B02 | 原题 191、192：内核构建与启动

内核构建通常包含选择架构与交叉工具链、配置、编译 Image/zImage、DTB 和模块，再与 bootloader/rootfs 配套部署。启动链可概括为 bootloader 准备

CPU/内存和启动参数，内核解压、初始化体系结构/内存/调度/驱动，挂载根文件系统并启动 PID 1。

P5 只验证驱动对象与 vendor 4.1.15 API/ARM 编译器兼容；当前 vendor .config 未启用 IIO buffer/trigger，因此不能声称完成了完整可加载 .ko。

A01 | 原题 193、194：上下文、内核态与用户态

- 一句话结论：上下文决定当前可以睡眠、可以访问什么、失败如何返回；用户态受隔离保护，通过系统调用进入内核态。
- 从零解释：同一段“读传感器”逻辑放在进程、线程化中断或 hrtimer 回调中，允许的操作完全不同。
- 底层机制：原子上下文不能调度睡眠；进程上下文可以阻塞。内核地址错误可能影响整个系统，用户进程错误通常被隔离。
- 项目场景：P5 hrtimer 只触发 IIO poll，SPI bulk read 放在线程化 handler；业务算法留用户态。
- 代码证据：P5
kernel/icm20608_iio.c::icm20608_timer_callback/trigger_handler。证据：CODE、CROSS。
- 工程取舍：内核只实现稳定机制，策略在用户态迭代，降低故障影响和测试成本。
- 常见误区：把“内核态更快”当成把算法放内核的理由；忽略可睡眠上下文。

- 30 秒回答：项目里最关键的上下文边界是 `hrtimer` 不可睡眠，所以只发 `trigger`；SPI 读取在线程化 `handler`，倾斜/冲击判断在用户态。
- 2 分钟展开：比较进程上下文、中断上下文、`softirq/workqueue/threaded IRQ`。

B03 | 原题 195、196：用户空间如何进入内核

用户空间与内核通信方式包括系统调用、设备文件

`read/write/ioctl/mmap`、`sysfs/procfs/netlink` 等。`open()` 经 `libc`

包装进入系统调用，VFS 根据路径查找 `dentry/inode`，得到 `file`，并通过 `file_operations` 到具体驱动。

P5 使用 `sysfs` 配置 IIO 属性、设备节点读取连续 `scan`；没有自定义 `ioctl`。IIO core 屏蔽了大量字符设备框架细节，因此面试时应说明“走标准 IIO ABI”，而不是冒充自己实现全部 VFS 路径。

2. 中断上下半部与内核同步

A02 | 原题 197、198：为什么分上下半部，如何实现

- 一句话结论：中断快路径只完成必须立即处理的工作，耗时或可睡眠操作延后；常见机制有 `threaded IRQ`、`workqueue`、`softirq/tasklet` 等，具体选择取决于上下文需求。
- 从零解释：硬中断长期占用会提高全系统中断延迟，且不能执行可能睡眠的 SPI/I2C。
- 底层机制：P5 无 `DRDY` 引脚，以 `hrtimer` 产生 `trigger`；IIO `pollfunc` 在线程化上下文执行 `regmap_bulk_read`。
- 项目场景：`timer callback` 等价承担通知快路径，`trigger handler` 承担慢路径。
- 代码证据：P5 `icm20608_timer_callback`、`icm20608_trigger_handler`。证据：CODE、CROSS。
- 工程取舍：有真实 `DRDY` 时可改硬件 `IRQ`；当前板级经历没有可用中断线，因此软件 `trigger` 更诚实。
- 常见误区：把 Linux `softirq` 与 Cortex-M SVC 混淆；在 `hrtimer` 中直接调用 SPI。
- 30 秒回答：因为 SPI 可能睡眠，不能放在 `hrtimer` 原子上下文。我的 `timer` 回调只调用 `iio_trigger_poll`，真正 `bulk read` 在线程化 `poll handler` 完成。
- 2 分钟展开：比较 `workqueue` 与 `threaded IRQ`，并说明为什么 `tasklet` 不适合可睡眠 I/O。

A03 | 原题 199~201：内核同步、自旋锁与信号量

- 一句话结论：自旋锁不可睡眠，适合极短原子上下文；`mutex/semaphore` 可能睡眠，只能在可睡眠上下文；中断与进程共享数据还需选择 `irqsave` 等变体。
- 从零解释：持自旋锁睡眠会让其他 CPU/上下文一直自旋并破坏调度；中断上下文自身不能睡眠。
- 底层机制：禁抢占/中断与自旋锁的组合取决于共享者可能在哪些上下文出现。

- 项目场景：P5 避免在 timer 与线程 handler 之间无锁修改周期：buffer 运行时修改频率返回 `-EBUSY`，状态由 IIO buffer 生命周期串行化。
- 代码证据：P5 `icm20608_write_raw` 与 buffer 状态检查。证据：CODE、CROSS。
- 工程取舍：能通过状态机/框架生命周期消除共享，就不急于增加自旋锁；若未来动态改频率需明确锁和上下文。
- 常见误区：说“信号量不能用于中断”却不说明因为会睡眠；把用户态 semaphore 与内核 semaphore 混为一谈。
- 30 秒回答：自旋锁用于不可睡眠的短临界区，mutex/semaphore 会睡眠，不能在硬中断。项目通过禁止 buffer 运行时改采样率，避免 timer period 并发修改。
- 2 分钟展开：说明 `spin_lock_irqsave`、mutex、completion、atomic_t 和 RCU 的适用边界。

B04 | 原题 202、203、205：内核内存与 MMU

内核常见分配包括 `kmalloc`（物理连续的小对象）、`vmalloc`（虚拟连续）、页分配器和 slab；GFP flags 表达是否可睡眠等上下文约束。P5 大部分资源使用 `devm_*`，底层仍依赖内核分配器，但项目没有实现内存管理器。

MMU 提供虚拟到物理映射、权限和隔离。驱动中的设备地址、DMA 地址与用户虚拟地址不能混用。

3. 模块、设备模型与设备树

A04 | 原题 207：模块加载、卸载与生命周期

- 一句话结论：模块加载执行注册入口，卸载执行注销出口；真正设计重点是引用关系、并发使用者和逆序释放，而不是只背 `module_init/module_exit`。
- 从零解释：注册 driver 后，匹配设备会调用 `probe`；注销时会触发 `remove`。模块仍被使用时不能安全卸载。
- 底层机制：P5 用 `module_spi_driver` 生成注册/注销样板，`probe/remove` 处理设备实例。
- 项目场景：驱动注册 IIO device 与 trigger；remove 先注销对外接口，再让 devm 回收资源。
- 代码证据：P5 `kernel/icm20608_iio.c`。证据：CODE、CROSS，加载卸载为 TODO-HIL。
- 工程取舍：devm 减少 `probe` 失败分支，但对外可见对象的注销顺序仍要明确。
- 常见误区：认为 devm 意味 `remove` 什么都不用做；把对象编译成功写成 `insmod` 成功。
- 30 秒回答：项目用 `module_spi_driver` 注册，设备匹配后 `probe`，卸载时 `remove`。当前只做 vendor 4.1.15 对象编译，没有声称完成 `insmod/rmmod`。
- 2 分钟展开：说明注册驱动与创建设备的区别、引用计数和卸载竞态。

B05 | 原题 208~211：字符/块/网络设备和 cdev

字符设备按字节/请求访问，块设备面向可随机访问块并进入块层，网络设备通过 `net_device` 与网络栈交互。传统字符设备涉及 `dev_t`、`cdev`、`class/device` 和 `file_operations`。

P5 没有自建 `cdev`，而是注册 IIO device，由 IIO core 提供 `sysfs` 和 `/dev/iio:deviceX`。因此这些题必须标为 **RELATED**：理解框架差异，但不能把 IIO core 的工作说成自己手写了 `cdev`。

A05 | 原题 212：总线-设备-驱动模型

- 一句话结论：字符设备描述用户 ABI；总线设备驱动模型描述设备发现、匹配、`probe/remove` 和电源/生命周期，两者可在一个驱动中同时出现。
- 从零解释：SPI core 管理 `spi_device` 与 `spi_driver` 匹配；IIO core 管理传感器 ABI。
- 底层机制：设备树创建 SPI device，`of_match_table` 匹配 `compatible`，core 调 `probe`。
- 项目场景：P5 是 SPI driver + IIO device，不是简单的“字符驱动”。
- 代码证据：P5
`icm20608_of_match`、`icm20608_driver`、`icm20608_probe`。证据：**CODE**、**CROSS**。
- 工程取舍：复用标准子系统能获得一致 ABI 和工具生态，减少私有 `ioctl`。
- 常见误区：说设备树直接“加载模块”；它主要描述硬件并参与匹配。
- 30 秒回答：SPI 总线模型解决设备和驱动如何匹配，IIO 解决传感器怎样暴露给用户；这两个维度不是二选一。
- 2 分钟展开：画 `bus/device/driver` 与 `class/subsystem` 的关系。

A06 | 设备树到 probe 的完整路径

- 一句话结论：DTS 描述板级连接，内核解析后创建设备，`compatible` 与驱动匹配表相交才调用 `probe`。
- 从零解释：节点包含控制器父子关系、片选、最大频率等资源，不应放业务阈值。
- 底层机制：ECSPI3 控制器先就绪，子节点生成 `spi_device`；SPI core 设置模式/频率并匹配 `driver`。
- 项目场景：P5 提供 ALIENTEK 板级 `dtsti` fragment，`compatible` 为 `invensense, icm20608`。
- 代码证据：P5
`deploy/*.dtsti`、`kernel/icm20608_iio.c::of_match`。证据：**CODE**、**CROSS**，真实 DTB 为 **TODO-HIL**。
- 工程取舍：板级引脚/片选留在 DTS，芯片通用逻辑留在 `driver`。
- 常见误区：`compatible` 随意写成文件名；驱动代码硬编码板级 GPIO。
- 30 秒回答：设备树描述 ICM20608 挂在 ECSPI3 的片选和频率，SPI core 创建 device，`compatible` 匹配后进入 `probe`。

- 2 分钟展开：说明 controller probe 顺序、deferred probe 和资源获取失败。

A07 | probe 为什么先读 WHO_AM_I

- 一句话结论：总线通信成功且芯片身份正确是建立后续 ABI 的前提；失败必须让 probe 返回错误，不能创建一个假的可用设备。
- 从零解释：设备节点存在只说明描述存在，不证明焊接、片选、模式和供电正确。
- 底层机制：regmap read 返回通信状态和值，驱动检查 0xAF，再执行复位和配置。
- 项目场景：P5 icm20608_hw_init 失败直接终止 probe。
- 代码证据：P5 kernel/icm20608_iio.c。证据：CODE、CROSS；真实返回值为 TODO-HIL。
- 工程取舍：fail-fast 避免用户态看到节点后读取垃圾；日志包含期望/实际值便于定位。
- 常见误区：忽略返回码只比较值；失败后仍注册设备。
- 30 秒回答：probe 先通过 regmap 读取 WHO_AM_I=0xAF，任何通信或身份错误都返回，不让 IIO 节点处于半初始化状态。
- 2 分钟展开：列出供电、片选、CPOL/CPHA、频率、复位时序的排查顺序。

A08 | devm 与 probe 失败回滚

- 一句话结论：devm 把资源释放绑定到 device 生命周期，简化多出口回滚；但注册顺序、外部可见状态和非 devm 资源仍需设计。
- 从零解释：probe 第五步失败时，前四步资源必须释放，手写 goto 容易遗漏。
- 底层机制：device-managed action 在 probe 失败或 device detach 时逆序执行。
- 项目场景：P5 用 devm 分配 IIO device、regmap 和 trigger，错误码逐层返回。
- 代码证据：P5 icm20608_probe。证据：CODE、CROSS。
- 工程取舍：devm 提高清晰度；需要早于 device detach 释放或复杂共享的资源可能仍手工管理。
- 常见误区：把所有指针都 devm 后忽略用户并发；重复手工 free devm 资源。
- 30 秒回答：probe 每步检查返回码，devm 负责已取得资源的逆序回收，因此中途失败不会留下半初始化内存；IIO 对外注册仍保持清晰顺序。
- 2 分钟展开：对比 goto unwind，并说明为什么 remove 仍要处理对外状态。

4. SPI、regmap 与 IIO

A09 | 为什么使用 regmap

- 一句话结论：regmap 把寄存器宽度、值宽度、读标志和访问 API 统一起来，隔离总线协议细节。
- 从零解释：SPI 芯片常在寄存器地址高位编码读写方向，散落手工拼字节会重复且易错。

- 底层机制：配置 `reg_bits/val_bits/read_flag_mask`，调用 `regmap_read/write/bulk_read`。
- 项目场景：P5 采用 8 位寄存器、8 位值和 SPI 读标志，连续读取 14 字节原始数据。
- 代码证据：P5 `icm20608_regmap_config`、trigger handler。证据：CODE、CROSS。
- 工程取舍：regmap 有抽象开销但换来统一错误处理和可维护性；高频路径使用 `bulk_read` 减少事务。
- 常见误区：把 regmap 说成一种总线；它是寄存器映射抽象，可后端接 SPI/I2C/MMIO。
- 30 秒回答：我用 regmap 集中描述 8 位寄存器和值以及 SPI 读标志，直读和 buffer 都不再重复拼总线帧。
- 2 分钟展开：说明 `volatile/precious register`、cache 选项在本项目为何未扩展。

A10 | 为什么选择 IIO 而不是私有 ioctl

- 一句话结论：IMU 属于标准工业 I/O 传感器，IIO 已提供 `channel`、`raw/scale/offset`、`trigger` 和 `buffer ABI`，复用比自定义字符协议更可维护。
- 从零解释：`raw` 是寄存器读数，`scale/offset` 把它转换为物理量；`buffer` 负责连续采样。
- 底层机制：`iio_chan_spec` 描述通道，`iio_info` 处理属性，IIO core 创建 `sysfs` 和设备节点。
- 项目场景：P5 暴露 `accel`、`anglvel`、`temp` 和 `timestamp`。
- 代码证据：P5 `kernel/icm20608_iio.c` `channel` 表与 `read_raw`。证据：CODE、CROSS。
- 工程取舍：标准 ABI
降低用户态耦合；特殊高性能协议仍可能需要其他接口，但必须有证据。
- 常见误区：认为用了 IIO 就完全不用理解字符设备；IIO core 仍通过内核文件接口提供数据。
- 30 秒回答：我没有重复写私有 `ioctl`，而是用 IIO 表达 `raw/scale/采样率` 和 `buffer`，使用户态按标准 `sysfs` 与设备节点工作。
- 2 分钟展开：比较 `hwmon`、`input`、IIO 和自定义 `cdev` 的选择依据。

A11 | direct mode 与 buffer 的互斥

- 一句话结论：直接读取和连续 `buffer` 不能无约束同时访问设备，驱动要用 IIO `direct mode` 保护或在运行状态拒绝冲突配置。
- 从零解释：一次 `sysfs raw read` 若与定时 `bulk read` 同时改变/读取寄存器，可能破坏采样一致性。
- 底层机制：`claim direct mode` 检查 `buffer` 是否活跃；运行中修改 `sampling_frequency` 返回 `-EBUSY`。
- 项目场景：P5 L1 先实现 `direct read`，L4 加 `buffer` 后补互斥边界。
- 代码证据：P5 `read_raw/write_raw`。证据：CODE、CROSS。

- 工程取舍：拒绝运行中改频率简化并发；更复杂动态更新需停止 timer、加锁并重启。
- 常见误区：sysfs 写成功就立即无锁改 hrtimer period。
- 30 秒回答：buffer 开启时 direct/config 操作要受限。我选择运行中改采样率返回 EBUSY，保证 timer 周期与 scan 布局一致。
- 2 分钟展开：说明直接模式、buffer enable 和配置事务顺序。

A12 | IIO scan、时间戳与信息泄漏

- 一句话结论：scan 布局由启用通道、storage bits、endianness 和对齐共同决定；padding 必须初始化，用户态按实际布局解析。
- 从零解释：七个 16 位值共 14 字节，8 字节时间戳放在偏移 16，结构大小 24。
- 底层机制：内核把整块 scan 推入 buffer；未初始化的 2 字节 padding 若不清零可能泄漏内核数据。
- 项目场景：P5 available_scan_masks 只允许完整布局，scan 每次 memset 后填值和时间戳。
- 代码证据：P5 kernel/icm20608_iio.c。证据：CODE、CROSS。
- 工程取舍：固定全通道简化用户态；未来支持任意 mask 必须动态计算偏移。
- 常见误区：把 14+8 算成 22；忽略 timestamp 对齐；用户态硬编码设备号和布局却不校验。
- 30 秒回答：IIO 布局实际是 24 字节：14 字节通道、2 字节 padding、8 字节时间戳。我 push 前清零整个 scan，避免泄漏未初始化内核内存。
- 2 分钟展开：手画字节偏移并关联用户态 pending parser。

A13 | sysfs 与 /dev/iio:deviceX

- 一句话结论：sysfs 适合低频配置/属性，字符设备 buffer 适合连续二进制数据；设备编号不稳定，应按 name 枚举。
- 从零解释：sysfs 是内核对象属性的文本视图，不是普通配置数据库。
- 底层机制：IIO core 根据注册设备和 channel 创建属性；probe 顺序决定 deviceX。
- 项目场景：P5 findDevice 遍历 name，configureBuffer 先 disable、配置全部项、最后 enable。
- 代码证据：P5 libiio/src/iio_sysfs.cpp 与 host fake-sysfs test。证据：CODE、HOST。
- 工程取舍：配置步骤形成最小事务，失败回到 disabled；不写死 iio:device0。
- 常见误区：边运行边修改 scan_elements；一次失败留下半配置 enable 状态。
- 30 秒回答：用户态按 IIO name 找设备，不依赖编号；配置先关闭 buffer，设置频率/长度/scan elements，最后开启，异常时尝试恢复关闭。
- 2 分钟展开：说明 sysfs 单文件写并非跨多个属性原子，所以应用自己组织事务。

5. 用户态算法与内核边界

A14 | 为什么算法放用户态

- 一句话结论：内核提供稳定机制和 ABI，变化频繁、可失败的业务策略放用户态，更易测试和更新，也缩小内核故障面。
- 从零解释：倾斜阈值、连续样本窗口和告警优先级属于产品策略，不是硬件访问必需。
- 底层机制：驱动输出带时间戳物理量，`libcondition` 接收纯 Sample，无系统调用依赖。
- 项目场景：P5 状态机在 x86 主机运行相同输入得到确定输出。
- 代码证据：P5 `libcondition`、`tests`、`replay`。证据：`CODE`、`HOST`。
- 工程取舍：多一次用户/内核数据传递，换来隔离、可测试和发布灵活性。
- 常见误区：为了“实时”把所有计算放内核；阈值算法异常会扩大为内核崩溃。
- 30 秒回答：驱动只做发现、采样和标准 IIO ABI，倾斜/冲击/失联在 `libcondition`。这样算法可在 PC 回放测试，修改策略不需要重装内核模块。
- 2 分钟展开：说明何种极低延迟闭环才可能考虑内核/eBPF/RT 扩展。

A15 | 原题 206、213：交叉编译及两种驱动构建方式

- 一句话结论：交叉编译是在主机生成目标架构产物；内建与模块两种方式必须使用与目标运行内核匹配的配置、源码和工具链。
- 从零解释：x86 编译成功不代表 ARM 可运行；同为 ARM 还要匹配 ABI、glibc、内核版本和 `module vermagic`。
- 底层机制：CMake `toolchain` 指定 ARM `g++`；内核外部模块使用 `make -C KERNEL_SRC M=... ARCH=arm CROSS_COMPILE=...`。
- 项目场景：P5 ARM 用户服务由 Linaro GCC 4.9.4 构建为 EABI5；驱动针对 ALIENTEK vendor 4.1.15 编译对象。
- 代码证据：P5 `cmake/toolchains`、`scripts/validate_driver_source.sh`。证据：`CROSS`。
- 工程取舍：板端主体用 C++14 兼容旧工具链；不为追求 C++17 牺牲可部署性。
- 常见误区：`file` 显示 ARM 就等于能在板上运行；忽略动态加载器/rootfs 版本；对象文件等于 `.ko`。
- 30 秒回答：我用目标 Linaro 工具链得到 ARM EABI5 用户态 ELF，并用 vendor 4.1.15 头编译驱动对象；由于当前 `defconfig` 未启用 IIO buffer/trigger，仍不宣称 `.ko` 已联编加载。
- 2 分钟展开：说明 `sysroot`、ABI、`vermagic`、内建与模块的差异。

B06 | 原题 214~216: U-Boot 启动与参数传递

U-Boot 完成早期硬件准备、加载内核/DTB/rootfs 并通过 bootargs、设备树等传递参数。进入内核前关闭/整理中断、看门狗、cache、MMU 的具体要求依赖架构和内核启动协议，不能背成所有平台绝对规则。

P5 提供设备树 fragment，但没有修改 U-Boot，因此这些题为 **BASE/RELATED**。

6. P5 上板时的真实验证链

1. 应用 defconfig fragment，确认 IIO_BUFFER/TRIGGER 已启用
2. 合入 DTS，重新生成 DTB
3. 用目标内核配置编译 .ko，核对 vermagic
4. 启动后检查 dmesg 与 WHO_AM_I
5. 查找 /sys/bus/iio/devices/*/name
6. 读取 raw/scale/sampling_frequency
7. 配置 scan_elements 与 buffer
8. 运行 conditiond，验证真实 24 字节 scan
9. 倾斜/冲击/停流/拔除/SIGTERM 故障注入
10. 记录 CPU、内存、丢样和 8 小时长稳

只有第 1~10 步实际执行并留存版本/日志后，才能把 **TODO-HIL** 改为 **HIL**。

7. 本章掌握度检查

- 能否从 DTS 节点一路讲到 **icm20608_probe**?
- 能否解释 hrtimer 为什么不能直接读 SPI?
- 能否说明 IIO 相比私有 cdev 的工程收益，而不是说“框架更高级”?
- 能否手画 24 字节 scan 布局并解释 padding 清零?
- 能否准确说出当前通过的是对象交叉编译，而不是 **.ko** 上板?

06 | 五个项目怎样讲成可信的工程故事

项目介绍不是把技术名词串成一段，而是回答：产品问题是什么、原设计哪里会失败、我怎样重构、用什么证据证明、还剩什么边界。

P1 | 手持体温检测仪

产品与数据流



目标不是“读取几个传感器”，而是只有新鲜温度、卡片在场和非零 UID 同时成立时，才形成有效业务记录。

重构前的风险

1. 空的 SVC handler 破坏 FreeRTOS 首任务启动链。
2. 队列满时行为不明确，生产任务可能被旧数据拖住。
3. 临界区包围慢 RTC/I2C 操作会增加中断延迟。
4. Flash CRC 覆盖不完整或版本校验不足，损坏配置可能被误用。
5. ADC/SPI 自检用假 PASS，使诊断结论不可信。
6. 默认配置和日志可能泄露网络凭据。

重构后的结构

- SVC_Handler → vPortSVCHandler，恢复 RTOS 正确启动路径。
- prvSendLatest 在队列满时丢旧保新，确保传感器链路有界。
- I2C 用 mutex，短临界区只复制标量，RTC 轮询移到外部。
- Flash 记录用 magic/version/tail/完整 CRC，自检无能力的项目明确 SKIP。
- FusionPolicy_IsMeasurementReady 成为纯函数并接受宿主测试。
- 仓库不存 SSID/密码，未配置网络时显式失败。

代码证据

证据	当前代码	固定证据
RTOS 启动	<code>main</code>	<code>study-step-1</code>
最新值队列	<code>main</code>	<code>study-step-2</code>
融合与测试	<code>fusion policy</code>	<code>host test</code>

证据等级：CODE、HOST；外设真实时序与 WiFi 链路为 TODO-HIL。

30 秒介绍

这是 STM32F103 + FreeRTOS

的多源体温检测仪。我重构的核心是运行时正确性和数据一致性：修正 SVC 启动入口，传感器用有界最新值队列，I2C 用 mutex，临界区只保护短快照；温度与有效 RFID 同时满足才生成记录。Flash 加入版本和完整 CRC，敏感配置不入库。融合策略与 CRC 已通过宿主测试，外设时序仍按 HIL 管理。

2 分钟展开

先说业务记录为什么不能复用旧 UID，再画任务/队列数据流；解释 queue/mutex/critical section 的选择；最后讲 CRC

只能检测损坏，若继续提高掉电一致性会增加双槽、序号和提交标志。

典型追问

- 为什么不用全局变量？ 队列同时传递数据和同步，所有权与新鲜度更清楚。
- CRC 能保证原子写吗？ 不能，只能检测；原子提交需双槽或日志结构。
- 为什么队列满丢旧？ 测温显示重视最新值；关键记录不能用同一策略。
- mutex 和临界区怎么选？ 慢总线用 mutex，几个标量复制用短临界区。

不能说的内容

不能说已经证明传感器精度、WiFi 长稳、Flash 掉电原子性或整机量产可靠性。

P2 | 智能送药小车

产品与数据流

```

OpenMV  USART  ISR  -> 最新视觉帧  ┌
HX711 任务  ──────────> 最新重量  ───┴──> 逻辑输入 -> 非阻塞 FSM -> 单一控制任务 -> 电机
灰度传感器  ───────────────────────────┘
  
```

重构前的风险

- ISR 里混入业务判断，延迟不可控。
- 多任务可能同时写电机，输出取决于抢占时序。
- 长 delay 完成转向，无法及时响应新事件。
- HX711 断线会无限等待 DOUT。
- Tick 绝对值比较在回绕时失效。

重构后的结构

- USART ISR 只收两字节、组帧、`xQueueOverwriteFromISR`。
- `vControlTask` 是电机唯一写入者。
- `CartState_Step` 以状态和进入时间推进转向/提示/掉头。
- HX711 所有等待有超时并返回有效位。

- `prvElapsed` 使用无符号差值，测试覆盖回绕。

代码证据

证据	当前代码	固定证据
ISR 边界	ISR	study-step-1
FSM	cart_state.c	study-step-2
超时	hx711.c	host test

证据等级：**CODE**、**HOST**；机械参数、PID/阈值和 HX711 标定为 **TODO-HIL**。

30 秒介绍

这是 STM32F103 + FreeRTOS 的视觉循迹送药车。我把 USART 中断缩到组帧和队列通知，电机只由控制任务写入，再用时间驱动 FSM 替代长阻塞 `delay`。HX711 增加超时，Tick 计算覆盖回绕。宿主测试验证状态转移和输出命令，但实际转向时长、重量标定和循迹阈值仍需实车标定。

2 分钟展开

重点讲“单写者”如何消除竞争、FSM 如何提升可测试性，再给出 Tick 回绕数值例子。最后解释为什么 `host test` 不能验证机械惯量。

典型追问

- 为什么长度 1 队列？视觉/重量是状态观测，处理旧样本没有价值。
- 为什么不用多个任务分别控制左右电机？
组合动作需要一致输出，单写者更容易维持不变量。
- 超时后怎么办？返回 `invalid`，状态机进入安全/保持策略；具体安全动作需产品定义。
- FSM 会不会太复杂？
状态增加，但逻辑可枚举、可测试、可中断，复杂度从时间隐式关系变成显式结构。

P3 | 三维 LiDAR 感知

产品与数据流

```

UDP socket / PCAP
  ↓
Input 层 (epoll、长度/来源检查)
  ↓
BoundedQueue<packet> -- DropOldest / Block
  ↓
解析 / 点云处理 / ROS 发布
  ↓
metrics 快照与受控 shutdown

```

重构前的风险

- `socket/epoll` 构造中途失败可能泄漏 `fd`。
- PCAP 截断帧按固定偏移读取，EOF 使用 `abort`。

- 无界队列把持续过载变成内存增长和延迟失控。
- 条件变量未把 shutdown 放进谓词，线程退出可能挂住。
- 指标跨线程读写普通结构体，存在 data race。

重构后的结构

- 非阻塞 socket 和 epoll，所有失败分支与析构配对关闭。
- PCAP 对 caplen、过滤结果和 EOF 返回状态。
- 通用 **BoundedQueue** 支持 Block/RejectNewest/DropOldest。
- push/pop 用带谓词 condition_variable，shutdown notify_all。
- 指标在同一 mutex 下快照；流水线按依赖顺序 stop/join。

代码证据

证据	当前代码	固定证据
I/O 生命周期	input.cc	study-step-1
有界队列	bounded_queue.h	study-step-2
测试	test_bounded_queue.cpp	同路径

证据等级：**CODE**、**HOST**；ROS/PCL 全量构建和 LS1550/Jetson 性能为 **TODO-HIL**。

30 秒介绍

这是 ROS1/PCL 的 LiDAR 数据接入与点云流水线。我重构了 UDP/epoll 生命周期、PCAP 边界检查和并发队列：所有 fd 失败路径可回收，级间队列有容量与 DropOldest/Block 策略，shutdown 进入条件变量谓词，线程可被唤醒并 join。宿主测试覆盖过载和关闭语义，没有设备时不声称真实吞吐和丢包率。

2 分钟展开

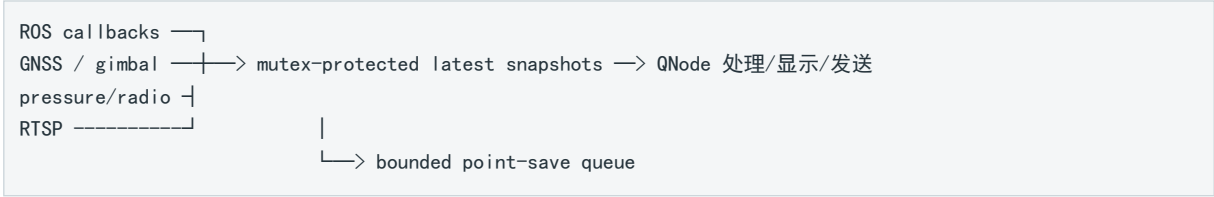
先说明实时感知为什么选择丢旧，再讲 condition_variable 谓词与 stop 顺序；最后解释单 socket 使用 epoll 是架构一致性，而不是性能宣传。

典型追问

- epoll 一定比 poll 好？ 不；当前单 fd 收益小，主要为了扩展统一事件循环。
- DropOldest 会不会丢关键数据？ 会，因此只用于可被新帧替代的实时观测，且要记录 dropped。
- 为什么 EOF 不能 abort？ EOF 是离线输入生命周期的一部分，需让上层正常收尾。
- 如何验证没有 fd 泄漏？ 重复构造/销毁、错误注入并监控 `/proc/pid/fd`。

P4 | 多源光电数据融合

产品与数据流



重构前的风险

- detached spinner/工作线程可能在对象析构后继续访问。
- ROS AsyncSpinner 与 spinOnce 混用，回调所有权不清。
- 复合结果普通读写导致 data race 和跨帧字段。
- 四元数 x/y/z 被直接当欧拉角。
- RTSP 断流立即循环重连形成风暴。
- 点云落盘速度慢时队列可能无界增长。

重构后的结构

- QNode 成为 ROS 回调单一所有者，不使用 detached 工作线程。
- 单值控制标志用 atomic，复合传感器结果用 mutex 快照。
- tf2 正确完成 quaternion→Euler。
- RTSP 使用 250 ms 到 5 s 的封顶指数退避。
- 点云保存队列最多 3 帧，满时丢最旧。
- 所有线程 stop→join/wait→释放设备。

代码证据

证据	当前代码	固定证据
生命周期与快照	qnode.cpp	study-step-1
RTSP 退避	rtsp_capture.cpp	study-step-2
测试	host test	同路径

证据等级：CODE、HOST；ROS/Qt/OpenCV/PCL/串口/RTSP 联调为 TODO-HIL。

30 秒介绍

这是 ROS、Qt、RTSP、串口与点云融合应用。重构重点是线程生命周期和快照一致性：移除 detached 与混合回调调度，单值标志用 atomic，复合传感器结果用 mutex 整体复制；RTSP 使用封顶退避，点云落盘队列有容量。宿主测试覆盖退避，完整依赖环境和真实时间同步仍需目标系统验证。

2 分钟展开

用“GNSS 经度来自新帧、纬度来自旧帧”的反例讲快照；再用 destructor 与 detached 的时序讲悬空；最后说明为什么重连等待分小步以响应 stop。

典型追问

- atomic 能代替 mutex 吗？只能保证单对象，不能保证一组字段不变量。
- 为什么不 detach？成员线程必须受对象生命周期控制，否则会 use-after-free。
- 为什么点云队列只有 3？限制慢磁盘对内存和实时链路的影响，保留较新帧。
- 四元数为什么不能直接取 xyz？xyz 是虚部，不是欧拉角；需按旋转矩阵转换并明确坐标系。

P5 | I.MX6ULL 设备姿态与异常冲击监测器

产品边界

这是基于正点原子 I.MX6ULL 学习经历独立构建的教学/作品项目，场景是仓储搬运车或户外控制箱的姿态、冲击和传感器失联监测。它不是公司量产经历，也不是预测性维护 AI。

分层结构

```

DTS / ECSP13
  ↓
SPI driver + regmap
  ↓
IIO direct / trigger / buffer
  ↓
sysfs + /dev/iio:deviceX
  ↓
conditiond: epoll(IIO, timerfd, signalfd)
  ↓
libcondition: tilt / impact / sensor_fault
  
```

为什么这样构建

1. 先做标准 IIO direct mode，证明设备模型和 ABI。
2. 把算法拆成纯库，用确定 CSV 测试，不等硬件。
3. sysfs 设备发现按 name，不写死 device0；配置形成可回滚事务。
4. 没有 DRDY 线时用 hrtimer trigger，但 SPI 读放线程化 handler。
5. 用户态用 epoll 收口三个 fd，减少线程与退出复杂度。
6. 板端选择 C++14，服从 Linaro GCC 4.9，而不是追逐 C++17。

代码证据

层级	当前代码	固定证据
L1 IIO direct	driver	study-step-1
L2 算法回放	libcondition	study-step-2
L3 sysfs	libiio	study-step-3
L4 trigger/buffer	driver	study-step-4
L5 epoll	conditiond	study-step-5

证据等级：CODE、HOST、CROSS；可加载 .ko 和实板采样为 TODO-HIL。

30 秒介绍

这是我基于 I.MX6ULL 学习经历独立构建的 Linux 驱动作品。ICM20608 通过 SPI/regmap 接入 IIO, hrtimer 只发 trigger, 可能睡眠的 bulk read 在线程化 handler; 用户态按 sysfs name 找设备, 用 epoll 统一 IIO、timerfd、signalfd, 纯状态机判断倾斜、冲击和失联。Linux 4/4 测试、ARM 用户态和 vendor 4.1.15 驱动对象交叉编译通过, .ko 与实板仍明确待验证。

2 分钟展开

先讲为什么从私有字符例程升级到 IIO, 再画 timer→threaded handler→buffer→epoll; 随后解释内核/用户策略分层和 C++14 工具链取舍; 最后诚实说明源码级与 HIL 的差距。

典型追问

- 为什么不用字符设备? IIO 已有传感器 channel/scale/trigger/buffer ABI。
- 为什么 timer 里不读 SPI? hrtimer 是原子上下文, SPI 可能睡眠。
- 为什么用户态不写死 device0? probe 顺序不稳定, 按 name 枚举。
- 为什么算法不放内核? 策略变化快、易失败, 用户态更可测且隔离更好。
- 你真的上板了吗? 没有; 当前证据是 host 与 source/cross, HIL 计划写在验证矩阵。

五项目横向对比

主题	P1	P2	P3	P4	P5
事件入口	传感器任务	USART ISR/周期任务	UDP/PCAP	ROS/串口/RTSP	hrtimer/IIO fd
并发边界	FreeRTOS queue/mutex	ISR→queue→单控 制任务	thread+bounded queue	atomic/mutex snapshot	单线程 epoll + threaded handler
过载策略	丢旧保新	overwrite 最新值	DropOldest/Block	3 帧点云队列	drain + pending
生命周期	RTOS 创建断言/hooks	传感器超时/FSM	shutdown+join	stop+join	RAII/devm
主要验证	融合/CRC host	FSM/tick host	queue host	backoff host	host+ARM cross+driver object
未验证	外设/WiFi	实车参数	ROS/设备吞吐	完整联调	.ko/HIL

面试时的真实性规则

1. “我重构了”后必须能指出文件、函数和验证。
2. “我选择了”后必须说明被放弃方案和代价。
3. “测试通过”后必须说明测试级别。
4. “没有做过”之后可解释迁移方案，但不能改口成“项目里就是这样”。
5. 任何精度、吞吐、延迟、长稳数字都要有实验记录。

07 | 原 216 题分级映射与双向索引

使用说明

- **A**: 有项目代码证据，正文给出项目化深讲。
- **B**: 相关知识，可解释迁移关系，但不声称已经实现。
- **C**: 通用基础，保留正确回答与边界，不强行绑定项目。
- **P1~P5** 的含义见总目录。多个题由同一主题卡覆盖，例如 40~43 统一落到“指针、引用与传参”。

文本提取把原文开头三行序言误识别为题号 1~3；本索引已人工校正为 `static`、`extern`、`extern "C"`。

一、C/C++ (1~71)

题号	原题主题	级别	项目/补充章节
1	<code>static</code> 的作用	B	01: 编译单元与链接
2	<code>extern</code> 的作用	B	01: 编译单元与链接
3	<code>extern "C"</code>	B	01: C/C++ ABI
4	<code>const</code> 使用场景	A	P3/P4/P5; 01-A01
5	<code>const</code> 与 <code>define</code>	B	01: 类型系统与预处理
6	<code>volatile</code> 作用	A	P1/P2/P4; 01-A02
7	<code>volatile</code> 用法	A	P1/P2/P4; 01-A02
8	<code>strlen</code> / <code>strcpy</code> / <code>strstr</code> / <code>strcat</code> / <code>strcmp</code>	B	01: 边界与字符串安全
9	<code>memset</code> / <code>memcpy</code> / <code>memmove</code> / <code>memcmp</code>	B	P4/P5; 01: 对象表示
10	<code>atoi</code> / <code>atof</code>	B	01: 解析与错误报告
11	MIN 宏	B	01: 宏副作用与 <code>inline</code> / <code>template</code>
12	无临时变量交换宏	C	10: 不推荐技巧题

题号	原题主题	级别	项目/补充章节
13	宏求数组元素数	B	01: 数组退化与模板
14	C 访问寄存器	A	P1/P2/P5; 01-A03
15	定义、声明及报错阶段	B	01: 编译/链接
16	不用第三变量交换	C	10: 可读性优先
17	指定位清零	B	01: 位操作
18	指定位反转	B	01: 位操作
19	为什么少用 <code>goto</code>	B	01: 结构化控制与内核例外

题号	原题主题	级别	项目/补充章节
20	死循环写法	B	02: 任务循环与阻塞
21	C 内存分配/分区	A	P1/P2/P5; 01-A04
22	堆与栈	A	五项目; 01-A05
23	大局部数组	A	P1/P2; 01-A05
24	malloc 超过物理内存	B	03: 虚拟内存/overcommit

题号	原题主题	级别	项目/补充章节
25	malloc 与 new	A	P3/P4/P5; 01-A06
26	malloc/free 原理	B	01: 分配器边界
27	new/delete 原理	B	01: 对象生命周期
28	delete this	B	01: 自毁对象风险
29	为什么需要 new/delete	A	P3/P4/P5; 01-A06
30	类成员内存分布	B	01: 对象模型
31	类大小	B	01: 对齐/vptr/空类
32	内存泄漏检测与避免	A	P3/P4/P5; 01-A07
33	字节对齐	A	P4/P5; 01-A08
34	二级指针用途	B	01: 修改指针/二维结构
35	指针常量与常量指针	B	01: const 组合
36	a 与 &a	B	01: 类型和步长

题号	原题主题	级别	项目/补充章节
37	数组名和指针	B	01: 数组退化
38	数组指针和指针数组	B	01: 声明解析
39	函数指针和指针函数	B	P1 回调; 01
40	指针和引用异同	A	P5; 01-A09
41	为什么需要引用	A	P5; 01-A09
42	值/指针/引用传参	A	P3/P4/P5; 01-A09
43	指针/引用传参底层	A	P5; 01-A09
44	野指针	A	P4; 01-A10
45	避免野指针	A	P4/P5; 01-A10
46	智能指针	A	P5; 01-A11
47	常用智能指针	A	P3/P4/P5; 01-A11
48	RAII	A	P3/P4/P5; 01-A11

题号	原题主题	级别	项目/补充章节
49	指针大小	B	01: ABI/目标架构

题号	原题主题	级别	项目/补充章节
50	绝对地址访问	B	P1/P2/P5; 01-A03 (RELATED)
51	面向对象基本特征	B	P3; 01: 组合优先
52	重载/重写/隐藏	B	P3 输入接口; 01
53	访问与继承权限	B	01: 对象模型
54	构造函数类型	A	P5; 01-A12
55	构造/拷贝/赋值	A	P5; 01-A12
56	基类虚析构	B	P3; 01: 多态边界
57	构造函数不能虚	B	01: 对象建立阶段
58	构造/析构顺序	A	P4/P5; 01-A12
59	深拷贝与浅拷贝	A	P5 UniqueFd; 01-A12
60	初始化列表效率	B	01: 直接初始化
题号	原题主题	级别	项目/补充章节
61	必须使用初始化列表	B	01: const/reference/base
62	this 指针	B	P4 生命周期; 01
63	多态底层原理	B	P3; 01
64	虚函数表存储	B	01: ABI 非标准细节
65	虚函数	B	P3 输入接口; 01
66	哪些函数不能虚	B	01
67	纯虚函数与虚函数	B	P3; 01
68	容器性质	A	P3/P5; 01-A13
69	C++11 新特性	A	P3/P4/P5; 01-A14
70	左右值引用	A	P5 UniqueFd; 01-A15
71	Lambda	A	P4 生命周期关联; 01-A16

二、硬件、ARM 与 FreeRTOS (72~99)

题号	原题主题	级别	项目/补充章节
72	ARM 寄存器数量	B	P1/P2/P5; 02-B01
73	CPSR/SPSR	B	ARM 模式基础; 02-B01
74	ARMv8 创新	C	02: ARM 版本边界; 当前硬件非 ARMv8
75	中断产生与处理	A	P1/P2; 02-A01
76	中断向量与嵌套	A	P1/P2; 02-A01
77	中断优缺点	A	P2; 02-A02
78	软中断	A	P1 SVC; 02-A03
79	软中断与函数调用	A	P1; 02-A03
80	中断参数和返回	A	P2; 02-A02
81	main/ISR 共享变量	A	P1/P2; 02-A04
82	main/ISR 共用函数	A	P1/P2; 02-A04
83	ISR 使用 printf	A	P2; 02-A02
题号	原题主题	级别	项目/补充章节
84	异常种类	B	Cortex-M 异常; 02
85	STM32F1 与 F4	C	02: MCU 系列边界; 当前项目均 F103
86	实时系统与 Linux	A	五项目; 02-A05
87	FreeRTOS 配置/中断	A	P1/P2; 02-A06
88	优先级反转	A	P1; 02-A07
89	FreeRTOS 调度	A	P1/P2; 02-A08
90	FreeRTOS IPC	A	P1/P2/P3; 02-A09/A10
91	接口/协议特点	B	P1/P2/P5; 02-B02
92	哈佛/冯诺依曼	C	02: 体系结构基础
93	存储系统	B	02: MCU/Linux 对比
94	寄存器和内存	B	01/02
95	Cache 一致性	B	03: 并发/架构边界
题号	原题主题	级别	项目/补充章节
96	NOR 与 NAND	C	02: 存储基础
97	SRAM/DRAM/SDRAM	C	02: 存储基础
98	RISC 与 CISC	C	02: 架构基础
99	ARM 流水线	C	02: 架构基础

三、操作系统、网络与算法（100~159）

题号	原题主题	级别	项目/补充章节
100	系统调用与库函数	A	P3/P5; 03-A01
101	进程状态	B	03-B01
102	Linux IPC	B	R0S/socket 关联; 03-B05
103	进程调度算法	B	03-B01
104	进程和线程	A	P4/P5; 03-A02
105	内核线程和用户线程	B	P5 handler 关联; 03-B02
106	进程/线程同步	A	P3/P4; 03-A03
107	死锁四条件	A	P2/P3/P4; 03-A07
108	预防死锁	A	P2/P3/P4; 03-A07
109	各类地址	B	P5; 03-B03
110	CPU/MMU/cache/内存协同	B	03-B03
111	非连续内存管理	C	03: 虚拟内存基础

题号	原题主题	级别	项目/补充章节
112	TLB	B	03-B03
113	虚拟内存	B	Linux 应用; 03-B03
114	虚拟内存目的	B	Linux 应用; 03-B03
115	OSI 七层	B	P3/P4; 04-B01
116	应用层协议	B	P4 RTSP; 04-B01
117	传输层协议	B	P3/P4; 04
118	网络层协议	B	P3/P4; 04
119	数据链路层协议	B	P3/P4; 04
120	HTTP 报文	C	04-C01
121	HTTP/1.0 与 1.1	C	04-C01
122	HTTP/2 与旧版本	C	04-C01
123	HTTP 特点	C	04-C01

题号	原题主题	级别	项目/补充章节
124	HTTPS 加密方式	C	04-C01
125	数字证书	C	04-C01
126	HTTPS 安全流程	C	04-C01
127	GET 与 POST	C	04-C01
128	Cookie/Session/Token	C	04-C01
129	DNS 原理	C	04-C01

题号	原题主题	级别	项目/补充章节
130	URL 到页面	C	04-C01
131	搜索网站涉及各层	C	04-C01
132	TCP 头部	A	P4 关联；04-A01
133	TCP 建连/断连	A	P4；04-A01
134	三握四挥原因	A	P4；04-A01
135	TIME_WAIT	B	P4 关联；04-B02

题号	原题主题	级别	项目/补充章节
136	2MSL	B	04-B02
137	拔网线后的 TCP	B	P4；04-B02
138	TCP 为什么可靠	A	P3/P4；04-A02
139	TCP 与 UDP	A	P3/P4；04-A02
140	顺序表与链表	C	04：数据结构边界
141	链表有环	C	04：数据结构边界；快慢指针
142	双向链表插入	C	04：数据结构边界；指针操作
143	数据接收处理题	B	P2/P3/P5 framing；04：流式解析
144	循环队列	B	P3 有界队列对比；04：数据结构边界
145	二叉树分类	C	04：数据结构与算法边界
146	哈夫曼树/编码	C	04：数据结构与算法边界
147	二叉树退化	C	04：数据结构与算法边界

题号	原题主题	级别	项目/补充章节
148	红黑树	C	04：数据结构与算法边界
149	红黑树优缺点	C	04：数据结构与算法边界
150	B+ 树	C	04：数据结构与算法边界
151	图的存储	C	04：数据结构与算法边界
152	图遍历	C	04：数据结构与算法边界
153	最小生成树	C	04：数据结构与算法边界
154	最短路径	C	04：数据结构与算法边界
155	哈希冲突	C	04：数据结构与算法边界
156	哈希结构	C	04：数据结构与算法边界
157	排序/快排	C	04：数据结构与算法边界
158	快排最坏情况	C	04：数据结构与算法边界
159	大根堆/堆排序	C	04：数据结构与算法边界

四、嵌入式 Linux (160~216)

题号	原题主题	级别	项目/补充章节
160	段错误定位	A	P4; 03-A10
161	常用 Linux 命令	B	03-B06/09
162	查看内存	B	03: <code>/proc</code> 与资源观察; <code>free</code> 、 <code>top</code>
163	查看磁盘	B	03: 文件系统观察; <code>df/du/lslblk</code>
164	Vim 模式和命令	C	09: 实验工具边界
165	GCC 四步骤	B	P5 CROSS; 03-B06
166	动态库与静态库	B	P3/P4/P5; 03-B06
167	Makefile 规则	B	P5 kernel Makefile; 05: Kbuild
168	软/硬链接	B	03-B06
169	文件描述符	A	P3/P5; 03-A11
170	目录项和 inode	A	P5 设备节点; 03-A11
171	<code>/proc/<pid></code>	A	P3/P4/P5 验证; 03-A12

题号	原题主题	级别	项目/补充章节
172	进程地址空间	B	03-B03
173	进程组/作业/会话	B	03: 进程生命周期; <code>systemd</code> 服务关联
174	<code>fork</code>	B	03-B04 (未使用)
175	<code>fork</code> 继承资源	B	<code>fd</code> 语义; 03-B04
176	<code>exec</code>	B	03: 进程生命周期; <code>CLONEEXEC</code> 关联
177	孤儿/僵尸/守护进程	B	03: 进程生命周期; <code>systemd</code> 对比
178	<code>wait</code>	B	03-B04
179	清理僵尸	B	03-B04
180	管程	A	P3 BoundedQueue; 03-A13
181	网络字节序	A	P3/P4/P5; 04-A03
182	<code>socket</code> 客户/服务端流程	A	P3; 04-A04
183	并发服务器进程/线程	B	P5 <code>epoll</code> 对比; 04-B03

题号	原题主题	级别	项目/补充章节
184	I/O 多路复用	A	P3/P5; 04-A05
185	<code>select</code>	A	对比证据; 04-A06
186	<code>epoll</code>	A	P3/P5; 04-A07
187	<code>epoll</code> ET/LT	A	P5; 04-A08
188	<code>select/poll/epoll</code>	A	P3/P5; 04-A06
189	线程池	B	03: 线程模型; P3 pipeline 对比

题号	原题主题	级别	项目/补充章节
190	内核五大子系统	B	P5; 05-B01
191	内核编译安装	B	P5 上板路径; 05-B02
192	内核启动	B	P5; 05-B02
193	上下文概念	A	P5; 05-A01
194	内核态/用户态	A	P5 分层; 05-A01
195	用户/内核通信	B	P5 IIO/sysfs; 05-B03
题号	原题主题	级别	项目/补充章节
196	open 到内核路径	B	P5; 05-B03
197	中断上下半部原因	A	P5; 05-A02
198	下半部实现	A	P5 threaded handler; 05-A02
199	内核同步方式	A	P5; 05-A03
200	自旋锁/信号量能否睡眠	A	P5; 05-A03
201	自旋锁/信号量用于中断	A	P5; 05-A03
202	内存资源管控	B	P5 devm; 05-B04
203	MMU 作用	B	05-B04
204	线程栈是否相同	A	P4/P5; 03-A02
205	内核内存申请	B	P5 devm; 05-B04
206	驱动编译两种方法	A	P5; 05-A15
207	insmod/rmmod 执行函数	A	P5; 05-A04
题号	原题主题	级别	项目/补充章节
208	驱动分类	B	P5 IIO; 05-B05
209	字符设备框架	B	P5 对比; 05-B05
210	注册字符设备	B	未直接实现; 05-B05
211	/dev 设备文件创建	B	P5 IIO core; 05-B05
212	总线模型与字符设备	A	P5; 05-A05
213	交叉编译	A	P5; 05-A15
214	U-Boot 启动	B	05-B06
215	U-Boot/内核参数	B	DTS/bootargs; 05-B06
216	关闭中断/看门狗/cache/MMU	B	启动协议边界; 05-B06

A 级覆盖统计

本版共有 76 个 A

级题号。多个相邻题号共享一个完整主题卡，但每个题号都能反向找到对应章节。B/C 题不会被包装成已完成项目能力。

按项目反查

项目	重点原题号
P1 手持体温	6、7、14、21~23、32、75~90
P2 送药小车	6、7、14、22、23、75~90、107、108
P3 LiDAR	4、25、32、40~48、68~71、100、104~108、169~189
P4 光电融合	4、6、7、32、33、40~48、58、59、69~71、104~108、132~139、160、204
P5 I.MX6ULL	4、14、21、25、29、32、33、40~48、54、55、58、59、68~71、100、104、169~171、181~188、193~213

08 | 40 个核心问题的三层追问树

使用方法：先只看根问题口述 30 秒，再展开一级追问；最后用代码证据和验证边界回答二级追问。每题的“判断点”说明面试官真正想确认什么。

1. `volatile` 能保证线程安全吗？

```
volatile 做什么？
├─ 为什么不能保证 counter++ 原子？
│   └─ atomic 解决了什么？
│       └─ mutex 又解决了什么？
├─ 中断共享变量只加 volatile 够吗？
└─ 五项目里分别怎样使用？
```

- 判断点：能否区分编译器访问、原子性、内存序和复合不变量。
- 项目证据：P1/P2 CMSIS 寄存器；P4 atomic 标志与 mutex 快照。
- 合格收口：硬件寄存器用 volatile；线程标志用 atomic；复合数据用 mutex/队列。
- 陷阱：说“每次从内存读，所以线程安全”。

2. `const` 和不可变对象是同一回事吗？

```
顶层/底层 const？
├─ const 成员函数承诺什么？
├─ mutable 为什么存在？
└─ const 能否解决多线程同步？
```

- 判断点：接口契约和底层对象状态的区别。
- 项目证据：P5 `Monitor::update(const Sample&)`；P4 `const latest()` 内部仍需锁。
- 陷阱：把 const 当深度不可变或线程安全。

3. RAII 为什么不等于智能指针？

```
RAII 管什么？
├─ unique_ptr/shared_ptr 的所有权差别？
├─ fd 怎样 RAII？
└─ 设备 enable 状态怎样 RAII？
```

- 判断点：能否把资源概念扩展到 fd、锁、线程和设备状态。
- 项目证据：P5 `UniqueFd`、`BufferGuard`；内核 `devm`。
- 陷阱：只背 `shared_ptr` 引用计数。

4. `shared_ptr` 为什么不是默认选择？

```
共享所有权真的存在吗？
├─ 控制块和引用计数成本？
├─ 循环引用怎样发生？
└─ unique_ptr/值语义能否表达？
```

- 判断点：所有权建模而不是 API 熟练度。
- 项目证据：P5 fd 明确唯一所有权，不使用 `shared_ptr`。

- 陷阱：为了避免 delete 到处 shared_ptr。

5. move 到底移动了什么？

std::move 做了什么？
 └─ 移动构造在哪里真正转移资源？
 └─ 移动后对象能否使用？
 └─ 为什么移动构造常标 noexcept？

- 判断点：值类别、资源所有权和容器行为。
- 项目证据：P5 UniqueFd 移动后源 fd=-1。
- 陷阱：说 std::move 会复制/搬运内存。

6. 为什么不能把协议字节直接强转成结构体？

端序、对齐、padding、别名规则？
 └─ UDP 与本机结构布局相同吗？
 └─ float 怎么安全解析？
 └─ l10 scan 为什么是 24 字节？

- 判断点：对象表示和线协议的边界。
- 项目证据：P3 长度检查；P4 memcpy；P5 padding 清零。
- 陷阱：只在 x86 小端机器上“测过能跑”。

7. ISR 为什么越短越好？

ISR 长会影响什么？
 └─ 哪些 RTOS API 可调用？
 └─ printf 为什么危险？
 └─ 业务应怎样下沉到任务？

- 判断点：上下文限制和最坏延迟。
- 项目证据：P2 USART ISR 只组帧和 FromISR queue。
- 陷阱：只说“中断优先级高”。

8. Cortex-M 的 SVC、PendSV、SysTick 分别做什么？

首任务怎样启动？
 └─ SVC 为什么不是普通函数？
 └─ PendSV 为什么适合切换上下文？
 └─ 空 SVC handler 会发生什么？

- 判断点：RTOS 端口与业务层边界。
- 项目证据：P1 SVC_Handler→vPortSVCHandler。
- 陷阱：把 Linux softirq 与 SVC 混为一谈。

9. Queue、Semaphore、Mutex 怎么选？

要传数据还是只传事件？

- └— 是否存在资源所有者？
- └— 是否需要优先级继承？
- └— 是否允许丢弃？

- 判断点：从业务语义选原语。
- 项目证据：P1/P2 queue；P1 I2C mutex；长度 1 mailbox。
- 陷阱：binary semaphore 和 mutex 随意互换。

10. 优先级反转怎样发生？

L 持锁、H 等待、M 抢占？

- └— 优先级继承怎样缓解？
- └— 为什么不能解决所有情况？
- └— 如何从设计上减少共享？

- 判断点：能否画出时间线。
- 项目证据：P1 短临界区；P2 单一电机写入者。
- 陷阱：把提高 H 优先级当解决办法。

11. 为什么长度 1 队列有价值？

什么是 latest-value 语义？

- └— 哪些数据可丢旧？
- └— 哪些命令绝不能 overwrite？
- └— 如何观测丢弃？

- 判断点：数据新鲜度和完整性的取舍。
- 项目证据：P2 OpenMV/重量；P1 传感器。
- 陷阱：所有队列都设长度 1。

12. 为什么“加大队列”不能解决持续过载？

生产率 > 消费率会怎样？

- └— 延迟如何增长？
- └— Block/Reject/DropOldest 怎么选？
- └— dropped 指标为什么必要？

- 判断点：容量和稳定性的区别。
- 项目证据：P3 BoundedQueue；P4 3 帧保存队列。
- 陷阱：只谈内存，不谈旧数据价值。

13. 非阻塞 FSM 比 delay 好在哪？

delay 期间还能处理什么？

- └— 状态、事件、进入时间怎样组织？
- └— Tick 回绕怎样处理？
- └— 如何单元测试时间逻辑？

- 判断点：控制逻辑可响应性与可测试性。

- 项目证据：P2 `CartState_Step`。
- 陷阱：switch 很多就自称状态机。

14. Tick 回绕为什么用无符号减法？

```
now >= deadline 哪里错？
├─ 模运算怎样得到 elapsed？
├─ 比较区间有什么限制？
└─ 测试怎样跨 UINT32_MAX？
```

- 判断点：固定宽度计数器推理。
- 项目证据：P2 `prvElapsed` host test。
- 陷阱：转成有符号数比较。

15. 超时、重试、退避、看门狗有什么区别？

```
一次等待上界？
├─ 临时失败是否重试？
├─ 持续失败怎样降低频率？
└─ 系统无响应由谁复位？
```

- 判断点：分层故障恢复。
- 项目证据：P2 HX711 timeout；P4 RTSP backoff。
- 陷阱：所有错误都立即无限重试。

16. 进程和线程的真正区别是什么？

```
共享哪些资源？
├─ 每线程是否有独立栈？
├─ 故障隔离与通信成本？
└─ 为什么 P5 选择单线程 epoll？
```

- 判断点：隔离、共享和调度实体。
- 项目证据：P4 多线程；P5 事件循环。
- 陷阱：线程没有独立内存/栈。

17. 普通 bool 停线程为什么是 UB？

```
data race 定义？
├─ volatile 能否修复？
├─ atomic 的顺序语义？
└─ 阻塞线程如何被唤醒？
```

- 判断点：语言内存模型与退出协议。
- 项目证据：P4 atomic flags；P3 queue shutdown。
- 陷阱：在 x86 测试没出错就认为安全。

18. Atomic 能保护结构体快照吗？

每字段 atomic 是否等于整帧一致？

- └─ 不变量跨哪些字段？
- └─ mutex 快照怎样缩短锁？
- └─ 何时用 seqlock/双缓冲？

- 判断点：原子对象和事务一致性的区别。
- 项目证据：P4 `latest()`。
- 陷阱：把所有字段改 atomic。

19. Condition variable 为什么必须有谓词？

虚假唤醒？

- └─ lost wakeup 怎样避免？
- └─ 多消费者竞争？
- └─ shutdown 怎样进入谓词？

- 判断点：通知与状态的关系。
- 项目证据：P3 `BoundedQueue`。
- 陷阱：`wait` 后默认条件成立。

20. detach 有什么实际风险？

谁拥有线程？

- └─ 对象析构后 this 怎么办？
- └─ 如何等待结束？
- └─ `stop+join` 的正确顺序？

- 判断点：线程和对象生命周期耦合。
- 项目证据：P4 移除 `detached spinner`。
- 陷阱：`detach` 能“避免阻塞”。

21. 怎样证明线程能优雅退出？

只设 `stop flag` 够吗？

- └─ `queue/cv/read` 阻塞怎样解除？
- └─ `join` 前能否析构依赖？
- └─ 停止顺序怎样测试？

- 判断点：完整关闭协议。
- 项目证据：P3 `shutdown+notify_all+join`；P4 `stop_thread`。
- 陷阱：依赖进程退出回收一切。

22. 死锁和优先级反转如何区分？

是否存在等待环？

- └─ 线程仍可能前进吗？
- └─ 四个必要条件？
- └─ 单写者怎样减少两类风险？

- 判断点：并发故障分类。

- 项目证据：P2 单控制任务，P3 固定退出协议。
- 陷阱：所有卡顿都叫死锁。

23. fd 是文件吗？

fd、file description、inode 的关系？
 └─ socket/timerfd 为什么也是 fd？
 └─ dup/fork 后共享什么？
 └─ RAII 怎样表达关闭责任？

- 判断点：Unix I/O 抽象。
- 项目证据：P5 **UniqueFd**。
- 陷阱：close 一个 fd 等于对象立即不存在。

24. epoll 为什么适合 P5？

有哪些事件源？
 └─ lio、timerfd、signalfd 怎样统一？
 └─ 为什么不每源一个线程？
 └─ CPU 密集任务怎么办？

- 判断点：选择机制而不是背性能。
- 项目证据：P5 condition。
- 陷阱：三个 fd 就宣称高并发性能。

25. epoll 一定比 poll 快吗？

fd 数量和活跃比例？
 └─ interest set 怎样维护？
 └─ 返回就绪项还需遍历吗？
 └─ 单 fd 为什么 poll 也合理？

- 判断点：适用条件和复杂度常数。
- 项目证据：P3 单 socket；P5 多类型 fd。
- 陷阱：背 $O(1)$ 结束。

26. ET 为什么必须读到 EAGAIN？

边沿之后还有剩余数据？
 └─ 阻塞 fd 会发生什么？
 └─ LT 是否也可 drain？
 └─ 怎样防止一个 fd 饿死其他事件？

- 判断点：就绪语义和公平性。
- 项目证据：P5 非阻塞 read loop。
- 陷阱：把 EAGAIN 当错误日志风暴。

27. 一次 read 为什么不等于一帧？

短读、多帧、残片？

- └─ 固定长度怎样拼？
- └─ 长度字段怎样防超大？
- └─ UDP 为什么不同？

- 判断点：流式协议基本功。
- 项目证据：P5 pending；P2 两字节组帧；P3 UDP/PCAP。
- 陷阱：read struct。

28. TCP 和 UDP 在 LiDAR 场景怎么选？

旧数据重传还有价值吗？

- └─ 丢包/乱序怎样观测？
- └─ UDP 拥塞怎么办？
- └─ 控制命令是否同样选择？

- 判断点：业务价值与传输语义。
- 项目证据：P3 UDP + Drop0ldest。
- 陷阱：UDP 一定快。

29. 网络字节序只是 ntohl 吗？

整数、浮点、结构体？

- └─ 未对齐访问？
- └─ 严格别名？
- └─ 版本和长度校验？

- 判断点：安全解析而非函数背诵。
- 项目证据：P3 长度检查、P4 memcpy。
- 陷阱：整结构体 cast。

30. 为什么需要指数退避？

立即重连的系统效应？

- └─ 上限为何必要？
- └─ 成功后为什么 reset？
- └─ 多设备为何加 jitter？

- 判断点：故障状态下的资源控制。
- 项目证据：P4 ReconnectBackoff host test。
- 陷阱：sleep 5 秒导致 stop 不响应。

31. 设备树怎样触发 probe？

DTS 是否直接加载驱动？

- └─ 谁创建 spi_device？
- └─ compatible 在哪里匹配？
- └─ controller 未就绪怎么办？

- 判断点：总线模型链路。

- 项目证据: P5 dtsi + of_match + spi_driver。
- 陷阱: 设备树放业务逻辑。

32. probe 为什么必须 fail-fast?

节点存在等于硬件正常吗?
 |— WHO_AM_I 能排查什么?
 |— 中途失败怎样回滚?
 |— defer 与永久错误区别?

- 判断点: 半初始化状态与错误传播。
- 项目证据: P5 hw_init/devm。
- 陷阱: 打印错误后继续注册。

33. regmap 解决什么问题?

它是总线吗?
 |— reg_bits/val_bits/read_flag?
 |— bulk read 的收益?
 |— cache 何时危险?

- 判断点: 抽象边界。
- 项目证据: P5 icm20608_regmap_config。
- 陷阱: 只说“少写代码”。

34. IIO 与自定义字符设备怎么选?

设备属于哪个内核子系统?
 |— 标准 channel/scale/buffer 有什么价值?
 |— 特殊 ioctl 何时合理?
 |— IIO 是否仍经过设备文件?

- 判断点: 框架复用与 ABI 设计。
- 项目证据: P5 IIO channels。
- 陷阱: 为展示“会写 cdev”重复造 ABI。

35. hrtimer 中为什么不能读 SPI?

当前上下文能睡眠吗?
 |— SPI 控制器传输可能怎样等待?
 |— threaded handler 在哪里运行?
 |— 有 DRDY 时怎样改?

- 判断点: 内核上下文约束。
- 项目证据: P5 timer callback→trigger handler。
- 陷阱: 把 hrtimer 当普通线程回调。

36. 自旋锁与 mutex 怎样选？

是否可睡眠？

- └─ 临界区多长？
- └─ 共享者是否包括 IRQ？
- └─ 能否通过状态机删除共享？

- 判断点：上下文、时长、竞争者。
- 项目证据：P5 buffer 运行时拒绝改频率。
- 陷阱：自旋锁“更快”所以都用。

37. IIO scan 为什么有 padding？

14 字节通道后时间戳放哪？

- └─ 结构对齐规则？
- └─ padding 不清零有什么安全问题？
- └─ 用户态怎样解码？

- 判断点：ABI、对齐和信息泄漏。
- 项目证据：P5 24-byte scan。
- 陷阱：14+8=22。

38. sysfs 为什么不能写死 device0？

编号由什么决定？

- └─ 怎样按 name 枚举？
- └─ 多属性配置怎样回滚？
- └─ buffer 运行时能否改布局？

- 判断点：动态设备与配置事务。
- 项目证据：P5 findDevice/configureBuffer host test。
- 陷阱：失败后 buffer 保持 enabled。

39. 交叉编译通过能证明什么？

架构、ABI、rootfs、内核版本？

- └─ ARM ELF 能否直接运行？
- └─ 对象文件是否等于 .ko？
- └─ HIL 还要验证什么？

- 判断点：证据边界。
- 项目证据：P5 Linaro EABI5 + vendor object。
- 陷阱：把 CROSS 写成 HIL。

40. 为什么算法不放内核？

机制与策略怎样分层？

- └─ 阈值变化是否需要重装模块？
- └─ 算法崩溃影响范围？
- └─ 用户态延迟是否满足需求？

- 判断点：最小内核原则与系统边界。

- 项目证据: P5 IIO driver + libcondition。
- 陷阱: 内核态“更快”就全部搬进去。

不会时的统一收口

遇到追问超出项目证据时, 可以使用:

这个机制我理解其原理, 但当前项目没有直接实现。当前代码采用的是……, 因为……。如果迁移到您说的场景, 我会先补……测试, 再决定是否替换, 所以这里我不把它说成现有项目成果。

这不是示弱, 而是把事实、推理和验证边界分开。

实验手册：把回答变成可复现证据

1. 为什么面试准备必须包含实验

背诵能回答“是什么”，实验才能回答“你怎么知道”。下面每个实验都固定记录七件事：目的、关联题、前置条件、操作、预期现象、失败注入、面试表达。命令只是入口；真正需要内化的是：改变一个条件后，系统为何呈现对应结果。

证据边界统一使用 **CODE/HOST/CROSS/HIL/TODO-HIL**。宿主机通过不等于实板通过，交叉编译通过也不等于模块能加载。

2. P1：融合条件与 Flash CRC 边界

目的：验证“多源数据不能仅凭到达就使用”，以及持久化数据为什么同时需要格式校验和 CRC。

- 关联题：12、16、17、20、21、41、81、83。
- 前置条件：进入 P1 仓库，准备可用的 C/C++ 编译器。
- 证据等级：融合门槛和 Flash 损坏注入为 **HOST**，ARMCC 完整构建与 map 边界为 **CROSS**；片内 Flash 擦写与掉电时序仍是 **TOD0-HIL**。

操作：按仓库 **README.md** 的 Host Test 命令构建并运行 **fusion_policy** 与 Linux 下的 **flash_store**。后者在测试进程内映射模拟 Flash 页，保存配置后翻转 WiFi 字段的一个字节，再调用实际的 **FlashStore_Load**；另注入擦除失败检查 **FlashStore_Init** 的返回值。

预期现象：不完整样本被融合门槛拒绝；损坏记录被 CRC 拒绝，已有 RAM 配置保持不变；模拟擦除失败时初始化返回失败，而不是假报成功。

失败注入的意义：只测正常路径只能说明函数会返回成功；主动破坏一个字节，才能证明校验真的参与了接受决策。CRC 可以检测常见误码，但不提供抗恶意篡改能力。

Keil 构建实验：用 ARMCC 5.06u6 构建 **Project/MDK-ARM/sud.uvprojx**，检查日志的 0 错误、0 警告和 HEX 生成，再读 **Project/MDK-ARM/Obj/Listings/sud.map** 中的 **LR_IROM1**。当前实测基址 **0x08000000**、占用 **0x90BC**、最大 **0xFC00**，而配置页从 **0x0800FC00** 开始。曾仅修改 IROM 大小但未启用工程的目标内存布局，map 的 Max 仍为无限值；启用后重编才真正形成链接约束。这个实验说明“界面配置了”与“链接器执行了”是两回事。

面试表达：

我把“数据有效”拆成来源完整性和持久化完整性。P1 的 Host 测试验证融合门槛，也用映射的模拟 Flash 页翻转一个字节，验证 CRC 拒绝损坏记录且不污染 RAM。这证明了代码逻辑，不证明片内 Flash 擦写和掉电恢复，后者仍是 TOD0-HIL。

3. P2：非阻塞状态机与 Tick 回绕

目的：验证控制流程不会因延时阻塞，并确认无符号差值写法在计数器回绕后仍成立。

- 关联题：30、31、76、77、79、81、84、85、87、88。
- 证据等级：状态机为 **HOST**，ARMCC 完整构建为 **CROSS**；电机、秤和串口联调是 **TODO-HIL**。

正常实验：运行 P2 的状态机测试，让事件按“接单一寻路—到位—称重—完成”进入。检查每步只依据当前状态、事件和时间推进，不在处理函数中长时间等待。

回绕实验：把起始 Tick 放到 **UINT32_MAX** 附近，使截止时间跨过零点。判断使用 `(uint32_t)(now - start) >= timeout`，而不是 `now >= start + timeout`。

预期现象：状态机在到期前保持原状态，到期时只迁移一次；跨零点结果与普通时间段相同。

失败注入：临时把差值判断替换成绝对时间比较，回绕样例应失败。这个失败不是编译错误，而是边界条件下的业务错误。

Keil 构建实验：ARMCC 5.06u6 构建 **Project/MDK-ARM/Fire_FreeRTOS.uvprojx**，当前日志为 0 错误、0 警告并生成 AXF/HEX；这验证定时器模块进入固件和完整链接，不验证 OLED 启动时序或电机动作。

面试表达：

我没有用阻塞延时串联动作，而是让单写者控制任务驱动 FSM。时间判断用无符号差值，因此 Tick 回绕仍可工作；这个结论由边界测试证明，真正的电机动作时长还要实车标定。

4. P3：有界队列、过载策略与优雅退出

目的：观察生产速度高于消费速度时系统如何退化，以及停止顺序为什么影响死锁和悬空访问。

- 关联题：40、42、53、104、105、109、112、113、119、121、122、127、128。
- 证据等级：**HOST**；真实雷达吞吐是 **TODO-HIL**。

正常实验：按 P3 **README.md**

构建并运行队列测试。记录容量、生产者数量、消费者数量和退出耗时。确认队列未滿时 FIFO 顺序成立，关闭后等待线程被唤醒并退出。

过载实验：减慢消费者或连续推送超过容量的数据。观察仓库定义的策略——阻塞、拒绝新数据或丢弃旧数据——并核对计数器，而不是假定“队列会自动解决过载”。

停止实验：先通知停止，再关闭输入源/队列，唤醒条件变量，最后 **join**，然后销毁共享资源。

失败注入：删去关闭时的 **notify_all**，等待线程可能永久睡眠；在 **join** 前销毁队列，可能出现 **use-after-free**。恢复顺序后测试应稳定结束。

面试表达：

有界队列的价值不是让数据不丢，而是让内存上限和过载策略显式化。P3 在关闭时先改变共享状态并唤醒，再 **join**，最后释放资源。宿主测试验证同步协议；设备吞吐和调度抖动还需要实机压测。

5. P4：重连退避、快照与生命周期

目的：验证重连不会形成忙循环，读线程不会拿到正在修改的复合对象，析构不会遗留 detached 线程。

- 关联题：17、25、32、39、40、45、47、48、51、52、53、104、105、106、109、112、113、119、126。
- 证据等级：CODE/HOST；完整 ROS/Qt/RTSP 集成是 TODO-HIL。

退避实验：连续模拟连接失败，记录下一次尝试间隔。预期呈有上限的增长；成功一次后退避状态复位。加入少量抖动可以避免多个客户端同相重连，但当前实现若未使用抖动，不应在面试中说已实现。

快照实验：写线程在 mutex 内更新完整结构，读线程在同一 mutex 下复制后立即解锁，再在锁外处理。预期读者得到同一版本的字段组合，锁的占用时间不包含耗时算法。

生命周期实验：触发停止，检查原子停止标志和 join 后释放的顺序。RTSP 的 VideoCapture::read

仍可能被底层网络调用阻塞，当前没有证明退出耗时上界；不要声称已实现可靠的阻塞唤醒。

失败注入：把重连间隔固定为零会导致

CPU/日志风暴；返回共享结构的裸引用会把锁外访问暴露给写线程；detach 会让对象析构无法证明线程已经结束。

面试表达：

我把 atomic 用于单变量停止状态，mutex 用于需要一致性的复合快照。连接失败采用有上限退避，析构时先停止并 join 再释放采集对象；不过 RTSP 底层读超时尚未完成集成验证，所以我不承诺固定退出时限。

6. P5：领域算法、回放与故障状态机

目的：在不依赖开发板的情况下验证姿态/冲击判定和传感器失联状态转换。

- 关联题：18、25、46、54、141、142、143、151、189、190。
- 证据等级：HOST。

操作：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build --output-on-failure
./build/apps/conditiond --simulate --samples 200
./build/apps/condition-replay tests/data/normal_then_impact.csv
```

预期现象：单元测试全部通过；模拟服务输出周期样本和状态；回放文件先保持正常，随后进入冲击/异常状态。具体阈值是教学参数，不应包装成产品认证指标。

失败注入：把样本时间戳停止推进，状态机应进入

stale/fault；将单个峰值置于阈值附近，观察防抖或连续样本规则是否避免来回抖动。

面试表达：

我把硬件无关算法抽成 `libcondition`，用合成数据和 CSV 回放验证状态演化；这样驱动没上板时仍能测试业务规则。它不证明真实 IMU 噪声分布，因此阈值最终必须通过板端采集重新标定。

7. P5：伪 `sysfs` 的设备发现与扫描配置

目的：证明 `sysfs` 适配层没有把 `iio:deviceX` 编号写死，并能正确处理缺失属性。

- 关联题：174、175、177、186、188、192、195。
- 证据等级：HOST。

操作：运行 `test_iio_sysfs`。测试在临时目录建立最小 IIO 树，写入 `name`、`scan_elements/*_en` 等文件，让适配器按设备名发现目录并配置通道。

预期现象：编号变化不影响发现；名称不匹配、文件缺失、写入失败均返回明确错误；析构或失败路径不遗留打开的 `fd`。

失败注入：删掉一个必需属性或改写设备名，测试应走失败分支，而不是默默使用错误设备。

面试表达：

IIO 编号由探测顺序决定，不能写死。我把 `sysfs` 根目录做成可注入参数，用伪文件树测试按 `name` 发现和错误路径。这是对用户态适配逻辑的证明，不是对内核驱动 `probe` 的证明。

8. P5：`epoll`、`timerfd`、`signalfd` 与半帧解析

目的：理解“事件就绪”不等于“业务消息完整”，以及为何把定时和退出信号纳入同一事件循环。

- 关联题：101、104、105、112、113、119、120、121、122、124、126、127、128、129、130、131、132。
- 证据等级：CODE/HOST。

运行 `conditiond --simulate --samples 200`。事件循环同时接收模拟定时器 `fd`、状态 `timerfd` 和 `signalfd`。定时器驱动状态检查，信号触发有序退出。

半帧实验：运行 `ctest -R scan-assembler --output-on-failure`。测试将 24 字节扫描帧拆成 1+22+余量输入，并一次送入多个帧；只有完整帧才回调，缓存最多保留不足一帧的数据。实际 IIO 字符设备读取仍需上板验证。

失败注入：把 `read` 的返回长度忽略，解析器可能读取旧数据或越界；在 ET 模式下不读到 EAGAIN，剩余数据可能没有新的边沿通知。当前代码使用何种触发模式，面试必须以仓库事实为准。

面试表达：

`epoll` 只通知 `fd` 当前可操作，不能提供应用层消息边界。我用有界的残帧缓存按扫描布局拆帧，并限制每轮最多读取 16 次，避免持续输入饿死超时和信号处理；`timerfd` 和 `signalfd` 让周期任务与退出进入同一事件循环。

9. P5: ARM 用户态交叉编译

目的：证明源码和所用标准库接口能被目标工具链接受，并检查产物架构。

- 关联题：69、70、92、93、160、161、162、163。
- 证据等级：CROSS。

操作：

```
export CROSS_COMPILE=/opt/gcc-linaro/bin/arm-linux-gnueabi-
cmake -S . -B build-arm \
  -DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/imx6ull-gcc.cmake \
  -DBUILD_TESTING=OFF -DCMAKE_BUILD_TYPE=Release
cmake --build build-arm
file build-arm/apps/conditiond
```

预期现象：构建成功；file 显示 32 位 ARM EABI5 ELF，动态加载器与板端 rootfs 约定一致。

边界：这不能证明板端动态库版本匹配、设备节点权限正确或运行稳定。部署前还要检查 sysroot、glibc/libstdc++ ABI 和服务用户权限。

10. P5: vendor 4.1.15 驱动对象编译

目的：用指定内核树验证模块源码与旧版 API 的编译兼容性。

- 关联题：164、165、166、167、169、170、171、172、173、174、175、176、177、178、179、181、182、183、184、185、186、187、188、189、190、191、192、193、194、195、199、202、203。
- 证据等级：CROSS，不是 HIL。

操作：按 docs/VALIDATION.md 指定 vendor Linux 4.1.15 树和交叉工具链构建 icm20608_iio.o。检查编译器返回码和目标文件格式。

失败注入：使用主线内核树可能出现 IIO/regmap API

差异；这不应通过随意条件编译掩盖，而应先明确支持矩阵。将 DTS compatible 拼错不会导致 C 编译失败，却会在实板上导致 probe 不发生，这正说明编译验证有边界。

面试表达：

驱动对象已在目标 vendor 4.1.15 头文件和 ARM 工具链下通过编译，证明 API 与架构层面的兼容性；我没有把它写成模块加载成功。下一步必须核对 DTB、SPI 波形、WHO_AM_I 和 IIO 缓冲数据。

11. fd 泄漏与错误路径观察

目的：把 RAII、系统调用错误与 /proc 观察联系起来。

- 关联题：18、19、20、21、25、100、101、102、103、132、205、206。
- 证据等级：HOST。

步骤：启动模拟服务，记录 `/proc/<pid>/fd`

数量；重复触发可以恢复的打开/关闭路径；比较前后数量。正常情况下，稳定状态的 `fd` 数不会随循环线性增长。再使用 `strace`（若环境提供）观察 `openat/close/epoll_ctl/read` 返回值。

失败注入：临时在错误返回前跳过 `close`，`fd` 数会随重试增加，最终可能遇到 `EMFILE`。恢复 `UniqueFd` 或统一清理路径后，计数应稳定。

不要把“某次计数不变”扩大成“绝无泄漏”；长期压力、第三方库和线程竞争仍需其他工具和场景验证。

12. Sanitizer 与并发诊断的适用边界

目的：选择与缺陷类型匹配的工具，而不是把所有问题都叫“内存问题”。

工具	主要发现	典型目标	不能替代
ASan	越界、use-after-free、部分泄漏	解析器和资源生命周期	实时性、内核态错误
UBSan	未定义行为	移位、溢出、错误转换	数据竞争
TSan	用户态数据竞争	P3/P4 线程共享状态	目标板时序、驱动锁验证
Valgrind	泄漏和非法访问，开销较大	宿主长期路径	ARM 实时性能
strace	系统调用序列与错误码	fd、网络、文件权限	用户态算法正确性

诊断顺序：先构造最小可复现输入，再记录版本和参数，使用对应工具定位，最后把复现转成自动测试。工具报告是线索；修复是否正确仍由回归测试和设计不变量决定。

13. 实板 HIL 闭环：尚未通过的项目

以下步骤是明确的下一阶段，不得提前改写成 HIL：

1. 固化 U-Boot、kernel、DTB、rootfs 和工具链版本。
2. 合入 DTS，启动后从 `dmesg` 确认 `compatible` 匹配与 `probe` 日志。
3. 用示波器或逻辑分析仪确认 `SPI mode`、频率、片选和 `WHO_AM_I` 事务。
4. 检查 `/sys/bus/iio/devices`、`scan elements`、`scale` 和采样频率。
5. 用 `iio_readdev` 或服务读取静止、六面体倾斜和运动数据。
6. 注入拔线、停流、重复冲击、进程重启和服务自启动。
7. 运行至少一次长稳，记录丢样、CPU、内存、温度与错误日志。
8. 保存命令、原始日志、波形和结论，更新 `docs/VALIDATION.md` 后再升级证据等级。

面试遇到“你上板了吗”时，正确回答是说明当前做到哪一级、还差哪一步、怎样验证。清晰边界比虚构“已经跑过”更可信。

14. 实验复盘模板

每次新增实验，按以下结构留下记录：

实验名称：
代码提交/标签：
环境与版本：
假设：
输入与操作：
预期结果：
实际结果：
失败注入：
原始证据位置：
结论与边界：
下一步：

如果无法填写“失败注入”和“边界”，通常说明实验只是在演示成功路径，还不足以支撑深入追问。

面试速记卡与掌握度检查

1. 使用方式

速记卡不是正文的替代品。正面问题能在 30 秒内回答后，还要随机抽取“反问”与“证据”两列；说不出代码位置或验证边界，就回到对应章节和实验。

2. C/C++ 与内存

正面	30 秒答案核心	反问	项目锚点
<code>const</code> 有什么用	建立只读接口契约；顶层/底层 <code>const</code> 不同；可通过其他别名改变并不矛盾	<code>const</code> 对象一定放只读段吗	P4/P5 参数接口，01章
<code>volatile</code> 能保证线程安全吗	不能；只约束可观察访问，缺少原子性和 happens-before	何时会同时看到 <code>volatile</code> 与中断	STM32 寄存器、ISR 标志
<code>atomic</code> 与 <code>mutex</code> 怎么选	单个独立状态可 <code>atomic</code> ；多个字段要保持不变量时用 <code>mutex</code>	<code>memory order</code> 用的是什么	P4 stop flag 与 snapshot
栈和堆本质区别	生命周期/所有权/分配机制不同，不用“一定快慢”绝对化	RTOS 任务栈如何估算	P1/P2 静态任务资源
<code>malloc/free</code> 与 <code>new/delete</code>	前者分配字节，后者参与对象构造析构；必须成对	<code>placement new</code> 是什么	Linux C++ RAII
内存泄漏如何定位	最小复现、观察资源增长、ASan/Valgrind、回归测试	fd 泄漏算内存泄漏吗	P3/P4/P5 生命周期
野指针和悬空指针	未初始化或已失效地址；关键是所有权与失效时机	<code>shared_ptr</code> 能完全避免吗	P3 队列关闭、P4 join
RAII 是什么	资源绑定对象生命周期，构造获取、析构释放	构造中途失败怎么办	P3 socket, P5 UniqueFd
拷贝与移动区别	拷贝复制资源，移动转移所有权并保持源可析构	<code>std::move</code> 会移动吗	P3/P4 消息传递
<code>vector</code> 扩容影响	重新分配并移动/拷贝，迭代器和指针可能失效	如何减少抖动	点云/帧缓存 reserve
<code>lambda</code> 捕获风险	引用捕获的生命周期必须覆盖异步执行	<code>[=]</code> 捕获 <code>this</code> 吗	P4 工作线程
C++11 对项目最有用什么	<code>thread/atomic/mutex/condition_variable</code> 、移动、智能指针	为什么 P5 用 C++14	工具链兼容边界

3. ARM、ISR 与 FreeRTOS

正面	30 秒答案核心	反问	项目锚点
中断发生时做什么	保存必要现场、查向量、执行 ISR、恢复；具体由架构和编译器配合	嵌套中断条件	P2 USART ISR
ISR 为什么要短	高优先级执行会推迟任务和低优先级中断，且可用 API 受限	能在 ISR 打日志吗	P1/P2 ISR 边界
ISR 如何唤醒任务	用 <code>...FromISR</code> API，必要时请求退出中断后调度	为什么不能用普通 <code>queue</code> API	P2 帧就绪通知
优先级反转是什么	低优先级持锁阻塞高优先级，中优先级又抢占低优先级	mutex 如何缓解	P1 总线互斥
semaphore 与 mutex 区别	semaphore 表示数量/事件；mutex 表示所有权并可有优先级继承	ISR 能 give mutex 吗	P1/P2
queue 与共享变量区别	queue 明确复制、顺序和阻塞；共享变量需自建同步协议	最新值为何用长度1覆盖	P1/P2 mailbox
实时等于快吗	不是，实时强调最坏时间可预测和截止期满足	Linux 是实时系统吗	STM32 与 Linux 对比
Tick 回绕如何处理	对无符号计数做差并限制超时时间小于半范围	绝对截止为何错	P2 FSM 测试
FSM 为什么优于 delay	每步短执行，可响应事件、超时与停止	状态爆炸怎么办	P2 控制任务
watchdog 怎么设计	喂狗条件应代表关键任务都健康，而非任意循环仍运行	谁来汇总心跳	P1/P2 B级迁移

4. Linux 进程、线程与同步

正面	30 秒答案核心	反问	项目锚点
进程与线程差异	进程隔离地址空间；线程共享进程资源但有独立栈和调度状态	崩溃影响范围	P3/P4 架构
用户态到内核态怎么切换	系统调用/异常进入内核，校验参数后执行并返回	切换等于进程切换吗	socket/read/epoll
条件变量为何要循环等待	唤醒不等于条件成立，存在虚假唤醒和竞争	notify 是否必须持锁	P3 BoundedQueue
死锁四条件	互斥、持有等待、不可剥夺、循环等待	项目怎么预防	P3/P4 锁序与短临界区
detach 与 join	detach 失去汇合点；可控资源生命周期通常应 join	阻塞 read 如何退出	P4 stop 顺序
快照模式是什么	锁内复制一致状态，锁外耗时处理	复制开销太大怎么办	P4 latest()
false sharing 是什么	不同变量共享 cache line 导致一致性流量	项目证明遇到了吗	RELATED, 不声称已遇到
虚拟内存价值	隔离、统一地址、按需映射和共享；缺页有成本	mmap 与 read 取舍	03章 B级
fd 是什么	进程 fd 表索引，指向打开文件描述和底层对象	dup 后偏移量是否共享	P3/P5
如何优雅退出多线程	通知停止、解除阻塞、join、释放资源	信号处理器能否直接 join	P3/P4/P5

5. Linux I/O 与网络

正面	30 秒答案核心	反问	项目锚点
TCP 与 UDP 怎么选	TCP 有序可靠字节流；UDP 保留报文边界、低开销但需自管丢包乱序	雷达为何常用 UDP	P3
TCP 粘包是什么	TCP 没有消息边界，接收端必须按长度/分隔符/固定帧解析	一次 <code>recv</code> 为何不等于一包	04章
<code>select/poll/epoll</code> 区别	<code>fd</code> 表达和就绪获取方式不同； <code>epoll</code> 避免每轮复制/扫描全部集合	<code>epoll</code> 一定更快吗	P3/P5
LT 与 ET	LT 未处理完会继续提示；ET 只在状态变化提示，通常非阻塞读到 <code>EAGAIN</code>	漏读会怎样	P3/P5
<code>epoll</code> 通知了什么	<code>fd</code> 就绪，不保证业务帧完整，也不保证后续调用必成功	半帧怎么办	P5 缓冲解析
非阻塞 I/O 如何处理	接受 <code>EAGAIN</code> ，维护状态和缓冲，等下一事件	<code>busy loop</code> 如何避免	P3/P5
<code>timerfd</code> 的价值	把定时器变成 <code>fd</code> 并入事件循环，可读出累计到期次数	与 <code>sleep</code> 区别	P5 <code>conditiond</code>
<code>signalfd</code> 的价值	屏蔽普通异步处理，把选定信号转换为可读事件	哪些信号不能捕获	P5 <code>conditiond</code>
字节序为何重要	多字节整数跨主机传输需约定顺序；网络通常大端	浮点怎么传	P3 数据协议
退避为何要上限	避免故障时忙重试，也限制恢复等待；可加 <code>jitter</code> 防同步风暴	成功后怎么处理	P4

6. Linux 驱动与 IIO

正面	30 秒答案核心	反问	项目锚点
DTS 到 probe 的链路	DT 节点实例化设备，与驱动 compatible 匹配后总线调用 probe	compatible 错会怎样	P5
probe 做什么	获取资源、校验硬件、配置设备、注册子系统；任一步失败要回滚	deferred probe 是什么	P5 icm20608_probe
devm 的作用	设备解绑时自动释放设备管理资源，缩短失败清理路径	是否所有资源都适合 devm	P5
regmap 解决什么	统一寄存器访问、缓存、锁和调试接口；总线细节由后端处理	volatile register 怎么配置	P5
为什么用 IIO	传感器有标准 channel、scale、buffer、trigger 和用户态接口	misc 字符设备何时更合适	P5
hrtimer 能否直接 SPI	timer 回调是原子上下文，不应调用可睡眠 SPI 传输	正确路径是什么	P5 trigger_poll
threaded handler 的价值	把可睡眠采样放在线程上下文，保留触发时序	与 workqueue 区别	P5
scan layout 怎么解析	按 enabled channel、index、storagebits、alignment 和 timestamp 计算	不能假设什么	P5 libiio
sysfs 与 /dev/iio:*	sysfs 做控制/属性，字符设备承载缓冲数据	编号为什么不能写死	P5
ioctl 的边界	适合不能用 read/write/sysfs 自然表达的控制；命令需 ABI 设计	本项目用了吗	BASE/RELATED，未使用
copy_to_user 为什么需要	用户指针不可信且可能缺页，必须通过内核访问器处理	返回值如何解释	05章 B级
驱动编译通过说明什么	说明语法、API 和架构工具链兼容；不证明 DT、总线和实板正确	怎么升级成 HIL	P5 CROSS

7. 高频辨析表

容易混淆	正确边界
<code>volatile</code> vs <code>atomic</code>	前者是访问可观察性；后者才提供原子操作和内存序语义
<code>atomic</code> vs <code>mutex</code>	<code>atomic</code> 适合单对象状态； <code>mutex</code> 可保护跨字段不变量和复合操作
<code>mutex</code> vs <code>binary semaphore</code>	<code>mutex</code> 有所有权语义，常带优先级继承； <code>binary semaphore</code> 更像事件/令牌
<code>queue</code> vs <code>mailbox</code>	<code>queue</code> 保存多个按序元素； <code>mailbox</code> 强调最新状态，可用长度 1 覆盖实现
实时 vs 高吞吐	实时看最坏延迟与截止期；高吞吐看单位时间处理量
并发 vs 并行	并发是任务交错推进；并行是在多个执行单元同时运行
线程安全 vs 可重入	线程安全可依赖锁；可重入要求并发/递归进入也不依赖不安全共享状态
阻塞 vs 同步	阻塞描述等待时线程状态；同步描述协调关系，两者不是同一维度
<code>epoll ET</code> vs 非阻塞	ET 通常必须配合非阻塞并读到 EAGAIN；二者概念不同
编译通过 vs 运行通过	编译证明接口与语法；运行还涉及配置、环境、时序与数据
HOST vs CROSS	HOST 在当前机器执行；CROSS 只由目标工具链生成产物
CROSS vs HIL	CROSS 不接触真实硬件；HIL 必须记录板卡、镜像、操作和结果
驱动 vs 用户态算法	驱动负责硬件机制和标准接口；算法策略尽量留在可测试用户态
<code>sysfs</code> vs 字符设备	<code>sysfs</code> 面向少量属性；字符设备适合流式/二进制数据和标准操作
DTS vs 驱动代码	DTS 描述板级实例和资源；驱动实现同类设备的通用行为

8. 五项目 30 秒卡

P1 手持体温检测仪

多源输入需要满足身份、测温质量和时效条件才能形成一次测量。重构把 ISR 限制为采集/通知，任务间通过 `queue` 与 `mutex` 协作，持久化记录带版本和 CRC。Host 测试覆盖融合策略和 CRC 损坏；外设时序、Flash 掉电一致性仍需上板。

P2 智能送药小车

核心难点是视觉帧、称重和执行器不能彼此阻塞。我使用单写者控制任务驱动非阻塞 FSM，ISR 只收帧并通知，最新传感器状态用邮箱式覆盖，超时按无符号 Tick 差值处理。Host 测试覆盖状态迁移和回绕，电机参数仍需实车标定。

P3 三维 LiDAR 感知

UDP/PCAP 输入进入有界队列和并行处理流水线。设计明确容量、过载和 EOF 语义，`socket/epoll` 通过 RAII 管理；退出按停止、关闭、唤醒、`join`、释放执行。宿主测试证明队列协议，真实设备吞吐和 ROS/PCL 集成仍需实机。

P4 多源光电数据融合

R0S、Qt、串口和视频线程之间最重要的是共享状态和生命周期。单状态使用 `atomic`，复合结果使用 `mutex`

保护并复制快照，连接失败做有上限退避，所有工作线程可停止并 `join`。源码和局部 Host 测试支持这些结论，完整依赖集成不冒充已验证。

P5 I.MX6ULL 状态监测器

ICM20608 由 DTS 实例化，SPI 驱动用 `regmap` 和 IIO 暴露通道与缓冲；`hrtimer` 只触发线程化采样，不在原子上下文做 SPI。用户态以 `epoll` 统一处理 IIO、`timerfd` 和 `signalfd`，算法可独立回放。Host、ARM 用户态和 `vendor` 驱动对象已验证，模块加载与实板数据仍是 TODO-HIL。

9. 追问失分警报

听到自己说出以下句子时，应立即纠正：

- “`volatile` 可以保证多线程同步。”
- “`epoll` 一定比 `select` 快。”
- “UDP 比 TCP 快，所以选 UDP。”
- “用了 `mutex` 就不会死锁。”
- “用了智能指针就不会泄漏。”
- “交叉编译成功说明能在板子上运行。”
- “设备树加载驱动。”——应说明匹配、设备实例和总线 `probe` 链路。
- “中断不能调用任何函数。”——应说只能调用满足上下文约束的函数。
- “实时系统就是运行速度快。”
- “一次 `read` 就是一帧数据。”
- “我做过完整驱动项目。”——当前应准确说明教学项目来源和 HIL 状态。

10. 掌握度清单

每项只有在能独立完成时打勾：

概念

- [] 能不借助“就是……”循环定义 30 个核心术语。
- [] 能解释 `volatile/atomic/mutex` 的边界。
- [] 能解释 ISR、线程上下文和原子上下文的限制来源。
- [] 能解释 `epoll` 就绪与业务消息边界的区别。
- [] 能解释 DTS、总线匹配、`probe`、IIO、用户态的完整链路。

代码

- [] 能在五个仓库各指出至少三个关键文件和函数。
- [] 能画出 P1/P2 的 ISR 到任务数据流。
- [] 能画出 P3/P4 的线程停止顺序。
- [] 能画出 P5 的数据从 SPI 到 conditiond 的纵向链路。
- [] 能指出哪些话只有 RELATED/BASE，不能说成已实现。

实验

- [] 能独立运行五个项目 Host 测试。
- [] 能解释至少五个失败注入为何应该失败。
- [] 能读懂 ARM ELF 的架构和动态加载器信息。
- [] 能说明驱动对象编译与模块上板的差距。
- [] 能写出 HIL 记录所需的版本、操作、原始输出和结论。

表达

- [] 每个项目都能讲 30 秒、2 分钟和一次深挖。
- [] 回答中包含场景、机制、证据、取舍和边界。
- [] 遇到未实现问题能诚实迁移，不强行绑定。
- [] 能承受三层“为什么/如果失败/怎么验证”追问。
- [] 能把面试官的问题自然引回最强的代码证据。

11. 随机抽测规则

每次抽 20 题：A级 10 题、B级 6 题、C级 4 题。A级必须说出文件/函数与证据级别；B级必须说出与现有设计的异同；C级必须给出准确基础答案，不需要编造项目经历。连续两轮 85 分以上，再进入模拟面试。

评分：定义 1 分、机制 1 分、项目/边界 1 分、追问 1 分。答非所问或虚构证据该题 0 分。随机题号从 07-question-mapping.md 取，追问从 08-follow-up-tree.md 取。

术语表：用准确语言回答

1. C/C++ 与运行时

中文	English / 缩写	面试中的准确含义
声明	declaration	告诉编译器名字及类型，不一定分配实体存储
定义	definition	提供实体或函数实现；同一程序通常受 ODR 约束
链接属性	linkage	名称在翻译单元之间是否指向同一实体
名字改编	name mangling	C++ 编译器把重载等类型信息编码进符号名
存储期	storage duration	对象存储存在多久：自动、静态、线程、动态
生命周期	lifetime	对象从开始存在到结束存在的语义区间，不完全等同存储期
未定义行为	undefined behavior, UB	标准不规定结果；不能依赖某次“运行正常”
对齐	alignment	对象地址需满足类型/平台的倍数约束
浅拷贝	shallow copy	只复制成员值，指针成员仍指向同一资源
深拷贝	deep copy	为所拥有资源创建独立副本
资源获取即初始化	RAII	用对象生命周期管理内存、fd、锁、线程等资源
右值引用	rvalue reference	T&&；支持移动语义和完美转发，但本身不是“必然移动”
移动语义	move semantics	转移资源所有权，减少昂贵复制；源对象仍须可析构
完美转发	perfect forwarding	保留实参值类别转交给目标函数，常用 <code>std::forward</code>
原子操作	atomic operation	对其他线程表现为不可分割，并受指定内存序约束
先行发生	happens-before	C++ 内存模型中保证一个操作效果对另一个可见的偏序关系
数据竞争	data race	不同步的并发冲突访问且至少一次写；在 C++ 中属于 UB
虚函数表	vtable	常见 ABI 实现机制，不是 C++ 标准强制的具体布局

2. ARM、实时与 FreeRTOS

中文	English / 缩写	面试中的准确含义
中断服务程序	interrupt service routine, ISR	中断入口执行的受限代码，应短小并遵守上下文 API 规则
中断向量表	interrupt vector table	异常/中断号到入口地址的映射结构
嵌套向量中断控制器	NVIC	Cortex-M 中断使能、优先级、挂起和嵌套控制单元
临界区	critical section	访问共享不变量时不可被指定并发源打断的代码区
任务	task	FreeRTOS 的可调度执行单元，概念上类似线程但环境不同
抢占	preemption	调度器暂停当前执行单元，运行更高优先级/更合适的单元
时间片	time slicing	同优先级就绪任务按 Tick 等规则轮换
优先级反转	priority inversion	高优先级等待低优先级持有资源，且低优先级受中优先级干扰
优先级继承	priority inheritance	临时提高锁持有者优先级以缩短反转阻塞，不能解决所有死锁
信号量	semaphore	表示事件或可用数量的同步原语，不一定有持有者所有权
互斥量	mutex	有所有权的互斥原语；通常用于保护共享不变量
队列	queue	在任务/中断之间按既定规则复制和排队消息
最新值邮箱	latest-value mailbox	只关心最新状态的长度 1 覆盖式队列模式
最坏执行时间	WCET	在定义硬件和输入约束下任务执行时间的上界估计
截止期	deadline	工作必须完成的时间约束；实时性关注是否可预测地满足
看门狗	watchdog	在软件失去健康进展时触发复位/告警的独立监控机制

3. Linux 进程、线程与内存

中文	English / 缩写	面试中的准确含义
进程	process	资源隔离与地址空间容器，也是 Linux 调度实体组织的一部分
线程	thread	与同进程线程共享地址空间/多数资源，拥有独立栈与调度状态
系统调用	system call	用户程序请求内核服务的受控入口，不等于普通函数调用
上下文切换	context switch	保存一个执行上下文并恢复另一个；系统调用不一定引发进程切换
虚拟内存	virtual memory	地址转换、隔离、映射、共享和按需调页的整体机制
页表	page table	虚拟页到物理页及权限等属性的映射结构
缺页	page fault	地址访问无法由当前页表直接完成而触发的异常，可正常也可致命
写时复制	copy-on-write, COW	初始共享只读页，写入时再复制，用于 fork 等场景
内存映射	mmap	把文件或匿名内存映射到虚拟地址空间
互斥锁	mutex	用户态线程保护复合共享状态的常用同步原语
条件变量	condition variable	等待“受 mutex 保护的谓词发生变化”的协调机制
虚假唤醒	spurious wakeup	条件变量等待可能在谓词未满足时返回，因此必须循环检查
死锁	deadlock	一组执行单元循环等待彼此持有且不可获得的资源
活锁	livelock	执行单元持续变化和响应但没有实质进展
饥饿	starvation	某执行单元长期得不到运行或资源机会
文件描述符	file descriptor, fd	进程 fd 表中的小整数索引，不是磁盘文件本身
inode	index node	Linux 文件系统中保存对象元数据并关联数据的核心结构
资源限制	rlimit	进程可用 fd、栈等资源的软/硬限制

4. I/O 与网络

中文	English / 缩写	面试中的准确含义
阻塞 I/O	blocking I/O	操作暂不能完成时调用线程可睡眠等待
非阻塞 I/O	non-blocking I/O	操作暂不能完成时立即返回，如 EAGAIN
I/O 多路复用	I/O multiplexing	一个线程等待多个 fd 的就绪事件，如 select/poll/epoll
水平触发	level-triggered, LT	条件持续满足时可反复报告就绪
边沿触发	edge-triggered, ET	主要在就绪状态变化时报告，通常需读写至 EAGAIN
反压	backpressure	下游容量不足时向上传播限速、阻塞、拒绝或丢弃策略
高水位	high-water mark	队列/缓冲达到阈值后启动反压或降级的界线
字节序	endianness	多字节值在内存/协议中字节排列顺序；网络字节序通常为大端
套接字	socket	由 fd 引用的通信端点，具有协议、地址和状态
监听队列	listen backlog	TCP 监听端对待处理连接的内核队列相关上限，不是业务消息队列
半连接	half-open connection	语境可能指握手未完成或一端失效未被另一端察觉，回答时需澄清
MTU	maximum transmission unit	链路层一次可承载的最大网络层包大小
粘包	stream coalescing	俗称 TCP 接收时多个应用消息连在字节流中，根因是无消息边界
半包	partial frame	一次读取只得到应用帧的一部分，需累积后解析
重传超时	retransmission timeout, RTO	TCP 根据往返时间估计用于触发重传的计时参数
指数退避	exponential backoff	失败后按倍数增长重试间隔，并设置上限；常配合 jitter
抖动	jitter	时间间隔变化；退避中的随机扰动可避免多个节点同步重试
PCAP	packet capture	保存抓包数据及时间戳等元信息的常见文件格式

5. Linux 驱动、设备树与 IIO

中文	English / 缩写	面试中的准确含义
内核模块	loadable kernel module, LKM	可动态装载/卸载的内核目标；运行在内核地址空间
设备树	device tree, DT	描述不可枚举硬件实例、连接和资源的层次化数据
设备树源文件	DTS	人类可读的设备树源文本
设备树二进制	DTB	由 DTC 编译、由引导程序传给内核的二进制设备树
compatible	compatible property	表示设备兼容型号，参与总线设备与驱动匹配
探测	probe	匹配后驱动初始化一个设备实例的回调过程
移除	remove	设备解绑/模块退出时撤销注册与资源的回调过程
延迟探测	deferred probe	依赖资源暂不可用时稍后重试 probe 的机制
设备管理资源	devres / devm	与 device 生命周期绑定的自动释放资源机制
寄存器映射	regmap	统一寄存器访问、缓存、锁与调试的内核抽象
工业 I/O	Industrial I/O, IIO	面向 ADC、IMU 等传感器/转换器的 Linux 子系统
通道	IIO channel	某种测量量及其格式、索引、scale 等描述

中文	English / 缩写	面试中的准确含义
扫描元素	scan element	IIO 缓冲中启用通道的数据布局描述
触发器	IIO trigger	发起一轮缓冲采样的事件源
线程化轮询	threaded poll function	IIO 中把可能睡眠的采样工作放在线程上下文的处理方式
高精度定时器	hrtimer	内核高分辨率计时机制；回调上下文不可直接做可睡眠 I/O
工作队列	workqueue	把工作安排到内核工作线程执行的延后机制
上半部/下半部	top half / bottom half	将紧急中断响应与可延后工作分开的传统描述
原子上下文	atomic context	不允许睡眠的执行上下文，不等价于“正在做 atomic 变量操作”
进程上下文	process context	代表某任务执行、通常允许睡眠的内核上下文，仍受锁等约束
sysfs	sysfs	内核对象和设备属性的文本化视图，适合少量控制/状态
ioctl	input/output control	对 fd 发出设备/协议特定控制命令的系统调用接口
用户拷贝	copy_to/from_user	内核安全访问用户地址的接口，返回未复制字节数等语义需核对
主设备号/次设备号	major/minor number	字符/块设备号中分别标识驱动类别与具体实例的传统编号

中文	English / 缩写	面试中的准确含义
U-Boot	Universal Boot Loader	常见嵌入式引导程序，负责早期硬件、镜像加载和启动参数
交叉编译	cross compilation	在宿主平台生成另一目标架构/ABI 的产物
sysroot	system root	交叉工具链编译链接时使用的目标头文件和库根目录

6. 项目和证据术语

标识	全称/含义	使用规则
P1	Handheld Temperature	手持体温检测仪项目
P2	Smart Medicine Cart	智能送药小车项目
P3	3D LiDAR Perception	三维 LiDAR 感知项目
P4	MultiSource Optical Fusion	多源光电数据融合项目
P5	I.MX6ULL Condition Monitor	状态监测器与 IIO 驱动项目
CODE	code evidence	仓库有实现；不自动等于运行通过
HOST	host-verified	宿主测试或实验通过；不证明目标硬件
CROSS	cross-build verified	目标工具链生成产物；不证明板端运行
HIL	hardware-in-the-loop	在记录清楚的真实硬件环境中通过
TOD0-HIL	hardware validation pending	明确待验证，面试和简历不得省略 TOD0
RELATED	related concept	与项目可迁移关联，但当前实现没有直接证据
BASE	foundational knowledge	基础知识，正确回答即可，不强行项目化

7. 容易说错的完整句式

不准确表达	推荐表达
“volatile 防止数据竞争”	“volatile 约束访问优化；线程同步使用 atomic、mutex 或队列。”
“中断里不能睡眠因为优先级高”	“ISR/原子上下文没有可调度睡眠条件，且会影响中断延迟。”
“epoll 是异步 I/O”	“epoll 是就绪通知；实际 read/write 仍由应用执行。”
“一次 read 读一包”	“read 返回当前可得字节数，应用协议必须自己维护帧边界。”
“设备树调用 probe”	“内核根据 DT 创建设备，总线匹配 compatible 后调用驱动 probe。”
“hrtimer 精度高所以里面采 SPI”	“hrtimer 负责触发；SPI 可能睡眠，放在线程化处理。”
“编译过就是驱动验证过”	“已完成指定内核/工具链的 CROSS；加载、波形和数据仍待 HIL。”
“项目用了零拷贝”	“除非能指出具体机制和测量，否则只说减少了哪些复制。”
“无锁一定更快”	“无锁需要正确内存序和竞争模型，是否更快必须测量。”
“Linux 不实时”	“通用 Linux 默认不提供严格硬实时保证；配置、PREEMPT_RT 与负载会改变延迟特征。”

准确表达的目标不是堆术语，而是让面试官能分辨：你知道机制、看过代码、做过验证，也知道结论在哪一步停止。